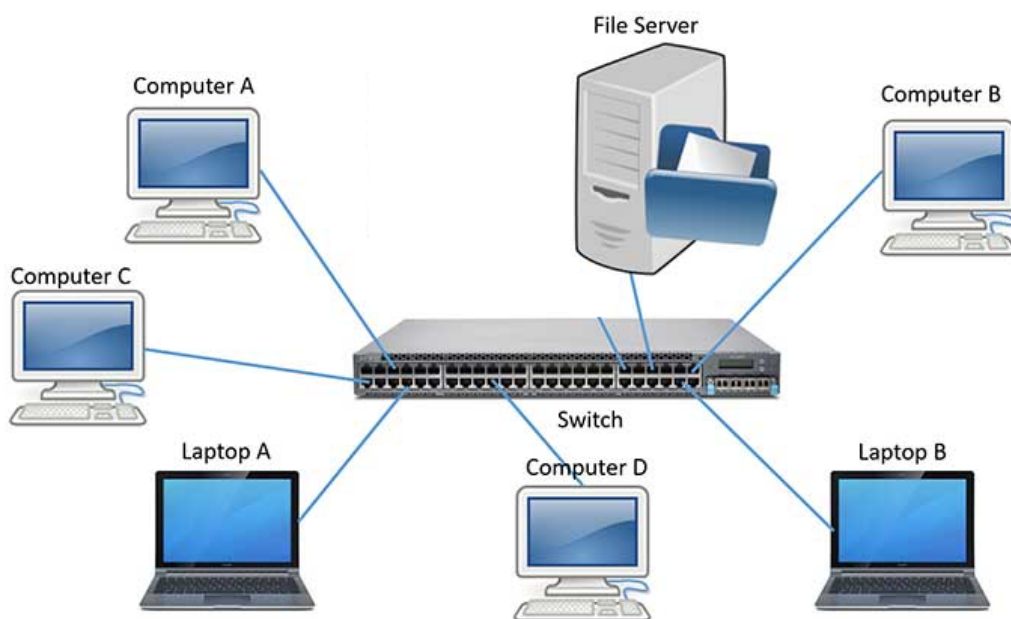


Tema 1: Pojam administracije računarskih sistema

- **Računarski sistem** je veoma generalan pojam; na ovom predmetu, pod njim će se podrazumevati računarska mreža koju čini više računara, koje koristi više korisnika; računari dele (eng. to share) resurse, komuniciraju između sebe, kao i sa računarima iz drugih (udaljenih) računarskih sistema (mreža)
- Primer jednog jednostavnog računarskog sistema dat je na Slici 1-1: šest računara su preko sviča povezani na jedan fajl server; naime, često se u kompanijama praktikuje da zaposleni ne drže fajlove lokalno, već na za to namenjenom (fajl) serveru; na taj način zaposleni lakše mogu raditi na istim fajlovima, a olakšani su bekap i zaštita



Slika 1-1: Primer računarskog sistema

- Pored fajl servera, u sistemu (mreži) se mogu naći i: server baze podataka (eng. database server), server za aplikacije (eng. application server), server za štampanje (eng. print server), i slično
- Prikazani računari, koji su klijenti fajl servera, nazivaju se često i **radne stanice** (eng. workstation -- takođe veoma generalan pojam); takođe, ako je klijent neki veoma slab

računar koji se za sve što je potrebno obraća serveru (znači potpuno nesamostalan), takav klijent se naziva "mršav" (eng. thin client); "najmršaviji" klijent, koji obično nema instaliran ni OS, se naziva "nula" klijent (eng. zero client)

- Prikazana konfiguracija se komplikuje kada se računarski sistem preko rutera poveže na internet: u tom slučaju, sistem može da sadrži i mejl server (koji omogućava korisnicima da šalju i primaju mejlove) i veb server (na kome se hostuje veb stranica kompanije)

- Da jedan računarski sistem ispravno funkcioniše, zadatak je njegovog **administratora** (admina); najbitniji zadaci koje admin obavlja su:

- 1) Upravljanje korisničkim nalogima: dodavanje i brisanje novih korisnika, promena zaboravljene lozinke, određivanje kvota

- 2) Upravljanje radom mrežnog štampača (ili servera za štampanje): upravljanje redom čekanja za štampanje, provera statusa tog reda, brisanje zahteva za štampanjem kada je to neophodno

- 3) Upravljanje radom svih drugih servera (veb, mejl,...)

- 4) Instaliranje, konfigurisanje, apgrejdoвање OS i pomoćnih programa

- 5) Instaliranje korisničkog softvera: on stiže u različitim formatima za instalaciju, kao što su (u slučaju Linuxa) RPM i DEB; preuzimanje softvera koji dolazi u obliku sors-koda, i generisanje izvršnih programa od njega, spremih za instaliranje

- 6) Upravljanje mrežnim hardverom: dodavanje novog hardvera, instaliranje drajvera tako da uređaj ispravno funkcioniše

- 7) Bekapovanje fajlova: da se podaci ne bi izgubili, treba ih redovno bekapovati: na optički disk, na fajl server, na eksterni hard disk, na klaud, i slično

- 8) Praćenje performansi sistema: praćenje zadataka koje procesor izvršava, praćenje količine memorije u sistemu i načina njenog korišćenja

- 9) Praćenje stanja mreže i mrežnih interfejsa

10) Osiguravanje bezbednosti sistema: sistem mora biti uspostavljen tako da su svi fajlovi zaštićeni u slučaju napada preko mreže; praćenje bezbedonosnih pretnji proveravanjem log fajlova

11) Vođenje dokumentacije o samom sistemu i o svom radu

- Admin može sebi olakšati posao tako što će da automatizuje zadatke koje treba redovno da obavlja: može da podesi OS tako da određeni zadatak izvršava u određeno vreme, a može da piše i kratke programe -- skripte; za to je Linux posebno pogodan; računarski sistem pod Linux OS na ovom predmetu biće nazivan "Linux sistem"

- Linux OS je najkorišćeniji OS za računarske sisteme; stoga će mu ceo predmet biti posvećen; bez ulaska u sitne detalje Linuxa, predmet ima za cilj da korisniku pruži širu sliku i predstavi mu osnove, i osposobi ga da se sâm snalazi kada mu to bude potrebno, za šta bude potrebno

Tema 2: O Linuxu

- Linux OS je na prvom mestu namenjen da se koristi sa komandne linije (eng. command-line interface, CLI); zbog toga će biti predstavljene najkorisnije komande za obavljanje raznih zadataka; Linux pruža mogućnost rada i pomoću grafičkog korisničkog interfejsa (graphic user interface, GUI), u okruženjima kao što su GNOME ili KDE, ali će rad sa GUI biti samo pomenut

- Razlog za insistiranje na radu sa komandne linije je: CLI pruža mnogo više mogućnosti i fleksibilnosti, u kojima zapravo leži moć Linuxa; mnoge zadatke iz GUI nije ni moguće obaviti

2.1 Šta je Linux?

- Linux je popularan besplatan operativni sistem (OS) otvorenog koda (eng. open source); on se, kao što je pomenuto, može koristiti na dva načina:
1) pomoću GUI, koji se zasniva na prozorima, ikonama, i kontroli pomoću miša (kao kod Windows OS), što je običnom korisniku računara lakše i intuitivnije
2) pomoću CLI, koji se naziva i šel (eng. shell, bez srpskog prevoda), koji predstavlja prozor u kome korisnik kuca komande, i u kome se one izvršavaju i prikazuje rezultat izvršenja
- Kao i Linux, i drugi popularni OS (kao što su Windows OS i MacOS) takođe mogu biti korišćeni pomoću CLI; kod svih OS, većina običnih korisnika može da odradi sve što je potrebno i pomoću GUI, ali je za administraciju CLI podrazumevan
- Idejni tvorac Linuxa je Finac Linus Torvalds (1991); baziran je na jednom starijem operativnom sistemu, Unixu (u slengu se kaže da je Linux "Unix-like"); tokom decenija, Linux je razvijala i dorađivala ogromna zajednica korisnika; ima reputaciju da je veoma stabilan; maskota mu je pingvin Tux (Slika 2-1)



Slika 2-1: Maskota Linuxa

- Android OS je baziran na Linuxu; Linux je vodeći OS na svetu za servere (96% veb servera je bazirano na njemu), mejnfrejmove i superračunare; takođe, koristi se intenzivno i u namenskim računarskim sistemima (eng. embedded systems) kao što su automobili, ruteri, smart televizori i satovi i konzole za video-igre

- Operativni sistem za Apple računare, MacOS, je takođe Unix-like, tako da su MacOS i Linux prilično bliski; čak je i operativni sistem za Apple smartfone i tablete, iOS, takođe Unix-like
- Mana Linuxa je sledeća: usled toga što je razvijan necentralizovano (to jest ne od strane jedne kompanije, od početka do kraja, kao na primer Windows), postoji mnogo nelogičnosti u komandama, često nema konzistentnosti, ima dosta preklapanja (na primer dve različite komande obavljaju sličan posao; davno zastarele komande se ne izbacuju)
- Čak je i Microsoft "prihvatio" konkurentni Linux, tako što je kreirao Windows Subsystem for Linux (WSL): ovaj alat omogućava korisnicima da direktno iz Windowsa (bez potrebe za virtualnom mašinom ili dualnim butovanjem) rade u Linux okruženju:
<https://docs.microsoft.com/en-us/windows/wsl/>

2.2 Šta je Linux distribucija?

- Linux se može konfigurisati po želji; sadrži na hiljade komandi, koje je razvijala ogromna zajednica korisnika; kao rezultat, nastao je veliki broj varijanti Linuxa, koji pokrivaju razne korisničke potrebe i ukuse, neke su besplatne a neke ne
- Pomenute varijante imaju istu osnovu, ali im je "nadgradnja" različita, i izgledaju različito; ove varijante se nazivaju **distribucije**, ili skraćeno distro; popularne distribucije su: Ubuntu Linux, Red Hat Enterprise Linux, Slackware, OpenSUSE, Mint; materijal u ovom dokumentu je univerzalan, to jest nezavisan od distribucije

2.3 Šta je komanda?

- Jedna Linux komanda se tipično sastoji od: naziva **programa** (koji će se zadavanjem komande izvršiti), za kojim slede **opcije** (obično označene crticom) -- preciznije: koje opcije

je korisnik odabrao od svih koje su na raspolaganju, za kojima slede **argumenti** koji su zadati programu da sa njima radi; komanda se piše, kao što je već pomenuto, u šelu

- Znači, generalna sintaksa jedne Linux komande je:

program options arguments

- Primer: komanda koja služi da izbroji linije u nekom tekstualnom fajlu je:

wc -l myfile

- U prethodnom primeru:

>> **wc** (skraćenica od "word count" = brojač reči) je naziv programa (koji se može posmatrati i kao pandan funkciji u programiranju) koji se nalazi negde na disku, i koga će šel da pronađe i izvrši

>> **-l** (skraćenica od "lines") je opcija koja nalaže da se broje linije/redovi

>> **myfile** je argument koji nalaže koji tekstualni fajl treba da se obradi, to jest naziv fajla u kome treba da se izbroje linije

Kao rezultat izvršenja komande, biće ispisana odgovarajuća brojna vrednost

- Komande mogu imati i više opcija i argumenata; opcije mogu biti zadate svaka posebno, kao na primer:

wc -l -w myfile

Two individual options

Opcijom **-w** zahteva se da se ispiše broj reči

- Opcije mogu biti zadate i zajedno, posle jedne zajedničke crtice:

wc -lw myfile

Same as -l -w

Neki programi ne prihvataju ovakav način zadavanja, tako da nije preporučljiv

- Primer upotrebe funkcije sa dva argumenta:

`wc -l myfile myfile2`

Count lines in two files

Ovde komanda broji linije u dva zadata tekstualna fajla

- Opcije nisu standardizovane: mogu biti formata crtica pa slovo (na primer, `-l`), ali i formata dve crtice pa reč (na primer, `--lines`)
- Takođe, jedno isto slovo može imati različito značenje u različitim programima; na primer, u komandi `wc -l`, opcija `-l` označava "linije teksta", dok u komandi `ls -l` opcija `-l` označava "duži izlaz" = "longer output", to jest detaljniji prikaz
- Ni argumenti nisu standardizovani: oni mogu predstavljati nazive fajlova za ulaz i izlaz (kao u prethodnom primeru), ali mogu biti i nešto sasvim drugo, kao na primer nazive foldera ili regularnih izraza
- Komande mogu biti i složenije strukture nego što je prikazano, to jest složenije od sekvence "program opcije argumenti"; tipični primeri za to su sekvenciranje i ulančavanje komandi, i programiranje u šelu (više detalja u nastavku)
- Naime, jedna komanda može da pokrene više programa u isto vreme: ili u **sekvenci** (prvo jedan program, pa drugi i tako redom), ili u **lancu**, to jest ulančano (eng. pipeline -- bukvalni prevod bi glasio "cevovod", od eng. pipe = cev; pipeline se ovde prevodi kao lanac; a glagol to pipe kao ulančati), gde izlaz jednog programa postaje ulaz narednog
- Linux CLI, to jest šel, u sebi sadrži i jedan jednostavni programski jezik, što znači da korisnik u Linuxu može i programirati; na primer, umesto da komanda zada "izvrši program X", ona može da zada "ako je danas utorak, izvrši program X; inače, izvrši program Y, šest puta za svaki fajl čija je ekstenzija txt"

Šel prompt

- Kada kuca komandu u šelu, korisnik to radi na mestu gde je to predviđeno, i koje Linux označava posebnim simbolom koji se naziva **prompt**; pomoću prompta znači, Linux korisniku saopštava "kucaj ovde svoju komandu koju ću ja izvršiti"

- Simboli za prompt su raznoliki, zavisno od toga kako je šel podešen; prompt može biti jednostavni simbol za dolar:

\$

a može biti i složeni string koji sadrži ime računara, korisničko ime, i slično, na primer:

myhost:~smith\$

- Na ovom predmetu, za prompt će se koristiti generička oznaka: strelica nadesno →, tako će na primer kompletna linija u šelu biti napisana ovako:

→ **wc -l myfile**

- Linije teksta koje se ispisuju kao rezultat izvršenja komande naravno ne sadrže prompt; primer:

→ wc -l myfile	<i>The command you type</i>
18 myfile	<i>The output it produces</i>

- Neke komande može da izvrši samo superkorisnik, a ostali (obični) korisnici to ne mogu; superkorisnik je specijalni korisnik (odgovarajuće korisničko ime je root) koji ima dozvolu da na sistemu radi sve što je raspoloživo; u tom slučaju, ispred željene komande treba napisati komandu **sudo**:

→ **sudo command**

- Više detalja o superkorisniku biće dato u nastavku; za sad treba zapamtiti ovu komandu, kao i to da će ona tražiti i lozinku; sistem mora biti konfigurisan da **sudo** komandu prihvati

- Na primer, da bi se izbrojale linije u zaštićenom fajlu `/etc/shadow`, bez komande **sudo** to neće biti moguće:

<code>→ wc -l /etc/shadow</code>	<i>This will fail</i>
<code>wc: /etc/shadow: Permission denied</code>	
<code>→ sudo wc -l /etc/shadow</code>	<i>Now with sudo</i>
<code>Password: *****</code>	
<code>51 /etc/shadow</code>	<i>It worked!</i>

2.4 Prikazivanje komandi

- Na ovom predmetu biće predstavljeno mnogo Linux komandi; objašnjenje svake od njih zapoćeće standardizovanim naslovom koji prikazuje njenu sintaksu; primer naslova za komandu `ls` ("list files" -- daje listu svih fajlova u datom direktorijumu, eng. directory, sinonim za folder) je:

`ls [options] [files]`

- Gornji tekst znači da korisnik treba da otuku `ls`, posle koga mogu slediti opcije, za kojima mogu slediti nazivi fajlova kao argumenti; naime, postoje dve konvencije kao što sledi
- Konvencija 1: Uglaste zagrade `[ ]` (koje korisnik naravno ne kuca, već služe kao indikator) označavaju da se tekst između njih može pisati ali i ne mora; na primer, korisnik može da odabere pet opcija, ali može i da ne odabere nijednu -- u tom slučaju ide difolt opcija; ako je neka stavka napisana bez uglastih zagrada, znači da je obavezna
- Konvencija 2: Tekst u *kurzivu/italiku* označava da korisnik treba da upiše svoje vrednosti sa kojima radi; na primer, ako piše *files* (kao u primeru), on treba da upiše konkretni naziv svog fajla (u gornjem primeru to nije obavezno, jer stoje uglaste zagrade)

- Vertikalna crta | (na engleskom se ovaj karakter naziva pipe = cev; na srpskom se ovaj karakter naziva i "kanal" ali se ovde neće koristiti), koja se piše između opcija ili između argumenata, sve u običnoj zagradi, u ovom kontekstu označava da se može raditi sa jednim ili sa drugim

- Na primer, zapis:

(file | directory)

označava da korisnik može kao argument upisati ili naziv fajla ili naziv direktorijuma

Specijalni tasteri

- U određenom kontekstu, kret karakter ^ (eng. caret, srpski prevod bi mogao biti "kapica" ali se ne koristi) služi da označi pritisnuti Control (Ctrl) taster; tako na primer ^D (čita se "kontrol D") znači "pritisni i drži taster Control i zatim taster D"; u istom kontekstu, ESC označava pritisnuti Escape taster

Komanda echo

- Jednostavna komanda **echo** ispisuje svoj argument na ekranu; primer:

→ **echo My dog has fleas**

My dog has fleas

→ **echo My name is \$USER** *Shell variable USER*

My name is smith

Očigledno, sav tekst koji se kuca posle komande se tretira kao jedan argument; u drugoj komandi, **USER** je jedna šel varijabla, i sadrži ime korisnika; više o njima u nastavku

Dugačke komande

- Nekada se desi da korisnik zada neku dugačku komandu, koja je duža od širine ekrana u šelu; u tom slučaju, može da koristi beksleš \ karakter: on u šelu, u ovom kontekstu, ima

značenje "line continuation" karaktera; na taj način može da napiše komandu u više linija, radi bolje preglednosti

- Primer:

```
→ echo This is a long command that doesn't fit on \
    one line
This is a long command that doesn't fit on one line
```

Korisnik znači otuče prvu liniju za bekslešom na kraju, pritisne Enter, posle čega šel premești prompt u naredni red, gde korisnik kuca nastavak linije

2.5 Korišćenje helpa

- Ako su korisniku potrebne detaljnije informacije ("help") o Linux komandama (to jest programima), do njih može doći na više načina, koji će ovde biti redom objašnjeni; korisnik naravno može pomoć naći, u obliku primera ili pitanja i odgovora, i na raznim veb sajtovima posvećenim Linuxu (na primer, www.linuxize.com, www.linuxquestions.org)

Upotreba man komande

- Komanda **man** ("manual") ispisuje onlajn manual stranicu (man page = manpage) o željenoj komandi (programu); na primer:

```
→ man wc
```

Postoji još jedna komanda slična ovoj, a to je **info**

- Za pretragu manual stranica za željeni pojam, koristi se opcija **-k** za kojom sledi željena ključna reč (keyword), na primer:

```
→ man -k database | less
```

- Ovo je jednostavan primer pomenutog ulančavanja; za njega se koristi već pomenuta vertikalna crta (ali ovde u novom kontekstu): izlaz komande **man** (tekst) postaje ulaz komande **less**, koja diktira da se rezultati prikazuju ekran po ekran (i ne šire nego ekran), dok za prelazak na novi ekran korisnik treba da pritisne taster Space (ili Q da prekine)
- Na ovakav način napravljen je znači lanac komandi koje se izvršavaju redom sleva na desno: prva drugoj "predaje" svoj rezultat na dalju obradu; tih komandi u lancu može biti proizvoljno mnogo

Upotreba opcije **--help** (tamo gde postoji)

- Neke Linux komande odgovaraju na opciju **--help** (pišu se dve crtice) tako što ispisuju na ekranu kratku informaciju o komandi; primer:

→ **wc --help**

Pregled direktorijuma **/usr/share/doc**

- Direktorijum **/usr/share/doc** sadrži dokumentaciju o raznim programima, obično organizovanu po nazivima programa i njihovoj verziji; na primer, dokumentacija za editor teksta emacs, verzija 24, se obično nalazi u direktorijumu **/usr/share/doc/emacs24**

Pregled sajta specifične distribucije

- Svaka veća Linux distribucija ima svoj zvanični sajt (kao na primer <https://ubuntu.com>), koji sadrži dokumentaciju, forume za odgovore i pitanja, i slično

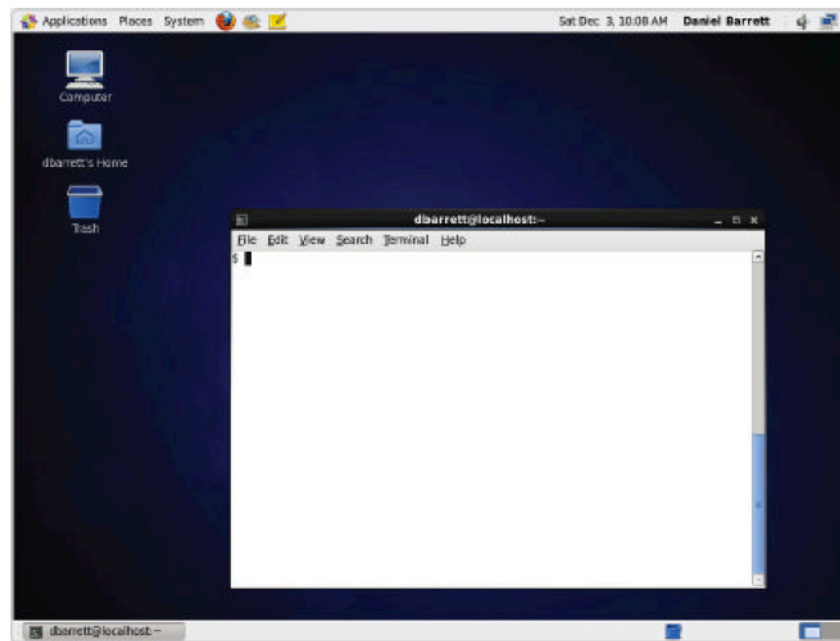
Tema 3: Pregled Linuxa

- Linux se sadrži od četiri osnovne komponente: kernel, namenski programi, šel i X

- **Kernel** je jezgro Linuxa, napisano u jeziku C, koje je (primitivan) OS sâm po sebi; kernel upravlja radom sa fajlovima, diskovima, mrežama i slično; tipični korisnici Linuxa vrlo retko i dolaze u dodir sa kernelom
- **Namenski programi** (pokušaj prevoda eng. supplied programs = dolaze uz Linux) su hiljade programa (to jest, kao što je objašnjeno, komandi) koji obezbeđuju širok spektar funkcionalnosti, kao na primer: rad sa fajlovima, editovanje teksta, matematika, brauzovanje veba, audio, video, programiranje, enkripcija, i mnogo toga drugog
- **Šel** je, kao što je pomenuto, korisnički interfejs (CLI) u kome korisnik kuca komande, i u kome vidi rezultate njihovog izvršenja; Linux poseduje nekoliko popularnih šelova; na ovom predmetu radiće se u onom koji se naziva **bash** (skraćeno od Bourne-Again Shell), koji je često i difolt šel; drugi šelovi (Bourne, Korn, C) slično izgledaju i funkcionišu
- **X** je grafički sistem (čudnog imena) koji omogućava prozore, menije, ikone, kontrolu mišem, i sve ostalo što je vezano za GUI; na njemu su bazirana grafička okruženja kao što su pomenuti KDE i GNOME; tipični korisnici ne dolaze u dodir sa X sistemom

3.1 Grafički desktop

- Kada se korisnik uloguje na Linux sistem, otvoriće mu se grafički desktop nalik onom na Slici 3-1; ako se uloguje daljinski preko mreže, videće samo komandni prompt
- Grafički desktop sadrži elemente koji su korisniku već poznati iz Windows OS: glavni meni ili taskbar, ikone, prozore koje korisnik može otvarati/zatvarati, spuštati/podizati i slično, sve jednostavno za korišćenje čak i totalnom početniku



Slika 3-1: GNOME grafički desktop sa otvorenim šelom

3.2 Pokretanje šela

- Ikone i meniji u GNOME ili KDE omogućavaju jednostavan i intuitivan način rada; tako korisnik pomoću njih jednostavno može da otvori aplikaciju za mejl, ili brauzer; može obaviti i neka podešavanja sistema (kao na primer instaliranje štampača); ali moć Linuxa leži "ispod" GUI, u šelu; korisnik se brzo navikne na kucanje komandi
- Da bi se pokrenuo šel iz GNOME ili KDE, potrebno je otvoriti šel prozor; za to je potrebno da korisnik pronađe u meniju ili klikne na ikonu i tako pokrene aplikaciju koja se (u zavisnosti od distribucije) zove Terminal (tako se zove i u MacOS), ili xterm, ili Konsole, ili tome slično
- Treba voditi računa i o tome da ni prozor koji se tom prilikom pojavi, formalno rečeno, nije šel (iako ih korisnici uvek poistovećuju); taj prozor je grafički program/aplikacija (neki dolaze sa raznim dodacima, na primer menijima, za lakši rad) u kome se šel izvršava za korisnika

- Drugim rečima, šel je program koji se izvršava u pozadini: njemu korisnik zada komande, šel ih izvrši i prikaže rezultat, a pomenuta aplikacija (Terminal i ostale) u tome služi kao "posrednik"; na ovom predmetu se o takvim detaljima neće voditi računa

3.3 Ulaz i izlaz

- Većina Linux komandi prihvataju od korisnika neke ulazne podatke, i generišu izlazne; ulazni podaci se mogu preuzimati iz fajlova ili sa standardnog ulaza (eng. standard input, skraćeno stdin) -- radi se o složenom Linux konceptu; u najprostijem slučaju to je tastatura
- Izlazni podaci se mogu upisivati u fajlove, ili na standardni izlaz (eng. standard output, skraćeno stdout); u najprostijem slučaju to je ekran, preciznije šel prozor koji je na njemu otvoren
- Poruke o greškama (eng. error messages) se u Linuxu tretiraju posebno, i prikazuju na standardnoj grešci (rogobatan pojam, prevod eng. standard error, skraćeno stderr), što je u najprostijem slučaju takođe ekran

3.4 Korisnici i superkorisnici

- Linux je višekorisnički (eng. multiuser) OS: više korisnika može da istovremeno koristi jedan Linux računar; na primer, procesor će obavljati izračunavanja prvo za jednog korisnika, pa za drugog, i tako redom
- Na jednom Linux računaru, svaki korisnik koji mu pristupa ima svoje jedinstveno korisničko ime = ime korisnika (eng. username); takođe, svaki od njih ima svoj "privatni" deo sistema na kome može da radi nezavisno od drugih (na primer, ima svoje direktorijume i fajlove na disku koje drugi korisnici ne vide)

- Kao što je pomenuto, postoji i specijalni korisnik koji se naziva superkorisnik, koji ima privilegije za totalni pristup svemu što je dostupno na tom računaru
- Obični korisnici imaju privilegije koje su im date, ali uz dosta ograničenja (eng. restrictions): oni obično mogu da pokrenu i neke instalirane programe, ali mogu da pristupaju samo svojim fajlovima (tuđe fajlove ne vide); superkorisnik može da pokreće svaki program, i da vidi, briše i modifikuje sve fajlove na Linux sistemu

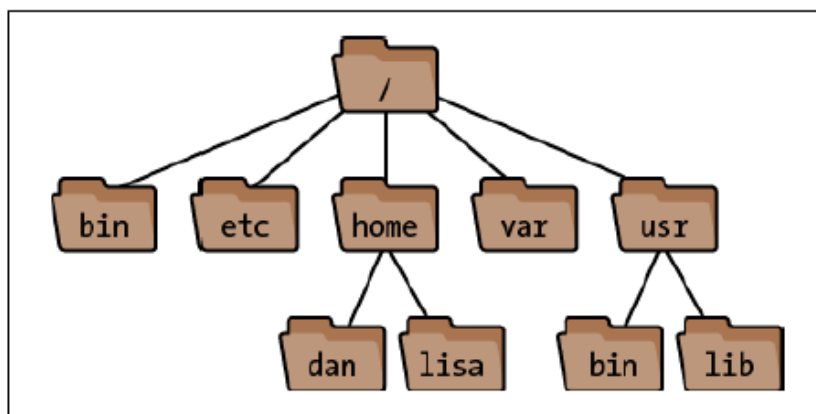
Tema 4: Fajlsistem

- **Fajlsistem** (eng. filesystem ili file system) je jedan složeni i višeznačni koncept u računarstvu, a i u Linuxu; u najprostijem smislu, on označava skup podataka organizovanih po fajlovima i direktorijumima u hijerarhijskom poretku; biće objašnjen u narednoj sekciji

4.1 Osnovni pojmovi fajlsistema

- Rad sa fajlovima i direktorijumima (to jest folderima = skupovima fajlova) je u CLI (to jest u šelu) manje intuitivan u odnosu na GUI, jer korisnik, na primer, ne može "vizuelno" da proveriti gde se "nalazi" u hijerarhiji direktorijuma i fajlova, ne može da klikom promeni direktorijum, ili da iskopira fajl
- Za pomenuti rad, korisnik koristi šel komande kao što su na primer **cd** i **pwd** (promena radnog direktorijuma i ispisivanje njegovog naziva -- više detalja u nastavku)
- Pre prelaska na pregled fajlsistema, dve bitne napomene koje se tiču fajlova i direktorijuma: prvo, Linux ne koristi ekstenzije u nazivima fajlova; ako ekstenzija i postoji, Linux je ignoriše; da bi ustanovio o kom se tipu fajla radi, Linux "zaviri" u njegov sadržaj (preciznije, pregleda prvih nekoliko bajtova, gde se nalazi tražena informacija)

- Sa druge strane, ekstenzija je praktična jer može biti od koristi korisniku, da odmah zna o kakvom se tipu fajla radi; uz to, na raspolaganju mu je i komanda **file** (više detalja u nastavku), koja za zadati fajl daje obaveštenje o tipu
- Druga napomena: za Linux je i direktorijum formalno jedan jednostavni fajl, u kome su (prema određenom standardu) samo ispisana imena drugih fajlova (koje taj direktorijum sadrži); ako se komandi **file** zada naziv nekog direktorijuma, ona će kao izlaz vratiti `directory`
- U Linuxu, na vrhu cele hijerarhije nalazi se direktorijum **root** (isti naziv kao za korisničko ime superkorisnika, što je zbunjujuće); ovaj direktorijum se označava slešom `/`; direktorijumi i fajlovi formiraju stablo (eng. *tree*), to jest hijerarhiju (u kojoj je root znači uvek na vrhu) -- primer je dat na Slici 4-1



Slika 4-1: Primer hijerarhije direktorijuma (fajlovi nisu prikazani)

- Kao i u Windowsu, ako direktorijum one direktno sadrži direktorijum two, kaže se da je two **poddirektorijum** (eng. *subdirectory*) od one; obrnuto, kaže se da je one roditeljski direktorijum (eng. *parent directory*) od two -- i engleski termin i srpski prevod su nezgodni pa se ovde neće koristiti; najlogičnije je prevesti kao **naddirektorijum** (kada se već ne koristi termin *child directory*)

- Pozicija direktorijuma ili fajla u hijerarhiji označava/definiše se **putanjom** (eng. path); na primer, putanja:

/one/two/three/four

označava da se u direktorijumu root (prvi znak /) nalazi direktorijum one, u kome se nalazi two, u kome se nalazi three, u kome se nalazi four; obratiti pažnju da se znak sleš koristi i da odvoji foldere u okviru hijerarhije (što takođe može itekako da zbuni)

- Ako putanja počinje root direktorijumom, za nju se kaže da je **apsolutna** (to jest kompletna, od vrha pa nadole u hijerarhiji, dokle treba); na primer, apsolutna putanja direktorijuma dan sa Slike 4-1 je: /home/dan; u suprotnom, kaže se da je **relativna**

- Prilikom rada u šelu, jedan direktorijum je uvek odabran i definisan (zadat) kao aktuelni, to jest aktivni, to jest **radni** (eng. current working directory); kada se zadaje relativna putanja, smatra se da ona važi relativno u odnosu na radni direktorijum (on je referenca)

- Na primer, ako je radni direktorijum /one/two/three, i ako se zada komanda koja ima kao argument myfile, ta komanda će raditi sa fajlom /one/two/three/myfile; slično, ako se nekoj komandi zada kao argument direktorijum four/five, ta komanda će raditi sa direktorijumom /one/two/three/four/five

- Dva specijalna direktorijuma su . (jedna tačka) i .. (dve tačke); to su zapravo simboli koji se koriste za skraćeni zapis: prvi označava radni direktorijum, a drugi označava njegov naddirektorijum

- Na primer, ako je radni direktorijum /one/two/three onda:

1) simbol . označava taj direktorijum, pa tako putanja /one/two/three/four može kraće da se zapiše kao ./four

2) simbol .. označava direktorijum /one/two, pa tako putanja /one/two/three može kraće da se zapiše kao ../three

- Radni direktorijum se menja pomoću komande **cd** ("change directory"); na primer:

→ **cd /usr/local/bin**

Ovde je radni direktorijum zadat pomoću apsolutne putanje

- Ako se ovoj komandi zada simbol **..** kao argument, naddirektorijum radnog direktorijuma biće proglašen za novi radni direktorijum ("podiže se" za jedan nivo u hijerarhiji fajlsistema); ako se zada simbol **.** neće se desiti ništa

- Radni direktorijum se može zadati i pomoću relativne putanje, kao na primer:

→ **cd d**

→ **cd ../mydir**

U prvoj liniji, za radni direktorijum se proglašava poddirektorijum **d** radnog direktorijuma; u drugoj, proglašava se poddirektorijum **mydir** naddirektorijuma radnog direktorijuma

- Nazivi direktorijuma i fajlova mogu sadržati (velika i mala) slova, cifre i specijalne karaktere (to su svi oni koji nisu slova ili cifre); treba imati u vidu da su u Linuxu svi nazivi **case-sensitive**, to jest da se pravi razlika između malih i velikih slova (**one** i **One** nije isto)

- Sa druge strane, **preporuka** je da se za nazive direktorijuma i fajlova koriste samo mala slova, cifre i donje crte (**_**, eng. underscore), što važi i za sve druge nazive u računarstvu (i sasvim je dovoljno); naziv uvek treba da počne slovom; srpska latinična slova, i ćirilicu treba takođe izbegavati

- Jedan razlog za ovu preporuku: većina specijalnih karaktera (razmak, zvezdica, dolar, zagrade...) ima u šelu specijalno značenje; kad bi se oni koristili u nazivima direktorijuma, morali bi da se "eskejpuju" (više detalja u nastavku) što bespotrebno komplikuje rad

4.2 Home direktorijum

- Svaki korisnik ima svoj **home** ("lični" – koristiće se engleski termin) direktorijum, u kome se nalaze svi njegovi fajlovi (koje drugi korisnici ne vide); home direktorijumi svih korisnika se obično nalaze u direktorijumu /home (mada mogu biti i u /Users); fajlovi superkorisnika nalaze se u direktorijumu /root

- Preciznije, home direktorijum korisnika je tipično /home/korisnicko_ime, kao na primer /home/lisa (Slika 4-1); korisnik je naravno slobodan da svoje fajlove i direktorijume, u okviru svog home direktorijuma, organizuje kako njemu odgovara

- Postoje tri načina da se home direktorijum korisnika postavi za radni direktorijum: prvi, trivijalni način je da se otkuca:

cd

Kada se upotrebi bez argumenata, komanda **cd** obavlja navedeni zadatak

- Drugi način da se zadatak odradi je pomoću **HOME** šel varijable (više detalja u nastavku): ova varijabla sadrži naziv home direktorijuma korisnika; primer:

→ **echo \$HOME** *The echo command prints its arguments*
/home/smith

Pošto korisnik na ovaj način sazna koji mu je home direktorijum, ostaje mu da primeni komandu **cd**

- Treći način je pomoću specijalnog karaktera ~ (tilda): ona predstavlja simbol (skraćeni zapis) korisnikovog home direktorijuma:

→ **echo ~**
/home/smith

4.3 Sistemski direktorijumi

- Tipični Linux sistem sadrži ogroman broj sistemskih direktorijuma: oni sadrže fajlove vezane za sâm operativni sistem, kao i za aplikacije i dokumentaciju, odnosno sve one fajlove koji nisu lični fajlovi korisnika sistema (i koji se nalaze u /home)
- Sistemskim direktorijumima, vrlo pažljivo, uglavnom pristupa samo administrator sistema (superkorisnik); obični korisnik treba da poseduje elementarna znanja, jer mu ponekad mogu zatrebati (na primer, ako traži dokumentaciju)
- Nazivi sistemskih direktorijuma obično se sastoje iz tri dela (koji su u nazivu odvojeni kosim crtama), koji se nazivaju: domen (eng. scope), kategorija (eng. category) i aplikacija (eng. application)
- Na primer, sistemski direktorijum /usr/local/share/emacs, koji sadrži podatke vezane za emacs editor teksta, ima:
 - 1) domen: /usr/local
 - 2) kategoriju: share
 - 3) aplikaciju: emacs (vidi se po nazivu da se radi o programu emacs)Ovo je prikazano grafički na Slici 4-2



Slika 4-2: Domen, kategorija i aplikacija na primeru jednog direktorijuma

Kategorija

- Kategorija nekog direktorijuma govori o tipu fajlova koje on sadrži; na primer, ako je kategorija bin, to označava da sadrži programe; najčešće kategorije date su u narednoj listi

Categories for programs

<i>bin</i>	Programs (usually binary files)
<i>sbin</i>	Programs (usually binary files) intended to be run by the superuser
<i>lib</i>	Libraries of code used by programs

Categories for documentation

<i>doc</i>	Documentation
<i>info</i>	Documentation files for emacs's built-in help system
<i>man</i>	Documentation files (manual pages) displayed by the <code>man</code> program; the files are often compressed and are sprinkled with typesetting commands for <code>man</code> to interpret
<i>share</i>	Program-specific files, such as examples and installation instructions

Categories for configuration

<i>etc</i>	Configuration files for the system (and other miscellaneous stuff)
<i>init.d</i>	Configuration files for booting Linux
<i>rc.d</i>	Configuration files for booting Linux; also <i>rc1.d</i> , <i>rc2.d</i> , ...

Categories for programming

<i>include</i>	Header files for programming
<i>src</i>	Source code for programs

Categories for web files

<i>cgi-bin</i>	Scripts/programs that run on web pages
<i>html</i>	Web pages
<i>public_html</i>	Web pages, typically in users' home directories
<i>www</i>	Web pages

Categories for display

<i>fonts</i>	Fonts (surprise!)
<i>X11</i>	X window system files

Categories for hardware

<i>dev</i>	Device files for interfacing with disks and other hardware
<i>media</i>	Mount points: directories that provide access to disks
<i>mnt</i>	Mount points: directories that provide access to disks

Categories for runtime files

<i>var</i>	Files specific to this computer, created and updated as the computer runs
<i>lock</i>	Lock files, created by programs to say, "I am running"; the existence of a lock file may prevent another program, or another instance of the same program, from running or performing an action
<i>log</i>	Log files that track important system events, containing error, warning, and informational messages
<i>mail</i>	Mailboxes for incoming mail
<i>run</i>	PID files, which contain the IDs of running processes; these files are often consulted to track or kill particular processes
<i>spool</i>	Files queued or in transit, such as outgoing email, print jobs, and scheduled jobs
<i>tmp</i>	Temporary storage for programs and/or people to use

Domen

- Domen u putanji nekog direktorijuma označava njegovu namenu; najčešći domeni su:

<code>/</code>	System files supplied with Linux (pronounced "root")
<code>/usr</code>	More system files supplied with Linux (pronounced "user")
<code>/usr/local</code>	System files developed "locally," either for your organization or your individual computer
<code>/usr/games</code>	Games (surprise!)

- Na primer, za kategoriju lib (library = biblioteka), u sistemu mogu postojati direktorijumi kao što su `/lib`, `/usr/lib`, `/usr/local/lib`, `/usr/games/lib`, i slično

- Još nekoliko napomena:

>> jasna razlika između `/` i `/usr` ne postoji, ali se smatra da je `/` nižeg nivoa, to jest bliži kernelu

>> `/bin` sadrži fundamentalnije programe (kao što su `ls` i `cat`) od `/usr/bin`

>> `/lib` sadrži fundamentalnije biblioteke od `/usr/lib`

>> `/usr/local/bin` sadrži programe koji nisu uključeni u distribuciju

Aplikacija

- Deo putanje koji se naziva aplikacija, tamo gde postoji, obično nosi naziv programa; u ovom direktorijumu nalaze se fajlovi vezani za taj program (mogu biti raspoređeni u više poddirektorijuma)

4.4 Kernel direktorijumi

- Neki direktorijumi su vezani za Linux kernel, to jest "jezgro" Linuxa, kao na primer:

>> `/boot`: fajlovi potrebni za butovanje (eng. booting, to jest podizanje) sistema; ovde je smešten i sam kernel, tipično u direktorijumu `/boot/vmlinuz`

>> `/lost+found`: oštećeni fajlovi koje je tu smestio disk recovery tool

>> `/proc`: opisuje procese koji su u toku izvršenja

- Fajlove koji se nalaze u /proc uglavnom koriste programi, kao na primer:

`/proc/ioports` A list of your computer's input/output hardware.

`/proc/cpuinfo` Information about your computer's processors.

`/proc/version` The operating system version. The `uname` command prints the same information.

`/proc/uptime` System uptime (i.e., seconds elapsed since the system was last booted). Run the `uptime` command for a more human-readable result.

`/proc/nnn` Information about the Linux process with ID *nnn*, where *nnn* is a positive integer.

4.5 Kontrola pristupa fajlovima

- Linux sistem može da ima veći broj korisnika koji imaju svoje korisničke naloge za login; svakom korisniku treba da bude obezbeđena privatnost i sigurnost; svaki od njih treba da ima pristup (eng. access) nekim fajlovima u sistemu, nikad svima
- Kontrola pristupa (eng. access control), za dati fajl ili direktorijum, se zasniva na dva pitanja: ko ima dozvolu za pristup, i koju vrstu pristupa?

Ko ima dozvolu?

- Za svaki fajl (isto važi i za direktorijume) postoji jedan korisnik koji je njegov **vlasnik** (eng. owner); naravno, vlasnik ima dozvolu za pristup "svom" fajlu; tipično, korisnik koji je kreirao fajl je njegov vlasnik (ali superkorisnik ima privilegiju i da promeni vlasnika)

- Uz to, vlasnik fajla je obavezno i jedna **grupa** korisnika, koja takođe ima pristup "svom" fajlu (to jest, pristup ima svaki član grupe); grupe formira i njima upravlja admin (više detalja u nastavku)

- Konačno, za neke fajlove (i direktorijume), admin dozvolu za pristup može dati **svim ostalim** korisnicima (svim drugim, eng. others); pod "ostalima" se podrazumevaju svi korisnici koji nisu vlasnici fajla: ni pojedinačno, ni u okviru grupe; najispravniji termin bi bio "nevlasnici"

- Tako na primer, za fajl dokument.txt važi sledeće:

>> korisnik-vlasnik je korisnik_1

>> grupa-vlasnik je grupa_drugari, a članovi te grupe su korisnik_2 i korisnik_3

>> pod "ostali" se podrazumevaju svi drugi korisnici, osim tri navedena

Na ovaj način, znači, za dati fajl se mogu uočiti tri skupa korisnika, i admin svakoj može dodeliti neki pristup navedenom fajlu nezavisno od druge dve, u okviru kontrole pristupa

Koja vrsta pristupa je dozvoljena?

- Vlasnici fajla (korisnik i grupa), kao i svi ostali korisnici mogu imati dozvolu da očitavaju (to jest vide sadržaj -- eng. read), upisuju (to jest modifikuju sadržaj -- eng. write) i izvršavaju (eng. execute/run) fajlove (naravno samo one koji su izvršni)

- Slično, admin dodeljuje dozvole za pristup i za direktorijum:

>> read znači da korisnik može da pristupa fajlovima u tom direktorijumu

>> write znači da korisnik može da kreira i briše fajlove u tom direktorijumu

>> execute znači da korisnik može da izabere taj direktorijum kao radni (da "uđe" u njega), pomoću komande **cd**

- Da bi se ustanovilo vlasništvo (eng. ownership), i dozvole (eng. permissions) za pristup nekom fajlu ili direktorijumu, koristi se komanda **ls**: ona vraća jedan string u kome se za korisnika-vlasnika, grupu-vlasnika, i sve ostale korisnike ispisuju slova **r** (za read), **w** (za write) ili **x** (za execute), kao oznaku kome je šta dozvoljeno -- više detalja u nastavku

Tema 5: Šel

- Šel je Linuxov CLI (command-line interface): u njemu korisnik otkuca komandu i pritisne Enter; zatim šel izvrši odgovarajući program, i prikaže korisniku rezultat izvršenja
- Na primer, da bi se videlo koji su korisnici ulogovani na sistem, i kada, koristi se komanda **who**:

```
→ who
silver      :0      Sep 23 20:44
byrnes     pts/0      Sep 15 13:51
barrett    pts/1      Sep 22 21:15
silver     pts/2      Sep 22 21:18
```

pts (skraćenica od "pseudo-terminal slave") označava način kako je korisnik povezan na sistem (bez daljih objašnjenja)

- Jedna komanda može da pozove više programa istovremeno, a može i da ih ulanča (uveže) tako da interaguju, kao što je već objašnjeno; primer:

```
→ who | wc -l
4
```

- U primeru, program **who** se izvrši, pa njegov izlaz postaje ulaz programa **wc**, koji broji linije teksta; po difoltu, ovom programu se zada fajl (kao što je pokazano); u ovom slučaju njegov ulaz neće biti fajl, već lista koju je generisao **who** (a koja je prikazana iznad); tako će konačni rezultat biti broj ulogovanih korisnika

- Preciznije rečeno, komanda **who** daje listu svih šelova koje su korisnici otvorili; ako jedan korisnik otvori dva šela, njegovo ime će se pojaviti na listi dva puta, kao što je to slučaj sa korisnikom silver iz primera

- Šel je takođe program; kao što je pomenuto, ovde je stavljen fokus na bash šel, koji se nalazi u direktorijumu /bin/bash; da bi korisnik proverio koji šel koristi, treba da proveriti vrednost šel varijable **SHELL**:

```
→ echo $SHELL  
/bin/bash
```

- Ako korisnik koristi neki drugi šel, i hoće da pređe na bash, treba da otkuca komandu **bash**; za povratak u prethodni šel treba da otkuca **exit**

5.1 Komande-programi i ugrađene komande

- Neke komande (kao na primer **who**) koje korisnik otkuca pokreću (izvršavaju) odgovarajuće Linux programe, kao što je već pomenuto; sa druge strane, neke komande (kao na primer **cd**) su takozvane ugrađene (eng. build-in) u sâm šel, pa se nazivaju "builtin"

- Builtin komanda se, znači, izvršava u sâmom šelu; kada se kuca obična komanda, šel mora da pronađe i izvrši odgovarajući odgovarajući program; stoga su builtin komande brže nego obične; one se koriste za razne jednostavne zadatke, kao što je rad sa tekstom ili sa direktorijumima

- Ako korisnik hoće da sazna (jer mu nekad može biti od koristi) da li komanda pokreće program ili je ugrađena, treba da koristi komandu **type**: ona daje informaciju o Linux komandi koja joj je zadata kao argument

- Primer:

```
→ type who
who is /usr/bin/who
→ type cd
cd is a shell builtin
```

Ovde se vidi da **who** poziva odgovarajući Linux izvršni program (uz to se vidi i putanja do njega), a da je **cd** šelov "builtin", to jest ugrađena u šel

5.2 Pomoćni alati bash šela

- Glavni zadatak šela je da izvršava komande koje korisnik otkuca; istovremeno, šel sadrži i mnoge pomoćne alate koji olakšavaju obavljanje tog zadatka, kao na primer:

>> wildcard (bez srpskog prevoda): karakteri za pomoć pri pretrazi fajlova

>> command history (istorija komandi): za pregled prethodno unetih komandi

>> vertikalna crta za ulančavanje

>> šel varijable za čuvanje vrednosti

Ovi alati (i još neki drugi) će biti predstavljeni redom

Wildcard

- Wildcard je karakter (*, ? i još neki), ili generalno sekvenca karaktera, pomoću koga se pravi skraćeni zapis/oznaka za skup fajlova sličnog naziva; ovo je koncept ugrađen u sâm šel

- Prvi wildcard je zvezdica *: on označava (menja/obuhvata) proizvoljan broj karaktera u sekvenci, preciznije 0, 1, 2... uzastopnih karaktera; tako na primer:

>> b* označava skup svih fajlova čiji naziv počinje slovom b (na primer: b, bac, basic, bash)

>> b*c označava skup svih fajlova čiji naziv počinje slovom b, a završava slovom c (na primer: bc, b2c, basic)

- Tako, na primer, komanda:

→ **ls a***

znači: "ispiši listu svih fajlova, čiji naziv počinje slovom a, i koji se nalaze u radnom direktorijumu"; trivijalno, da je pisala samo zvezdica, ispisala bi se lista svih fajlova

- Wildcard * ne obuhvata samo dva karaktera: početnu tačku u nazivu fajla (ima i takvih naziva fajlova), i kosu crtu (sleš) koja odvaja nazive direktorijuma u putanji: ova dva karaktera moraju biti eksplicitno zadati, kao na primer:

```
>> .pro* obuhvata .profile
```

```
>> /etc/*conf obuhvata sve fajlove čiji se naziv završava sa conf, a nalaze se u direktorijumu /etc
```

- Napomena: kada naziv fajla (isto važi i za direktorijume) počinje tačkom, on neće biti prikazan pomoću komande **ls**; zato se ovakvi fajlovi nazivaju skriveni fajlovi (eng. hidden file), ili dot fajlovi (dot = tačka); da bi se i oni prikazali, koristi se opcija **-a**

- Naime, neki fajlovi u Linux sistemu, koji se mogu naći u raznim direktorijumima, nastaju kao rezultat nekih automatskih procesa, i stoga je bolje da ih Linux "proglasi" za skrivene (tako što im doda tačku na početku naziva); tako se takav fajl neće prikazati (po defaultu) pomoću komande **ls**, da se korisnik ne bi zbunio

- Znači, fajlovi se ne proglašavaju za skrivene iz razloga sigurnosti (veoma lako se mogu prikazati, kao što je pokazano), već da se korisniku na ekranu ne bi ispisivao naziv fajla sa kojim on nema nikakve direktne veze; ako korisnik hoće da vidi baš sve, onda znači koristi opciju **-a**

- Drugi wildcard koji se često upotrebljava je upitnik ?; on označava (menja/obuhvata) tačno jedan karakter; tako na primer:

```
>> b? označava skup svih fajlova čiji naziv ima dva karaktera, od kojih je prvi b (na primer, ba, bb, b3)
```

>> b?c označava skup svih fajlova čiji naziv ima tri karaktera, od kojih je prvi b, a treći c (na primer: bac, bbc, bxc)

Ako je potrebno označiti tačno dva karaktera, pišu se dva upitnika: ??, i tako dalje

- Treći wildcard je par uglastih zagrada []: u njima su navedeni svi karakteri koji dolaze u obzir, ili njihov opseg definisan pomoću crtice, na primer:

>> o[ac]d označava skup fajlova sa nazivima: oad i ocd

>> o[a-e]d označava skup fajlova sa nazivima: oad, obd, ocd, odd i oed

- Konačno, kada se u zagradama nalazi kret, on označava negaciju, pa na primer o[^a-e]d označava skup svih svih fajlova čiji troslovni naziv počinje sa o, završava se sa d, a srednje slovo nije a/b/c/d/e

- Sledeća tabela rezimira wildcard karaktere:

Wildcard	Meaning
*	Zero or more consecutive characters
?	Any single character
[set]	Any single character in the given <i>set</i> , most commonly a sequence of characters, like [aeiouAEIOU] for all vowels, or a range with a dash, like [A-Z] for all capital letters
[^set]	Any single character <i>not</i> in the given <i>set</i> , such as [^0-9] to mean any nondigit

Vitičaste zagrade

- Vitičaste zagrade (eng. curly braces) { } označavaju skup vrednosti, odvojene zarezima, koje će argument neke komande redom uzimati; na primer, {X,YY,ZZZ} označava da će se komanda prvo izvršiti sa argumentom koji uzima vrednost X, zatim YY, i na kraju ZZZ

- Primer:

```
→ echo sand{X,YY,ZZZ}wich  
sandXwich sandYYwich sandZZZwich
```

Vitičaste zagrade rade sa svakim stringom, za razliku od wildcard koji su ograničeni samo na nazive fajlova

Šel varijable/promenljive

- Šel pruža mogućnost rada sa varijablama (koristi se i termin promenljive), nalik na klasično programiranje; tehnički najispravniji termin je "šel varijabla" ali se uglavnom koristi samo termin "varijabla"; po konvenciji, nazivi varijabli se u Linuxu pišu svim VELIKIM SLOVIMA

- Dodela vrednosti varijabli izgleda isto kao u programiranju:

```
→ MYVAR=3
```

Obratiti pažnju da se pre i posle znaka jednakosti **ne** pišu razmaci

- Prilikom rada sa varijablom, to jest pristupa njenoj vrednosti, ispred njenog naziva stavlja se znak \$:

```
→ echo $MYVAR  
3
```

Znači, u ovom kontekstu, karakter \$ označava "vrednost", to jest \$MYVAR označava (vraća) vrednost varijable MYVAR

- Šel ima i neke rezervisane varijable, čije vrednosti on automatski dodeljuje prilikom logovanja korisnika, s tim što nekima od njih korisnik može promeniti vrednost; korisnik treba da izbegava da svojim varijablama daje nazive rezervisanih

- Sledeća tabela prikazuje najbitnije rezervisane varijable, i njihova značenja

Variable	Meaning
DISPLAY	The name of your X window display
HOME	Your home directory, such as <i>/home/smith</i>
LOGNAME	Your login name, such as <i>smith</i>
MAIL	Your incoming mailbox, such as <i>/var/spool/mail/smith</i>
OLDPWD	Your shell's previous directory, prior to the last <i>cd</i> command
PATH	Your shell search path: directories separated by colons
PWD	Your shell's current directory
SHELL	The path to your shell (e.g., <i>/bin/bash</i>)
TERM	The type of your terminal (e.g., <i>xterm</i> or <i>vt100</i>)
USER	Your login name

- **Domen** (eng. scope) neke varijable je skup svih programa koji joj imaju pristup; to je po defaultu samo onaj šel u kome je definisana

- Da bi nekoj varijabli (to jest njenoj vrednosti) imali pristup i svi drugi programi koji se iz šela pokreću, koristi se komanda **export**:

→ **export MYVAR**

a može se istovremeno odraditi i dodela vrednosti:

→ **export MYVAR=3**

- Takva javna/globalna varijabla se u Linuxu naziva **varijabla okruženja** (eng. environment variable), jer je pristupačna ne samo šelu nego i njegovom "okruženju"; preciznije, ona je pristupačna svim programima koje pokreće taj šel, uključujući i šel skripte (biće uvedeni u nastavku)

- Da dobije listu svih varijabli okruženja za dati šel, korisnik koristi komandu **printenv**:

→ **printenv**

Na toj listi bi se, naravno, pojavila i varijabla **MYVAR** definisana u prethodnom primeru; (više detalja o ovoj komandi biće dato u nastavku)

Pretraga putanje

- Programi nisu svi smešteni u jednom direktorijumu (kako bi bilo logično), već su "razbacani" svuda po fajlsistemu; neki se nalaze u direktorijumima kao što su **/bin** i **/usr/bin** (koji su predviđeni za čuvanje programa), ali se mnogo njih (bez nekih jasnih pravila) nalazi i na drugim mestima, za koja ni korisnik ni šel inicijalno ne znaju
- Postavlja se stoga sledeće pitanje: kada korisnik zada neku komandu (koja treba da pokrene odgovarajući program), na primer **who**: kako šel zna gde da traži odgovarajući program, a da ne pretražuje svuda redom?
- Odgovor: postoji rezervisana varijabla **PATH**, koja sadrži, u obliku sekvence, nazive svih direktorijuma u kojima bi program mogao da se nalazi; šel pristupa varijabli **PATH**, i traži program tako što pretražuje svaki od direktorijuma iz te sekvence
- Primer:

```
→ echo $PATH  
/usr/local/bin:/bin:/usr/bin
```

Nazivi direktorijuma su u okviru sekvence razdvojeni dvotačkama

- Znači, šel traži program redom u svakom od ovih direktorijuma; ako nađe program, izvršiće ga; ako ga ne nađe, daće odgovarajuće obaveštenje o grešci, kao na primer:

```
bash: command not found: proba
```

- Korisnik sâm može dodati direktorijume u ovu sekvencu:

```
→ PATH=$PATH:/usr/sbin  
→ echo $PATH  
/usr/local/bin:/bin:/usr/bin:/usr/sbin
```

- Ovaj dodatak važi samo privremeno, jer važi samo u trenutno otvorenom šelu; da bi promena postala trajna, u (skrivenom) fajlu `.bash_profile` u home direktorijumu treba modifikovati varijablu **PATH**; najbolje je to uraditi u nekom editoru
- Pomenuti fajl je jedan od više konfiguracionih fajlova; oni služe da korisnik podesi svoje okruženje za rad u šelu; svi se obično nalaze u njegovom home direktorijumu; stoga se ovaj konfiguracioni fajl obično označava kao `~/.bash_profile`

Alias

- Ugrađena komanda **alias** (eng. ali i srp. alias = drugo ime, pseudonim) definiše pogodan alias (najkorektniji prevod bi bio: skraćeni zapis ili skraćenica), za neku dužu komandu, da bi korisnik manje morao da kuca; na primer linija:

```
→ alias ll='ls -lG'
```

definiše novu komandu koja se zove **ll**, i koja je ekvivalentna sa **ls -lG**

- Da bi se alias definisao trajno (što je pogodno za komande koje korisnik svakodnevno koristi), treba ga u nekom editoru uneti u konfiguracioni fajl `~/.bash_aliases`; konačno, da bi se videla lista svih aliasa, kuca se samo komanda **alias**

Preusmeravanje ulaza i izlaza

- U šelu je moguće obavljati **preusmeravanje** (eng. redirection) ulaza i izlaza komandi (to jest programa); za to se koriste karakteri `<` i `>`

- Preusmeravanje ulaza neke komande znači da joj se ulazna vrednost, umesto sa tastature (u generalnom slučaju, sa standardnog ulaza), zadaje iz fajla (koji se stoga naziva ulazni -- eng. *infile*); u tu svrhu se koristi karakter < (manje):

→ **command < infile**

- Primer:

→ **sort < file_list.txt**

Komanda **sort** sortira sadržaj fajla *file_list.txt* (preciznije, njegovih linija), i rezultat ispisuje na ekranu

- Slično, preusmeravanje izlaza neke komande znači da joj se izlazna vrednost, umesto na ekranu (u generalnom slučaju, na standardnom izlazu), ispisuje u fajl (koji se stoga naziva izlazni -- eng. *outfile*); u tu svrhu se koristi karakter > (veće):

→ **command > outfile** *create outfile*

- Pri tome, ako izlazni fajl ne postoji, kreiraće se; ako postoji, novi podaci biće upisani preko već postojećih; ako korisnik hoće da u postojeći izlazni fajl nastavi da upisuje podatke **posle** već postojećih (da nastavi), treba da koristi dupli znak veće, to jest >>:

→ **command >> outfile** *append to outfile*

- Primer:

→ **ls > file_list.txt**

Komanda **ls** svoj izlaz ispisuje u fajl *file_list.txt*; na ekranu ne ispisuje ništa; ako korisnik hoće da se dobijeni izlaz ispiše na kraj postojećeg fajla (to jest da se u njemu sačuvaju postojeći podaci, a da se novi na njih nastave/nadovežu), treba da otkuca istu liniju samo sa >> umesto >

- Konačno, moguće je kombinovati preusmeravanje ulaza i izlaza, kao na primer:

→ **sort < file_list.txt > sorted_file_list.txt**

Komanda **sort** preuzima podatke iz fajla `file_list.txt`, sortira ih, i upisuje u fajl `sorted_file_list.txt`; redosled zapisivanja nije važan jer se prate znaci `>` i `<`

- Važno je na kraju još napomenuti da znaci `>` i `<` moraju biti napisani na samom kraju, tek pošto se posle imena komandi ispišu sve opcije i argumenti

Ulančavanje

- Kao što je pomenuto, vertikalna crta `|` služi za postupak ulančavanja: u njemu, izlaz jedne komande (to jest programa) postaje ulaz naredne, čime se pravi lanac komandi; primer:

→ **who | sort**

Izlaz programa **who** postaje ulaz programa **sort**: tako se ispisuje lista ulogovanih korisnika, sortirana po abecednom redu

- Moguće je ulančavati proizvoljan broj puta, kao na primer:

→ **who | sort | awk '{print \$1}' | less**

Pomoću komande **awk** (biće uvedena) izdvaja se jedna (u primeru: prva) kolona sortirane liste korisnika (a to su njihova korisnička imena), i na kraju pomoću komande **less** ispisuje se rezultat strana po strana

Sekvenca komandi

- Moguće je u jednoj liniji pozvati jednu za drugom (redom u sekvenci) više komandi; one se u toj liniji odvajaju tačka-zarezom (uz obavezne razmake):

→ **command1 ; command2 ; command3**

Znači izvršiće se redom: prva, pa druga, pa treća navedena komanda

- Moguće je zadati tako da se izvršenje sekvence prekine čim jedna komanda, usled bilo kog razloga, ne uspe da se izvrši (ili krahira, pokušaj prevoda eng. to fail); drugim rečima,

da se, čim jedna komanda krahira, izvršenje linije zaustavi uz ignorisanje svih komandi koje slede u sekvenci posle te komande; u tom slučaju, komande treba odvojiti simbolom **&&**:

→ **command1 && command2 && command3**

Ovo je na primer pogodno kada nema smisla pozivati neku komandu ako se neka koja se pokreće pre nje nije uspešno završila

- Slično, ako korisnik hoće da prekine izvršenje sekvence čim se jedna od komandi uspešno izvrši, treba da ih odvoji simbolom **||** (dve vertikalne crte):

→ **command1 || command2 || command3**

Ovo je na primer pogodno kada se neki zadatak može izvršiti na više načina, pa ako se uspe na jedan onda drugi više nisu potrebni

Zamena komande

- Pomoću konstrukcije **\$()** pravi se takozvana **zamena komande** (eng. command substitution); komanda se menja svojim izlazom; preciznije, ono što piše u zagradama se tretira kao šel komanda koja se smesta izvrši, i zatim cela konstrukcija menja izlazom (rezultat) te komande; ovo je pogodno za kombinovanje komandi

- Primer:

→ **date +%Y** *Print the current year*

2016

→ **echo This year is \$(date +%Y)**

This year is 2016

- U istu svrhu može se upotrebiti i par karaktera **`** (backquote ili backtick, ASCII 96, srpski termin ne postoji, "kosi polunavodnik" bi bio najbliži prevod):

→ **echo This year is `date +%Y`**

This year is 2016

Ovaj zapis sve više ispada iz upotrebe, pa se ne preporučuje

- Može se koristiti i šel promenljiva:

```
→ NUM_USERS=$(who | wc -l)
→ echo There are currently $NUM_USERS users
```

U primeru, NUM_USERS označava broj trenutno ulogovanih korisnika; izlaz je:

There are currently 2 users

Eskejpovanje i citiranje

- Većina specijalnih (onih koji nisu slova i cifre -- nealfanumerički) karaktera u šelu ima specijalno značenje, pa se nazivaju i **metakarakter**i; to su, na primer (napisani bez zareza između): razmak \$ < > * ? [] ; { } i slično; njih šel interpretira na specijalan način; na primer, * je već pomenut kao wildcard, \$ služi za označavanje vrednosti neke promenljive, < za preusmeravanje, a ; za pravljenje sekvence komandi
- Ako korisnik hoće da takav karakter koristi bukvalno, to jest kao običan karakter bez tog specijalnog značenja (na primer, da * koristi kao karakter zvezdicu umesto kao wildcard), on ispred tog karaktera treba da stavi obrnutu kosu crtu \
- Navedeni postupak se naziva **eskejpovanje** karaktera (eng. escaping, bez srpskog prevoda); tako se znači karakteru "ukida" specijalno značenje tako da postane bukvalan, to jest da umesto metakaraktera postane običan karakter
- Kao trivijalan primer, linija koja ima za cilj da ispiše jednostavan tekst u obliku pitanja:
→ **echo Is 5<3?**
generisaće poruku o grešci, jer karakteri < i ? imaju specijalno značenje
- Da bi se navedeni tekst pravilno ispisao, navedeni karakteri se moraju eskejpovati:

```
→ echo Is 5<3\?  
Is 5<3?
```

- Ukoliko hoće da izbegne eskejpovanje svakog metakaraktera pojedinačno, korisnik ima još jednu mogućnost, a to je da ceo tekst stavi u polunavodnike; taj postupak se naziva **citiranje** (eng. quoting)

- Pomoću citiranja, svim metakarakterima koji se nalaze između para polunavodnika se ukida specijalno značenje, to jest svaki postaje bukvalan/običan; ovo čini da ceo zapis postaje jednostavniji; za citiranje se mogu koristiti i navodnici, ali oni pri ukidanju specijalnog značenja prave neke izuzetke -- neće se pominjati

- Tako bi prethodna linija mogla da glasi:

```
→ echo 'Is 5<3?'  
Is 5<3?
```

- Kao praktičniji primer, pokazaće se kako se radi sa fajlovima i direktorijumima koji u svom nazivu imaju razmak, koji takođe u šelu ima specijalno značenje

- Kao što je pomenuto, upotrebu razmaka u nazivima fajlova i foldera treba obavezno izbegavati; ako korisnik mora da radi sa tuđim fajlom/folderom koji u svom nazivu ima razmak (na primer, *my file* ili *my file.txt* ili *new directory*), pri radu treba da obrati pažnju; u okviru naziva fajla, razmak je običan/bukvalni karakter (isto kao i slovo *m* iz primera)

- Sa druge strane, (metakarakter) razmak ima sledeće specijalno značenje: na komandnoj liniji služi kao separator između komande i njenih argumenata, kao i između argumenata međusobno (naravno i opcija)

- Zato, ako korisnik na komandnoj liniji kao argument upiše naziv fajla koji sadrži (na primer) dve reči odvojene razmakom, generisaće se greška: Linux će taj razmak koga je

korisnik upisao sa namerom da bude bukvalan (u okviru jednog argumenta) tretirati kao da ima specijalno značenje (separator), pa će zato dve reči iz naziva fajla tretirati kao dva nezavisna argumenta

- Na primer, ako korisnik pokuša da prikaže sadržaj fajla *my file* pomoću:

→ **cat my file**

Linux će kao grešku (tačnije, dve greške) prijaviti da nije pronašao fajl *my* ni fajl *file*; Linux znači smatra da su mu zadata dva fajla

- Navedeni problem se rešava tako što se razmak iz naziva fajla eskejpuje:

→ **cat my\ file**

Na taj način, tom razmaku se ukida specijalno značenje (separatora na komandnoj liniji), i tretira se kao običan karakter, deo naziva fajla

- Naravno, isto se može postići i na jednostavniji način, pomoću citiranja:

→ **cat 'my file'**

- Kao još jedan primer, kuca se linija pomoću koje se ispisuju svi fajlovi (u radnom direktorijumu) čiji naziv počinje sa a:

→ **echo a*** *As a wildcard, matching "a" filenames*
aardvark adamantium apple

Ovde se (metakarakter) *** koristi kao wildcard, kao što je već pokazano

- Sa druge strane, ako korisnik hoće da ispiše tekst *a**, treba da eskejpuje zvezdicu:

→ **echo a*** *As a literal asterisk*
a*

Prečice na komandnoj liniji

- Bash omogućava korisniku fleksibilnost pri kucanju komandi na komandnoj liniji, korišćenjem "prečica" sa tastature, a po uzoru na editore teksta emacs i vi; korisnik treba samo da odabere da li hoće "emacs prečice" ili "vi prečice"

- Za korišćenje "emacs prečica", kuca se:

→ **set -o emacs**

a za korišćenje "vi prečica" kuca se:

→ **set -o vi**

Komanda **set** služi za podešavanje nekih opcija u šelu – bez više detalja; da bi izbor bio permanentan, komandu treba upisati u fajl ~/.bash_profile

- Pregled najbitnijih prečica iz oba editora nalazi se u narednoj tabeli

emacs keystroke	vi keystroke (after ESC)	Meaning
^P or up arrow	k	Go to previous command
^N or down arrow	j	Go to next command
^R		Search for a previous command interactively
^F or right arrow	l	Go forward one character
^B or left arrow	h	Go backward one character
^A	0	Go to beginning of line
^E	\$	Go to end of line
^D	x	Delete next character
^U	^U	Erase entire line

Istorija komandi

- Šel pamti koje je redom komande korisnik kucao, što se naziva njihova **istorija** (eng. command history); korisnik može da ih "prizove" to jest ponovo zada; najbitnije komande vezane za istoriju su date u narednoj tabeli

Command	Meaning																
<code>history</code>	Print your history																
<code>history N</code>	Print the most recent <i>N</i> commands in your history																
<code>history -c</code>	Clear (delete) your history																
<code>!!</code>	Re-run previous command																
<code>!N</code>	Re-run command number <i>N</i> in your history																
<code>!-N</code>	Re-run the command you typed <i>N</i> commands ago																
<code>!\$</code>	Represents the last parameter from the previous command; great for checking that files are present before running a destructive operation, like removing them: → <code>ls z*</code> <code>zebra.txt zookeeper</code> → <code>rm !\$</code> Same as " <code>rm z*</code> "																
<code>!*</code>	Represents all parameters from the previous command: → <code>ls myfile emptyfile hugefile</code> <code>emptyfile hugefile myfile</code> → <code>wc !*</code> <table><tr><td>18</td><td>211</td><td>1168</td><td>myfile</td></tr><tr><td>0</td><td>0</td><td>0</td><td>emptyfile</td></tr><tr><td>333563</td><td>2737539</td><td>18577838</td><td>hugefile</td></tr><tr><td>333581</td><td>2737750</td><td>18579006</td><td>total</td></tr></table>	18	211	1168	myfile	0	0	0	emptyfile	333563	2737539	18577838	hugefile	333581	2737750	18579006	total
18	211	1168	myfile														
0	0	0	emptyfile														
333563	2737539	18577838	hugefile														
333581	2737750	18579006	total														

Kompletiranje naziva fajla

- Ako korisnik u toku kucanja naziva nekog fajla pritisne taster Tab, šel će automatski kompletirati taj naziv umesto korisnika; pored toga:

>> ako za kompletiranje postoji više kandidata (prema onome šta je korisnik dotad otkucao), šel će dati zvučni signal, kao obaveštenje da nije u stanju da kompletira

>> ako korisnik tada još jednom pritisne Tab, šel će ponuditi listu kandidata, tako da korisnik može da odabere

- Primer:

```
→ cd /usr/bin  
→ ls un<Tab><Tab>
```

Šel će ovde prikazati sve nazive fajlova iz */usr/bin* čiji naziv počinje sa *un* – to su na primer *uniq* i *unzip*, tako da korisnik može da odabere jedan od njih; naravno, korisnik može da otkuca još neki karakter da bi smanjio broj kandidata; na primer, ako posle *un* otkuca z pa Tab, ostaje samo jedan kandidat (jer *uniq* otpada), i šel će kompletirati *unzip*

Izračunavanje na komandnoj liniji

- Za obavljanje raznih aritmetičkih (za izračunavanje), logičkih, i operacija nad stringovima na komandnoj liniji služi komanda **expr** ("expression"), kao na primer:

```
→ expr 7 + 3  
10
```

- Treba obratiti pažnju da svi argumenti komande, to jest svi elementi izraza koji se računa (brojevi, aritmetički i logički operatori, zagrade i slično) moraju biti razdvojeni razmacima; takođe, upotreba komande je obavezna; kad bi korisnik otkucao samo 7+3 ili 7 + 3, javila bi se poruka o grešci

- Ukoliko se u izrazu koriste zagrade, znak za množenje, znak za veće i slično (pomenuti metakarakter), oni se naravno moraju eskejpovati:

```
→ expr \( 7 + 3 \) \* 14  
140
```

→ **expr 15 \> 16**
0

0 means False, 1 means True

- Citiranje celog izraza u ovom slučaju ne daje rezultat (jer bi ukinulo specijalno značenje razmaku); ono šta daje rezultat je citiranje svakog metakaraktera posebno (mada zapis i dalje ostaje veoma nečitljiv):

→ **expr '(' 7 + 3 ')' '*' 14**
→ **expr 15 '<' 16**

- Neki operatori za komandu **expr** nalaze se u narednoj tabeli:

Operator	Numeric operation	String operation
+, -, *, /	Addition, subtraction, multiplication, and integer division, respectively	
%	Remainder (mod)	
<	Less than	Earlier in dictionary
<=	Less than or equal	Earlier in dictionary, or equal
>	Greater than	Later in dictionary
>=	Greater than or equal	Later in dictionary, or equal
=	Equality	Equality
!=	Inequality	Inequality
 	Boolean "or"	Boolean "or"
&	Boolean "and"	Boolean "and"

Slanje i-mejla

- Korisnici Linux sistema često imaju potrebu da pišu i-mejlove; oni naravno imaju svoje GUI alate/aplikacije (na primer, postoji Thunderbird za Linux) za uobičajeno pisanje mejlova, ali ponekad je potrebno napisati kratki mejl sa komandne linije šela, ili iz nekog skripta (napomena: postoje i komande za četovanje u okviru šela)

- Takođe, može se podesiti da se mejl napiše automatski nekome pod nekim uslovom (na primer, korisniku koji je blizu da premaši dodeljenu kvotu memorije, ili novom korisniku kao mejl dobrodošlice sa uputstvima za rad); nekada korisnik može poslati mejl i samom sebi kao podsetnik

- Za jednostavno slanje mejlova koristi se komanda **mail**; primer:

```
→ mail smith@example.com
Subject: my subject
I'm typing a message.
To end it, I type a period by itself on a line.
.
Cc: jones@example.com
→
```

Da bi mejl mogao da se pošalje, sistem mora biti tako podešen (neće se ulaziti u detalje)

- Slanje pomoću jedne linije obavlja se pomoću ulančavanja:

```
→ echo "Hey!" | mail -s "subject" smith@example.com
```

Opcije su: **-s** za subject, **-c** (posle koga idu adrese odvojene zarezima) za CC slanje, **-b** za BCC slanje

- Za slanje fajla kao atačmenta, koristi se preusmeravanje:

```
→ mail -s "my subject" smith@example.com < filename
```

Preuzimanje stranice ili fajla sa veba

- Korisnik može da preuzme neku HTML stranicu, ili neki fajl sa veba; za to se naravno uvek može koristiti i neki GUI brauzer, na primer Firefox za Linux, ali je to moguće uraditi i bez brauzera, direktno sa komandne linije

- Za navedeni zadatak koristi se komanda **curl** (postoji i njoj vrlo slična koja se naziva **wget**); po defaultu, ona sadržaj preuzetog fajla samo ispisuje na ekranu; ovo je pogodno samo ako je taj fajl tekstualni (za binarne fajlove je takav ispis korisniku beskoristan); izlaz ove komande može se pomoću ulančavanja i predati nekoj drugoj

- Da bi se HTML stranica ili fajl sa veba sačuvao na korisnikovom računaru, koristi se opcija **-o** (od output): njome se zadaje destinacija (cela putanja) gde korisnik hoće da mu se fajl sačuva, znači sintaksa je:

→ **curl url_file -o destination_file**

- Primer:

→ **curl https://linuxpocketguide.com/sample.pdf -o /home/smith/sample.pdf**

Tokom preuzimanja, komanda ispisuje na ekranu neka obaveštenja (o veličini fajla, brzini preuzimanja i slično)

5.3 Poslovi u šelu

- U šelu postoji koncept koji se naziva **posao** (eng. job); posao je neka komanda (to jest program) koja se izvršava: pošto jedna komanda pokreće jedan ili više procesa, onda se posao može posmatrati kao skup procesa, onako kako ih vidi šel (u kome je komanda zadata)

- Posao je koncept na višem nivou od procesa; Linux kernel se bavi procesima (poslovi za njega i ne postoje), a šel poslovima; kada se zadata komanda izvrši, odgovarajući posao prestaje da postoji (nestaje), a sa njim i svi njegovi procesi

Vidljivi posao

- Kada se neka komanda zada na uobičajeni način, šel prompt nestane, tako da korisnik ne može da kuca ništa drugo (to jest, neku novu komandu); kaže se da je komanda "zauzela" prompt; tek kada se ona izvrši, prompt se ponovo pojavljuje i korisnik može da kuca novu komandu
- Za takvu komandu zadatu na uobičajeni način, to jest za njoj odgovarajući posao, koji zauzima prompt, kaže se da se obavlja **u prvom planu** (eng. foreground), odnosno da je **vidljiv**
- Da se trenutno prekine, i bez mogućnosti za nastavak, neki vidljivi posao, treba otkucati **^C**; na primer:

```
→ cat hugefile
Lorem ipsum dolor sit amet, consectetur adipiscing
odio. Praesent libero. Sed cursus ante dapibus diam.
quis sem at nibh elementum blah blah blah ^C
→
```

- U primeru, korisnik je pomoću komande **cat** zadao ispisivanje nekog dugačkog tekstualnog fajla; pritiskom na **^C** to ispisivanje je trenutno i bespovratno prekinuto, i prompt čeka narednu komandu
- Na navedeni način se uzrokuje brutalan prekid izvršenja programa; ako piše neki program u Linuxu, korisnik treba da predvidi način da se izvršenje, u slučaju da se pojavi neki problem koji sprečava normalan završetak, elegantnije završi (ovo naravno važi za svako programiranje)

- Problem sa upotrebom **^C** je što može uzrokovati da se šel nađe u nekom poremećenom/nedovršenom stanju, jer je program trenutno prekinut, i nije imao vremena da "očisti" za sobom; to može dovesti do blokiranja šela (na primer, izgubi se prompt)
- U slučaju da šel zablokira, korisnik ima dva rešenja da ga odblokira:
 - 1) da otkuca **^J** da dobije prompt -- ovim se generiše newline karakter, kao da je pritisnuo Enter, ali **^J** radi nekad kada Enter neće; ako ovo ne uspe, onda:
 - 2) da otkuca komandu **reset** (čak i ako se slova koja kuca ne pojavljuju na ekranu) i ponovo otkuca **^J** (umesto Enter)

Nevidljivi posao

- Ako se na kraju neke komande navede karakter **&** (eng. ampersand), time korisnik nalaže da odgovarajući posao bude **nevidljiv**, to jest da se obavlja **u pozadini** (eng. background)
- U pozadinu korisnik obično šalje poslove za koje pretpostavlja da će dugo trajati, koji ne zahtevaju njegovo učešće, i koji ne ispisuju tekst na ekranu (na primer, procesiranje velikog broja fajlova)
- Na taj način korisnik znači pokrene posao kao nevidljiv i odmah mu se oslobodi prompt; on na njemu može da kuca neku drugu komandu, to jest može da pokreće i druge poslove, čekajući da se nevidljivi posao obavi
- Znači, tipično korisnik može da pokrene više nevidljivih poslova, i dok se oni odvijaju u pozadini da radi "normalno", to jest u prvom planu, to jest da pokreće jedan za drugim neke vidljive poslove; svi ti poslovi (jedan koji je vidljiv, i svi nevidljivi) se odvijaju u paraleli
- Primer:

```
→ sleep 100 &  
[2] 16817  
→
```

prompt is immediately available

- U primeru, šel na komandu (koja znači "spavaj 100 sekundi") odgovara rednim brojem posla u uglastoj zagradi (2 u ovom slučaju), i ID odgovarajućeg procesa (16817) -- nebitan za ovaj kontekst, i prompt odmah postane raspoloživ za kucanje nove komande
- Da je kucano bez ampersanda, prompt bi tokom 100 sekundi bio zauzet; ovaj primer je ilustrativan i nije praktičan, ali komanda **sleep 100** se može zamisliti kao zamena za neku praktičnu komandu koja bi trajala 100 sekundi

Rad sa poslovima

- Za pregled svih nevidljivih poslova služi komanda **jobs**; koristi se bez argumenata
- Primer: korisnik pokrene dva nevidljiva posla:

```
→ sleep 100 &  
[1] 16989  
→ sleep 200 &  
[2] 16990
```

- Odmah zatim, kuca komandu **jobs** da napravi pregled:

```
→ jobs  
[1]-  Running                sleep 100 &  
[2]+  Running                sleep 200 &
```

Oba posla su navedena sa njihovim rednim brojevima (uz odgovarajuće komande); Running znači da se posao odvija; značenje + i - neće biti objašnjavano

- Pošto prođe više od 200 sekundi (i izvrše se obe komande **sleep**), ako korisnik ponovo otluca **jobs**, dobiće sledeći prikaz:

```
→ jobs
[1]-  Done                sleep 100
[2]+  Done                sleep 200
```

Done znači da je posao završen

- Ako korisnik hoće da zaustavi neki nevidljivi posao, potrebno je prvo da ga pomoću ugrađene komande **fg** (od foreground) učini vidljivim; naravno, da bi to mogao da uradi, nijedan drugi posao ne može biti vidljiv (jer je potreban prompt); zatim, taj sada vidljivi posao prekida pomoću **^C**

- Komanda **fg** znači služi da neki nevidljivi proglasi za vidljiv (da se iz pozadine izbaci u prvi plan); kao argument se zadaje redni broj posla; njena sintaksa je:

fg [%jobnumber]

- Primer:

```
→ sleep 100 &
[1] 18836
→ fg %1
sleep 100                command is displayed
^C                        command is terminated
→
```

- Ako korisnik hoće da neki vidljivi posao učini nevidljivim (na primer, ako se ispostavi da će dugo trajati, a potreban mu je prompt), prvo što treba da uradi je da taj vidljivi posao zaustavi pomoću **^Z**, a zatim da ga nastavi kao nevidljiv (da ga iz prvog plana stavi u pozadinu) pomoću ugrađene komande **bg** (od background)

- Znači, komanda **^Z** nalaže zaustavljanje nekog vidljivog posla (ako korisnik hoće da se tako zaustavljeni posao nastavi ponovo kao vidljiv, treba da koristi komandu **fg**); razlika između zaustavljanja (**^Z**) i prekida (**^C**) je što se zaustavljeni posao može nastaviti jer šel pamti zatečeno stanje, a prekid je bespovratan

- Komanda **bg** znači služi da neki zaustavljeni vidljivi posao nastavi gde je stao, ali kao nevidljiv; kao argument se zadaje redni broj posla; sintaksa je:

bg [%jobnumber]

- Primer:

→ **sleep 100**

^Z

[1]+ Stopped

sleep 100

Stopped označava da je posao zaustavljen

→ **bg %1**

[1]+ sleep 100 &

→ *prompt is immediately available*

Posle kucanja komande **bg**, posao se nastavlja u pozadini

5.4 Izlazak iz šela

- Za izlazak iz šela, treba otkucati komandu **exit** ili **^D**; ovo zapravo generiše "end of file" signal koji se upućuje programu koji se izvršava; u ovom slučaju, taj program je sâm šel, koji na taj način prekida sa izvršenjem

5.5 Podešavanje šela

- Ako korisnik hoće da obavi neka podešavanja šela po svojoj želji (na primer, da definiše aliase), to može da uradi (kako je već pomenuto) editovanjem konfiguracionih fajlova koji se nalaze u home direktorijumu: neki od ovih fajlova se izvršavaju svaki put kada se korisnik uloguje (pa se zato nazivaju i inicijalizacioni fajlovi), a neki svaki put kad otvori šel
- Već su pomenuti konfiguracioni fajlovi **.bash_profile** i **bash_aliases**; mogu se pomenuti i **.bashrc**, **.bash_login**, **.bash_logout** (neće se ulaziti u više detalja)

- Svi pomenuti fajlovi su primeri **šel skriptova** (eng. shell script): to su izvršni fajlovi koji sadrže šel komande u sekvenci (više detalja u nastavku)
- Ovim se završava osnovni pregled Linuxa i šela; u nastavku materijala biće predstavljene najbitnije Linux komande, koje se koriste za rad sa fajlovima, procesima, korisnicima, mrežom i slično



Tema 6: Rad sa fajlovima

- Glavne komande za rad sa fajlovima su:

ls List files in a directory.

cp Copy a file.

mv Rename ("move") a file.

rm Delete ("remove") a file.

ln Create links (alternative names) to a file.

Pomoću njih se obavlja: kopiranje fajlova, preimenovanje, brisanje i slično

- Svaki Linux sistem ima jednog ili više korisnika (eng. user), koji se preciznosti radi nazivaju i obični/normalni korisnici -- to su osobe koje se loguju i koriste resurse sistema; nasuprot njima, postoji i jedan korisnik koji se naziva superkorisnik (eng. superuser)

- Postoji pojam **vlasništva** (eng. ownership) korisnika nad fajlom (sve isto važi i za direktorijum); obični korisnici mogu da pristupaju samo fajlovima čiji su vlasnici (eng. owner), i fajlovima za koje su dobili dozvolu pristupa (od superkorisnika); superkorisnik ima pravo pristupa svim fajlovima u sistemu

- Svaki **korisnik** ima svoj korisnički ID (user ID = UID, uvek je numerička vrednost) i korisničko ime (user name); interno, UID je jedinstveni identifikator korisnika, a korisničko ime (koje znači postoji samo radi lakšeg označavanja od strane ljudi, a formalno je suvišno) se na njega "mapira", to jest uz njega je vezano

- Dalje, radi lakše organizacije pristupa fajlovima, postoje i **grupe** korisnika (eng. group); naime, lakše je administrirati jednu grupu nego veći broj korisnika pojedinačno; svaka grupa ima svoj grupni ID (group ID = GID, takođe numerička vrednost), koja je njen jedinstveni identifikator, i grupno ime (eng. group name) koje se mapira na GID

- Svaki korisnik mora pripadati bar jednoj grupi, a može pripadati i većem broju njih; od svih grupa kojima pripada, jedna se vodi kao njegova **primarna** grupa, a ostale se nazivaju njegove **dodatne** (sekundarne) grupe
- Prilikom dodavanja novog korisnika, automatski se generiše i grupa koja nosi isto ime (i čiji je GID isti kao njegov UID); ona se naziva njegova **privatna** grupa (on je njen jedini član); privatna grupa se inicijalno proglašava za njegovu primarnu grupu (kasnije se to može promeniti)
- Što se tiče vlasništva, kao što je objašnjeno, svaki fajl na disku ima:
 - 1) jednog korisnika koji se vodi kao vlasnik fajla ("korisnik-vlasnik fajla")
 - 2) jednu grupu koja se takođe vodi kao vlasnik ("grupa-vlasnik fajla"); svaki od članova te grupe se tretira kao vlasnik fajla
 - 3) sve one korisnike koji nisu ni korisnik-vlasnik, ni članovi grupe-vlasnika: oni se vode kao "ostali" (eng. others), to jest "nevlasnici"
- Kada korisnik kreira fajl, automatski on postaje vlasnik fajla, a takođe to postaje i njegova primarna grupa

ls [options] [files]

- Komanda **ls** (od "list") prikazuje fajlove i direktorijume, i njihove attribute; na primer, korisnik ovako dobija listu fajlova (i njihovih atributa) u radnom direktorijumu:

→ **ls**

ili istovremeno u više direktorijuma:

→ **ls dir1 dir2 dir3**

ili attribute zadatih fajlova:

→ `ls myfile myfile2 myfile3`

- Najvažnije opcije su `-a`, `-l`, `-d`; kao što je već navedeno, `ls` po defaultu ne prikazuje fajlove čiji naziv počinje tačkom; opcija `-a` ("all") nalaže da se prikažu i takvi fajlovi; primer:

```
→ ls
myfile  myfile2
→ ls -a
.hidden_file  myfile  myfile2
```

- Opcija `-l` ("long") nalaže da se generiše detaljniji prikaz:

```
→ ls -l myfile
-rw-r--r--  1 smith users  1168 Oct 28 2015 myfile
```

- Ovde se redom prikazuju atributi: dozvole pristupa (kao što je pomenuto u sekciji 4.5: `r` za očitavanje, `w` za upisivanje, `x` za izvršavanje, crtica označava "ništa"), korisnik-vlasnik (smith), grupa-vlasnik (users), veličina (1168 bajtova), datum poslednje modifikacije (28.10.2015), i naziv fajla (myfile)

- O dozvolama pristupa govori početni string od 10 karaktera:

- karakter 1: ako je `-` radi se o fajlu, ako je `d` radi se o direktorijumu
- karakteri 2 do 4 govore o dozvolama koje ima korisnik-vlasnik
- karakteri 5 do 7 govore o dozvolama koje ima grupa-vlasnik
- karakteri 8 do 10 govore o dozvolama koje imaju svi ostali korisnici

- Tako na primer, ako korisnik-vlasnik:

- ima dozvolu za očitavanje, karakter 2 će biti `r` (ako nema, biće `-`)
- ima dozvolu za upisivanje, karakter 3 će biti `w` (inače `-`)
- ima dozvolu za izvršavanje, karakter 4 će biti `x` (inače `-`)

Slično važi i za karaktere 5 do 7, i 8 do 10

- U primeru je string `-rw-r--r--`:
 - prva crtica - govori da je u pitanju fajl
 - string `rw` govori da korisnik-vlasnik ima dozvolu očitavanja i upisivanja
 - string `r--` govori da grupa-vlasnik ima dozvolu očitavanja
 - string `r--` govori da svi ostali imaju dozvolu očitavanja

- Opcija `-d` daje istu informaciju (atribute) ali o direktorijumu:

```
→ ls -ld dir1
drwxr-xr-x 1 smith users 4096 Oct 29 2015 dir1
```

- Pregled važnih opcija:

- a List all files, including those whose names begin with a dot.
- l Long listing, including file attributes. Add the `-h` option (human-readable) to print file sizes in kilobytes, megabytes, and gigabytes, instead of bytes.
- G In a long listing, don't print the group ownership of the file.
- F Decorate certain filenames with meaningful symbols, indicating their types. Appends "/" to directories, "*" to executables, "@" to symbolic links, "|" to named pipes, and "=" to sockets. These are just visual indicators for you, not part of the filenames!
- S Sort files by their size.
- t Sort files by the time they were last modified.
- r Reverse the sorted order.
- R If listing a directory, list its contents recursively.
- d If listing a directory, do not list its contents, just the directory itself.

`cp [options] files (file | directory)`

- Komanda `cp` ("copy") pravi kopiju zadatog fajla:

→ **cp file copy**

ili kopiju zadatog fajla (ili njih više) smešta u zadati direktorijum:

→ **cp file destination_dir**

Pomoću opcija **-a** i **-r** obavlja se rekurzivno kopiranje (kopira se i direktorijum, i svi njegovi poddirektorijumi, i tako redom)

- Pregled važnih opcija:

- p Copy not only the file contents, but also the file's permissions, timestamps, and if you have sufficient permission to do so, its owner and group. (Normally the copies will be owned by you, timestamped now, with permissions set by applying your umask to the original permissions.)
- a Copy a directory hierarchy recursively, preserving all file attributes and links.
- r Copy a directory hierarchy recursively. This option does not preserve the files' attributes such as permissions and timestamps. It does preserve symbolic links.
- i Interactive mode. Ask before overwriting destination files.
- f Force the copy. If a destination file exists, overwrite it unconditionally.

mv [options] source target

- Komanda **mv** ("move") služi da preimenuje fajl:

→ **mv file_name new_name**

ili da premesti fajlove i direktorijume (ili više njih) u neki drugi direktorijum:

→ **mv file_name dir_name destination_dir**

- Pregled važnih opcija:

- i Interactive mode. Ask before overwriting destination files.
- f Force the move. If a destination file exists, overwrite it unconditionally.

`rm [options] files | directories`

- Komanda `rm` ("remove") služi za brisanje fajlova (jednog ili više):

→ `rm deleteme deleteme2`

ili rekurzivno brisanje direktorijuma:

→ `rm -r dir1 dir2`

- Pregled važnih opcija:

-i Interactive mode. Ask before deleting each file.

-f Force the deletion, ignoring any errors or warnings.

-r Recursively remove a directory and its contents. Use with caution, especially if combined with the -f option, as it can wipe out all your files.

`ln [options] source target`

- Pomoću komande `ln` ("link") kreira se **hard link**, to jest referenca, na neki neki fajl; tako se jednom istom fajlu može pristupiti pomoću više naziva (Slika 6-1); postoji i soft (ili simbolički) link, koji pravi referencu na fajl pomoću njegove putanje, i kreira se pomoću opcije `-s` (neće se obrađivati)

- Hard link znači predstavlja bukvalno drugi naziv za originalni fajl; koncept podseća na deljene reference u jeziku Python, samo što se tamo radi o varijablama, a ovde o fajlovima

- Primer:

→ `ln myfile myhardlink`

Pošto se kreira hard link pod nazivom myhardlink, on će se naći u direktorijumu zajedno sa originalnim fajlom myfile; komanda `ls` ga prikazuje zajedno sa svim drugim fajlovima (i drugim hard linkovima)

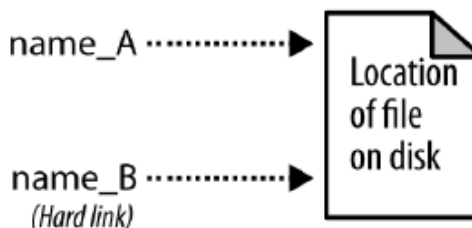
Make a file:

```
$ touch name_A
```



Hard link:

```
$ ln name_A name_B
```



Slika 6-1: Hard link

- Uz to, Linux tretira hard link potpuno ravnopravno kao i njemu odgovarajući fajl; nad hard linkom se mogu obavljati sve operacije za rad nad fajlovima; na primer:

```
→ cat myfile
```

daje isti rezultat kao i:

```
→ cat myhardlink
```

- Pri svemu tome, ako se na primer naprave neke modifikacije u fajlu koristeći njegov hard link, sve one će se videti kada se bude pregledao originalni fajl; naravno važi i obrnuto, jer se radi o jednim te istim podacima u memoriji, kojima se samo pristupa na dva načina
- Ako se originalni fajl obriše, preimenuje ili premesti, hard link će i dalje biti validan; hard link se briše kao i svaki fajl, pomoću komande `rm`

- Pregled važnih opcija:

- s Make a symbolic link instead of a hard link.
- i Interactive mode. Ask before overwriting destination files.
- f Force the link. If a destination file exists, overwrite it unconditionally.
- d Create a hard link to a directory (superusers only).

- Da bi se za neki hard link prikazao odgovarajući fajl, koristi se komanda `ls`; primer:

```
→ ls -l examplelink
lrwxrwxrwx 1 smith    ...    examplelink -> myfile
```

mada postoji i namenska komanda `readlink`

Tema 7: Rad sa direktorijumima

- Glavne komande za rad sa direktorijumima su:

<code>cd</code>	Change your current directory (i.e., "where you are now" in the filesystem).
<code>pwd</code>	Print the name of your current directory.
<code>basename</code>	Print the final part of a file path.
<code>dirname</code>	Print a file path without its final part.
<code>mkdir</code>	Create (make) a directory.
<code>rmdir</code>	Delete (remove) an empty directory.
<code>rm -r</code>	Delete a nonempty directory and its contents.

Pomoću njih se obavlja kreiranje, modifikacija, brisanje i još neke operacije nad direktorijumima

`cd [directory]`

- Komanda **cd** ("change directory") definiše (određuje) radni direktorijum:

→ **cd /usr/games**

Ako se direktorijum ne navede, home direktorijum korisnika će po defaultu biti proglašen za radni:

→ **cd**

pwd

- Komanda **pwd** ("path of working directory") ispisuje apsolutnu putanju radnog direktorijuma:

→ **pwd**
/users/smith/linuxpocketguide

basename path [extension]

- Komanda **basename** ispisuje samo naziv fajla, koji je zadat pomoću svoje apsolutne putanje:

→ **basename /users/smith/finances/money.txt**
money.txt

- Ako se zada direktorijum pomoću svoje apsolutne putanje, komanda će ispisati samo naziv tog direktorijuma:

→ **basename /users/smith/finances/**
finances

Drugim rečima, ova komanda ispisuje samo poslednju stavku u zadatoj putanji

- Ako se navede ekstenzija (sufiks) fajla, naravno tamo gde postoji, ispisaće se naziv fajla bez te ekstenzije:

→ **basename /users/smith/finances/money.txt .txt**
money

dirname path

- Komanda **dirname** je komplementarna prethodnoj: ako joj se zada fajl (ili direktorijum) pomoću apsolutne putanje, ona ispisuje celu tu putanju izuzev naziva fajla (direktorijuma):

→ **dirname /users/smith/mydir**
/users/smith

Drugim rečima, ova komanda ispisuje sve osim poslednje stavke u zadatoj putanji

mkdir [options] directories

- Komanda **mkdir** ("make directory") kreira jedan ili više direktorijuma u radnom direktorijumu:

→ **mkdir directory1 directory2 directory3**

- Pregled važnih opcija:

-p Given a directory path (not just a simple directory name), create any necessary parent directories automatically. The command:
→ **mkdir -p one/two/three**
creates *one* and *one/two* and *one/two/three* if they don't already exist.

rmdir [options] directories

- Komanda **rmdir** ("remove directory") briše jedan ili više **praznih** direktorijuma:

→ **mkdir /tmp/junk** *make a directory*
→ **rmdir /tmp/junk**

- Pregled važnih opcija:

-p If you supply a directory path (not just a simple directory name), delete not only the given directory, but the specified parent directories automatically, all of which must be empty. So `rm -p one/two/three` will delete not only *one/two/three*, but also *one/two* and *one*.

- Brisanje **nepraznog** direktorijuma (i sa njim celog njegovog sadržaja,) koristi se druga komanda, `rm`, sa opcijom `-r`:

→ **`rm -r directory`**

Da bi se to uradilo interaktivno (to jest uz zahtev korisniku da potvrdi), piše se opcija `-ri`; opcija `-rf` služi da se brisanje obavi bez ikakve poruke (forced)

Tema 8: Pregled fajlova

- Glavne komande za pregled fajlova su:

cat	View files in their entirety.
less	View text files one page at a time.
nl	View text files with their lines numbered.
head	View the first lines of a text file.
tail	View the last lines of a text file.
strings	Display text that's embedded in a binary file.
od	View data in octal (or other formats).

Tekst i binarni fajlovi

- Svaki fajl je jedna konačna povorka bitova (nula i jedinica); pri radu sa tom povorkom, radi se obično sa sekvencama od 8 bita, to jest bajtovima; u jednom bajtu može se sadržati

binarna vrednost između 00000000 i 11111111, to jest 0 i 255 u decimalnom sistemu, to jest 0 i FF u decimalnom sistemu, to jest 0 i 377 u oktalanom sistemu

- U Linuxu (i u računarstvu uopšte) postoje generalno dva tipa fajlova: tekst i binarni (koji se nazivaju i ne-tekst fajlovi); u ovoj temi će se prikazati kako se pregleda njihov sadržaj
- **Tekst** (ili tekstualni) **fajlovi** (eng. text file) su jednostavni fajlovi koji sadrže isključivo neformatirani tekst u obliku niza karaktera (slova, cifara i slično); za rad sa njima korisnik koristi neki tekst editor (kao što je Notepad u Windowsu, ili editore dostupne u Linuxu)
- **Binarni fajlovi** (eng. binary file) su, formalno, svi oni koji nisu tekst fajlovi; ima ih u raznim formatima; na primer, binarni fajl je fotografija u PNG formatu, ili dokument u DOCX formatu ili PDF formatu, ili izvršni fajl
- Na koji način je, na primer, informacija sadržana u fotografiji (podaci o pikselima i njihovim bojama) ili u dokumentu (podaci o tekstu, njegovom formatiranju, formatiranju stranice i slično) "spakovana" u povorki bajtova u odgovarajućem binarnom fajlu zavisi od formata fajla
- Ova organizacija podataka u binarnom fajlu je veoma komplikovana materija, i poznaju je samo specijalisti za pojedinačne formate fajlova, a obični korisnici to nisu dužni da znaju; naravno, svaka aplikacija (na primer, Microsoft Word) "zna" kako da interpretira povorku bajtova u odgovarajućem fajlu (na primer, u DOCX dokumentu)
- Na primer, specijalista za PDF format poznaje kako su zapisani ("enkodirani") podaci u PDF fajlu; on tako zna kako može da "spoji" dva PDF dokumenta u jedan; i još usputna napomena: podaci se u PDF formatu (i mnogim drugim) enkodiraju tako da budu komprimovani

- Tekst fajlovi su posebni po tome što je organizacija podataka u njima poznata i jednostavna: svaki karakter se kodira pomoću svog ASCII koda; tako na primer:

>> slovu a odgovara bajt 01100001 (141 oktalno = 97 decimalno = 61 heksadecimalno)

>> slovu b 01100010 (142 oktalno = 98 decimalno = 62 binarno)

>> slovu c 01100011 (143 oktalno = 99 decimalno = 63 heksadecimalno)

Lista ASCII kodova se nalazi na adresi <https://en.wikipedia.org/wiki/ASCII>

- Dalje, ako je na primer sadržaj tekstualnog fajla abc, taj fajl je zapisan u memoriji kao povorka bitova 011000010110001001100010 (ili po bajtovima 141 142 143 u oktalnom sistemu); ima veličinu od 3 bajta

- Znači, rad sa tekst fajlovima je jednostavan: odgovarajući program (na primer Notepad) redom očitava bajt po bajt ovakvog fajla, i svaki predstavlja na ekranu odgovarajućim karakterom po ASCII standardu; tako će korisnik moći jednostavno da pročita tekst

- Ako se binarni fajl otvori u tekst editoru, svaki očitani bajt će pokušati da se interpretira kao jedan karakter prema ASCII tabeli, ali ceo dobijeni izlaz će sadržati gomilu neupotrebljivih simbola i za korisnika će biti sasvim beskoristan (s obzirom da, kako je već objašnjeno, informacija nije ni namenjena da bude sadržana na taj način)

- Postoje i neki alati (aplikacije) koji služe za pregled sadržaja binarnih fajlova, bajt po bajt; takva aplikacija se naziva "Hex viewer"; ona radi tako što očitava bajt po bajt iz fajla, i svaki od njih prikazuje kao heksadecimalni (ili decimalni, ili oktalni, prema želji korisnika) broj

- Kao što je pomenuto, dobijena povorka brojeva od koristi može biti samo specijalisti, ali ne i običnom korisniku; ipak je potrebno znati da takve aplikacije postoje; postoje i njihove onlajn verzije, kao što je <https://hexed.it>, gde korisnik može da aploaduje svoj binarni fajl, i pregleda odgovarajuću povorku brojeva

- Konačno, binarni fajl (zavisno od tipa) može da u sebi ponegde sadrži i neki čitljiv (smisleni) tekst, nalik na tekst fajl; znači, može sadržati niz od nekoliko bajtova, koji ako se interpretiraju kao ASCII kodovi mogu generisati neki čitljiv tekst
- Pretraga takvog teksta se obavlja pomoću alata (aplikacije) koja se zove "Word extractor"; ovaj tekst može da korisniku pruži neke značajne informacije vezane za taj binarni fajl (na primer, format, verziju ili ime autora); na sličan način i sâm Linux pronalazi informaciju o formatu fajla tako što "zaviri" u njegov sadržaj

cat [options] [files]

- Najprostija komanda je **cat**, koja ispisuje sadržaj tekstualnog fajla na ekranu:

→ **cat myfile**

Ako se komandi zada više fajlova, ona će ih ispisati jedan za drugim, to jest nadovezati ih (eng. concatenation, od čega potiče i naziv komande)

- Ako je fajl dugačak, ispis će izaći izvan ekrana, tako da je bolje koristiti komandu **less** (više u nastavku)

- Komanda **cat** se često koristi u ulančavanju, kao na primer:

→ **cat myfile | wc**

- Korisnik može, pomoću opcija, da podesi da komanda **cat** prikazuje tekst uz neke dodatne obrade, kao što su ispis "nevidljivih" karaktera (eng. nonprinting character), numerisanje ispisanih linija (iako je komanda **nl** za tu namenu bolja), ili brisanje belih karaktera (eng. whitespace) – razmak, tab i još neki

- Izlaz komande **cat** može se i preusmeravati, kao na primer:

→ **cat test1.txt test2.txt > test3.txt**

U ovoj liniji, na sadržaj prvog fajla se nadoveže sadržaj drugog fajla, i sve to se upiše u treći fajl

- Pregled važnih opcija:

- T Print tabs as ^I.
- E Print newlines as \$.
- v Print other nonprinting characters in a human-readable format.
- n Prepend line numbers to every line.
- b Prepend line numbers to nonblank lines.
- s Squeeze each sequence of blank lines into a single blank line.

less [options] [files]

- Komanda **less** se koristi da bi se tekst (ili sadržaj tekstualnog fajla) prikazivao "stranicu po stranicu", preciznije prozor po prozor:

→ **less myfile**

- Ako se na primer piše lanac od više komandi, onda je **less** korisna da se napiše kao poslednja, da bi pogodno prikazala krajnji rezultat ukoliko je on dugačak:

→ **command1 | command2 | command3 | command4 | less**

- Tokom ispisivanja teksta na ekranu, korisnik ima na raspolaganju neke tastere (prečice, eng. keystroke) pomoću kojih sebi može da olakša pregled i praćenje teksta:

Keystroke	Meaning
h, H	View a help page.
Space bar, f, ^V, ^F	Move forward one screenful.

Enter	Move forward one line.
b, ^B, ESC-v	Move backward one screenful.
/	Enter search mode. Follow it with a regular expression and press Enter, and <code>less</code> will look for the first matching line.
?	Same as /, but it searches backward in the file.
n	Next match: Repeat your most recent search forward.
N	Repeat your most recent search backward.
v	Edit the current file with your default text editor (the value of environment variable <code>VISUAL</code> , or if not defined, <code>EDITOR</code> , or if not defined, the program <code>vi</code>).
<, g	Jump to beginning of file.
>, G	Jump to end of file.
:n	Jump to next file.
:p	Jump to previous file.

- Komanda `less` ima ogroman broj funkcionalnosti; na primer, ona može da prikaže i sadržaj zipovanog fajla (`less myfile.zip`)

- Pregled važnih opcija:

- c Clear the screen before displaying the next page. This avoids scrolling and may be more comfortable on the eyes.
- m Print a more verbose prompt, displaying the percentage of the file displayed so far.
- N Display line numbers.
- r Display control characters literally; normally `less` converts them to a human-readable format.

- s Squeeze multiple, adjacent blank lines into a single blank line.
- S Truncate long lines to the width of the screen, instead of wrapping.

`nl [options] [files]`

- Komanda `nl` ispisuje sadržaj tekstualnog fajla na ekranu (preciznije: na standardnom izlazu), i numeričke linije teksta:

→ `nl poem`

```
1 Once upon a time, there was
2 a little operating system named
3 Linux, which everybody loved.
```

- Ona je fleksibilnija (posebno zahvaljujući opcijama `-n` i `-b`) od `cat`, jer pruža veću kontrolu nad numeracijom

- Pregled važnih opcija:

- b [a|t|n|p R] Prepend numbers to all lines (a), nonblank lines (t), no lines (n), or only lines that contain regular expression R. (Default=a)
- v N Begin numbering with integer N. (Default=1)
- i N Increment the number by N for each line, so for example, you could use odd numbers only (-i2) or even numbers only (-v2 -i2). (Default=1)
- n [ln|rn|rz] Format numbers as left-justified (ln), right-justified (rn), or right-justified with leading zeros (rz). (Default=ln)
- w N Force the width of the number to be N columns. (Default=6)
- s S Insert string S between the line number and the text. (Default=Tab)

head [options] [files]

- Komanda **head** ispisuje prvih 10 linija nekog tekstualnog fajla; to je pogodno za preliminarni pregled njegovog sadržaja:

→ **head myfile**

→ **head * | less** *Preview all files in the current directory*

- Takođe je pogodna za preliminarni pregled izlaza nekog lanca:

→ **ls -lta | head**

Ovako se dobija lista 10 fajlova, u radnom direktorijumu, koji su poslednji modifikovani

- Pregled važnih opcija:

-n N Print the first *N* lines instead of 10.

-N Same as **-n N**.

-c N Print the first *N* bytes of the file.

-q Quiet mode: when processing more than one file, don't print a banner above each file. Normally, head prints a banner containing the filename.

tail [options] [files]

- Komanda **tail** (= rep) ispisuje poslednjih 10 linija nekog fajla:

→ **tail myfile**

→ **nl myfile | tail** *See line numbers too*

U drugoj liniji, linije se prikazuju numerisane

- Posebno je korisna opcija **-f**, zahvaljujući kojoj korisnik može simultano da prati sadržaj fajla dok neki program u njega upisuje novi tekst, to jest da čita nove linije fajla u trenutku

njihovog dodavanja na kraj fajla; to je na primer korisno prilikom praćenja Linux log fajla, dok drugi programi u njega upisuju:

→ **tail -f /var/log/syslog**

- Pregled važnih opcija:

-n N Print the last *N* lines of the file instead of 10.

-N Same as **-n N**.

-n +N Print all lines except the first *N*.

-c N Print the last *N* bytes of the file.

-f Keep the file open, and whenever lines are appended to the file, print them. This is extremely useful. Add the **-retry** option if the file doesn't exist yet, but you want to wait for it to exist.

-q Quiet mode: when processing more than one file, don't print a banner above each file. Normally **tail** prints a banner containing the filename.

strings [options] [files]

- Binarni fajlovi, kao što je pomenuto, ponekad sadrže u sebi neki čitljivi tekst (a sve osim toga je upotrebljivo samo za aplikaciju u kojoj se fajl otvara); komanda **strings** služi da izdvoji i prikaže na ekranu sav čitljivi tekst; ona znači služi kao "Word Extractor"

- Preciznije, ova komanda pregleda binarni fajl redom bajt po bajt; ako naiđe na povorku bajtova koji, interpretirani kao ASCII kodovi, generišu neki čitljiv tekst, to jest neku smislenu reč dužu od 4 slova, onda taj tekst ispisuje kao rezultat

- Primer:

```
→ strings /usr/bin/who
David MacKenzie
Copyright %s %d Free Software Foundation, Inc.
```

- Ovde se može iskombinovati i komanda **grep** (koja pretražuje tekst u fajlu) da bi se iz dobijenog čitljivog teksta dobile neke željene informacije; na primer mogu se iz teksta izvući sve mejl adrese (raspoznaju se po karakteru @):

```
→ strings -n 10 /usr/bin/who | grep '@'
bug-coreutils@gnu.org
```

- Pregled važnih opcija:

```
-n length Display only strings with length greater than length (the
default is 4).
```

od [*options*] [*files*]

- Ako korisnik hoće da pregleda neki binarni fajl i pokuša da pronađe neku korisnu informaciju (osim čitljivog teksta) koja se u njemu nalazi, na raspolaganju mu je komanda **od** ("octal dump" -- eng. dump = đubrište?!); ona znači služi kao "Hex viewer"
- U računarstvu, pojam **dump** (bez srpskog prevoda) označava preuzimanje sirovih podataka, tačno onakvih kako su zapisani (bez ikakvog formatiranja), iz memorije ili sa diska (što je ovde slučaj jer se radi o binarnom fajlu koji se učitava sa diska), i njihovo prikazivanje na ekranu
- Kao što je pomenuto, tako ispisani podaci, u obliku niza heksadecimalnih (ili oktalnih ili decimalnih) brojeva, su veoma teški za razumevanje, tako da ih može interpretirati samo korisnik koji detaljno poznaje format fajla, to jest šta koja ispisana vrednost znači -- u to se ovde neće ulaziti

- Komanda **od**, znači, ispisuje sadržaj binarnog fajla bajt po bajt, pri čemu korisnik može birati brojni sistem u kome će se odgovarajuće brojne vrednosti prikazivati

- Primer:

```
→ od -b /usr/bin/who
0000000 312 376 272 276 000 000 000 002 001 000 000 007 000 000 000 003
0000020 000 000 100 000 000 001 021 120 000 000 000 016 001 000 000 014
0000040 200 000 000 002 000 001 200 000 000 001 021 060 000 000 000 016
...
```

- U primeru, prikazuju se bajtovi binarnog fajla (Linux programa) `/usr/bin/who`; opcija **-b** traži da se prikazuju vrednosti u oktalnom sistemu; po defaultu se prikazuje 16 bajtova u jednoj liniji; prva kolona predstavlja ofset svake linije, takođe u oktalnom sistemu (0,20,40...), čemu odgovara (0,16,32...) u decimalnom sistemu; pomoću ofseta se određuje redni broj bajta u celom fajlu

- Izlazni zapis može se podešavati na mnogo načina (neće se objašnjavati); na primer, pomoću opcije **-c** se zadaje da se, tamo gde je to moguće, brojna vrednost interpretira kao ASCII kôd i prikaže odgovarajući karakter, a u cilju pronalaženja čitljivog teksta

- Kao instruktivniji primer može poslužiti upotreba ove komande na jednom tekstualnom fajlu: ako fajl **primer.txt** samo sadrži tekst `abc`, izlaz će izgledati veoma jednostavno:

```
→ od -b primer.txt
0000000 141 142 143
0000003
```

Kao što je ranije pokazano, oktalno 141 odgovara slovu `a`, i slično za slova `b` i `c`

- Trivijalno, ista komanda pomoću opcije **-c** će dati:

```
→ od -c primer.txt
0000000 a b c
0000003
```

- Na kraju se još može pomenuti komanda **acroread**, koja pokreće Adobe aplikaciju "PDF reader", i koja očitava zadati PDF fajl (koji je takođe binarni); PDF fajlovi se mogu čitati i pomoću komandi **xpdf** i **gv**
- Usput treba napomenuti: Linux ima jednostavne programe za rad sa Microsoft Office dokumentima: **soffice** (svi dokumenti), **abiword** (samo Word) i **gnumeric** (samo Excel) -- neće biti objašnjavani

Tema 9: Kreiranje i editovanje fajlova

- Glavne komande, a istovremeno i nazivi editora, za kreiranje i editovanje/uređenje fajlova su:

nano	A simple text editor included by default in popular Linux distros.
emacs	Text editor from Free Software Foundation.
vim	Text editor, extension of Unix vi.

- Pri radu sa Linuxom, često se javlja potreba za kreiranjem i uređenjem nekog tekstualnog fajla, za šta služi editor teksta; tri najviše korišćena su navedeni: nano, emacs i vim; veoma su slični, i imaju slične funkcionalnosti; korisnik treba da poznaje sva tri, jer se nikad ne zna kada će se za kojim ukazati potreba

- Da bi se kreirao/editovao tekstualni fajl, kuca se jednostavno jedna od sledeće tri linije:

```
→ nano myfile
→ emacs myfile
→ vim myfile
```

Ako fajl myfile ne postoji, ovim će se kreirati, naravno inicijalno kao prazan

9.1 Brzo kreiranje fajla

- Prazan fajl, koji će kasnije služiti za dalje editovanje, može se kreirati pomoću komande **touch**:

→ **touch newfile**

Fajl se ovim ne otvara za uređivanje, već se samo kreira kao potpuno prazan

- Isto to se može uraditi pomoću komande **echo**:

→ **echo -n > newfile2**

Opcija **-n** služi da spreči upisivanje jednog newline karaktera u fajl, kako je po difoltu; tako će kreirani fajl biti potpuno prazan

9.2 Difolt editor

- Razni Linux programi će pokretati editor kada im je to potrebno; difolt editor su obično ili nano ili vim; na primer, program za pisanje i-mejla će automatski otvoriti (neki) editor kada korisnik hoće da napiše novi i-mejl

- U slučaju da korisnik hoće da promeni difolt editor, treba da dodeli vrednosti dvema promenljivama okruženja **VISUAL** i **EDITOR** prema svojoj želji

- Primer:

→ **EDITOR=emacs**
→ **VISUAL=emacs**
→ **export EDITOR VISUAL**

Potrebno je promeniti obe varijable, jer neki programi proveravaju vrednost jedne, a neki vrednost druge; ako korisnik hoće da promeni difolt editor za trajno, treba da promeni vrednosti ovih varijabli u startap fajlu `~/.bash_profile`

nano [options] [files]

- Editor nano je vrlo mali i jednostavan (od čega mu i potiče naziv); ugrađen je u mnoge Linux distribucije (dok složeniji editori emacs i vim ne moraju da budu); za većinu potreba sasvim je dovoljan; da se pokrene, kuca se:

→ **nano**

posle čega se otvara prozor sličan onom sa Slike 9-1



Slika 9-1: Nano editor

- Komande su uglavnom prečice oblika Control+slovo, kao na primer ^o za Save, ili ^x za Quit; najčešće korišćene komande su prikazane na dnu prozora editora; nano koristi neke bizarne termine, kao na primer WriteOut umesto Save
- Neke nano komande uključuju i takozvani meta taster, koji je obično Escape ili Alt; u dokumentaciji je meta taster označen slovom M, tako da na primer M-F znači "pritisni meta taster i taster F"; najbitnije prečice biće date u zajedničkoj tabeli

- Tutorijal za rad u nano editoru:

<https://www.youtube.com/watch?v=45KO4KO2DTo>

dok se dokumentacija nalazi na adresi:

<http://nano-editor.org>

emacs [options] [files]

- Komanda **emacs** pokreće moćno okruženje za editovanje teksta sa hiljadama (!) komandi i opcija; uz to, ono sadrži i kompletni programski jezik (!), koji omogućava da korisnik isprogramira razne funkcionalnosti za editovanje po svojoj želji

- Da se emacs pokrene u novom X prozoru, kuca se:

→ **emacs**

a da se pokrene u postojećem šel prozoru, kuca se:

→ **emacs -nw**

- Za pokretanje ugrađenog emacs tutorijala, kuca se **^h t**; i u radu sa emacs se koriste prečice, uz pomoć tastera Control i meta tastera

vim [options] [files]

- Editor vim je obogaćena verzija starog standardnog Unix editora vi; da se pokrene u novom X prozoru, kuca se:

→ **gvim**

a da se pokrene u postojećem šel prozoru, kuca se:

→ **vim**

Da se pokrene tutorijal, kuca se komanda **vimtutor**

- Ovaj editor može da radi u dva moda (korisnik mora da prebacuje iz jednog u drugi): prvi je **insert** mod, u kome se tekst unosi na uobičajeni način, a drugi je **normal** mod u kome se briše tekst, radi copy/paste, i slično
- Najbitnije prečice za sva tri editora (za vim u normal modu) su prikazani u narednoj tabeli

Task	emacs	nano	vim
Type text	Just type	Just type	Switch to insert mode if necessary, by typing <code>i</code> , then type any text
Save and quit	<code>^x^s</code> then <code>^x^c</code>	<code>^o</code> then <code>^x</code>	<code>:wq</code>
Quit without saving	<code>^x^c</code> Respond “no” when asked to save buffers	<code>^x</code> Respond “no” when asked to save	<code>:q!</code>
Save	<code>^x^s</code>	<code>^o</code>	<code>:w</code>
Save As	<code>^x^w</code>	<code>^o</code> , then type a filename	<code>:w filename</code>
Undo	<code>^/</code> or <code>^x u</code>	<code>M-u</code>	<code>u</code>
Suspend editor (not in X)	<code>^z</code>	<code>^z</code>	<code>^z</code>
Switch to insert mode	(N/A)	(N/A)	<code>i</code>
Switch to command mode	(N/A)	(N/A)	ESC
Switch to command-line mode	<code>M-x</code>	(N/A)	<code>:</code>
Abort command in progress	<code>^g</code>	<code>^c</code>	ESC
Move forward	<code>^f</code> or right arrow	<code>^f</code> or right arrow	<code>l</code> or right arrow
Move backward	<code>^b</code> or left arrow	<code>^b</code> or left arrow	<code>h</code> or left arrow
Move up	<code>^p</code> or up arrow	<code>^p</code> or up arrow	<code>k</code> or up arrow
Move down	<code>^n</code> or down arrow	<code>^n</code> or down arrow	<code>j</code> or down arrow

Task	emacs	nano	vim
Move to next word	M-f	^SPACEBAR	w
Move to previous word	M-b	M-SPACEBAR	b
Move to beginning of line	^a	^a	0
Move to end of line	^e	^e	\$
Move down one screen	^v	^v	^f
Move up one screen	M-v	^y	^b
Move to beginning of document	M-<	M-\	gg
Move to end of document	M->	M-/	G
Delete next character	^d	^d	x
Delete previous character	BACKSPACE	BACKSPACE	X
Delete next word	M-d	(N/A)	de
Delete previous word	M- BACKSPACE	(N/A)	db
Delete current line	^a^k	^k	dd
Delete to end of line	^k	...	D
Define region (type this keystroke to mark the beginning of the region, then move the cursor to the end of the desired region)	^SPACEBAR	^^ (control caret)	v
Cut region	^w	^k	d
Copy region	M-w	M-^	y
Paste region	^y	^u	p
Get help	^h	^g	:help
View the manual	^h i	^g	:help

Tema 10: Svojstva fajlova

- Glavne komande za pregled i promenu svojstava (eng. property) fajlova su:

stat	Display attributes of files and directories.
wc	Count bytes, words, and lines in a file.
du	Measure disk usage of files and directories.
file	Identify (guess) the type of a file.
touch	Change timestamps of files and directories.
chown	Change owner of files and directories.
chgrp	Change group ownership of files and directories.
chmod	Change protection mode of files and directories.
umask	Set a default mode for new files and directories.
chattr	Change extended attributes of files and directories.
lsattr	List extended attributes of files and directories.

- Kada korisnik pregleda Linux fajl (isto važi i za direktorijume), treba da ima u vidu da fajl, pored svog sadržaja poseduje i svoje atribute kao što su: vlasnik, veličina, dozvole pristupa i slično; komanda **ls -l** prikazuje (kao što je pokazano) neke od ovih atributa, ali postoje i druge koje prikazuju i više; postoje i komande koje služe za promenu atributa

stat [options] files

- Komanda **stat** ("statistics") ispisuje detaljnu listu atributa zadatog fajla, i njihovih vrednosti; primer:

```
→ stat myfile  
  File: 'myfile'  
  Size: 1168           Blocks: 8
```

```
IO Block: 4096  regular file
Device: 811h/2065d      Inode: 37224455    Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 600/lisa)
      Gid: ( 620/users)
Access: 2015-11-07 11:15:14.766013415 -0500
Modify: 2015-11-07 11:15:14.722012802 -0500
Change: 2015-11-07 11:15:14.722012802 -0500
Birth: -
```

- Prikazani atributi od interesa su:

>> File: ime fajla (myfile), Size: veličina u bajtovima (1168), Blocks: veličina u blokovima (8)

>> Tip fajla (ovde regular file, može biti i directory)

>> Access: dozvole pristupa -- u oktalnom sistemu (0644), i u ranije objašnjenom formatu kakvog daje komanda `ls -l (-rw-r--r--)`

>> Uid: UID vlasnika fajla (600), odgovarajuće korisničko ime (lisa)

>> Gid: GID vlasnika fajla (620), odgovarajuće grupno ime (users)

>> Access, Modify, Change: vreme (eng. timestamp) poslednjeg pristupa fajlu, poslednje modifikacije fajla, i poslednje promene statusa fajla

- Pomoću opcije `-t`, ispisuju se vrednosti atributa ali u jednoj liniji (i u malo izmenjenom redosledu), i bez naziva atributa, što je pogodno za dalje preuzimanje i obradu (na primer, u šel skriptu); primer:

```
→ stat -t myfile
myfile 1168 8 81a4 600 620 811 37224455 1 0 0
      1446912914 1446912914 1446912914 0 4096
```

Tri velike celobrojne vrednosti u poslednjem redu su tri gornja vremena (Access, Modify, Change), napisana u formatu Unix time (bez daljih objašnjenja)

- Pregled važnih opcija:

- L Follow symbolic links and report on the file they point to.
- f Report on the filesystem containing the file, not the file itself.
- t Terse mode: print information on a single line.

wc [options] [files]

- Komanda **wc** ("word count") ispisuje broj linija, reči i bajtova u tekstualnom fajlu:

```
→ wc myfile
18 211 1168 myfile
```

Ovaj fajl sadrži 18 linija, 211 reči (pod tim pojmom se podrazumeva string odvojen sa obe strane belim karakterima, a ne reč kao jedinica informacije), i 1168 bajtova

- Pregled važnih opcija:

- l Print the line count only.
- w Print the word count only.
- c Print the byte count only.
- L Locate the longest line in each file and print its length in bytes.

du [options] [files| directories]

- Komanda **du** ("disk usage") je bitna za svakog administratora, jer je jedan od njegovih zadataka da prati količinu memorije na disku koju zauzimaju podaci: po difoltu, **du** ispisuje sve direktorijume koje sadrži radni direktorijum, broj blokova za svaki, i sumu na dnu (to jest ukupnu veličinu radnog direktorijuma)

- Primer:

```
→ du
36  ./Mail
340 ./Files/mine
40  ./Files/bob
416 ./Files
216 ./PC
2404 .
```

- Istá komanda služi i da prikáže veličinu fajla (takode u blokovima):

```
→ du myfile emptyfile hugefile
4      myfile
0      emptyfile
18144  hugefile
```

- Problem sa ovakvim izlazom je što pojam "memorijski blok" nije precizno definisan, tako da prikazana informacija praktično nije od koristi; zato se moraju dodati opcije, kao na primer `-h` koja diktira da ispis bude u uobičajenim jedinicama (za 1 gigabajt ispisaće 1G, a za 1 kilobajt ispisaće 1K i slično)

- Pregled važnih opcija:

```
-b      Measure usage in bytes.
-k      Measure usage in kilobytes.
-m      Measure usage in megabytes.
-B N    Display sizes in blocks that you define, where 1 block = N bytes.
        (Default = 1024)
-h -H   Print in human-readable units. For example, if two directories are of size
        1 gigabyte or 25 kilobytes, respectively, du -h prints 1G and 25K. The
        -h option uses powers of 1024, whereas -H uses powers of 1000.
-c      Print a total in the last line. This is the default behavior when measuring
        a directory, but for measuring individual files, provide -c if you want a
        total.
```

- L Follow symbolic links and measure the files they point to.
- s Print only the total size.

file [options] files

- Komanda **file** detaljno ispituje podatke o tipu/formatu zadatog fajla, to jest sve ono šta Linux iz sadržaja fajla može saznati; primer:

```
→ file /etc/hosts /usr/bin/who letter.docx
/etc/hosts:      ASCII text
/usr/bin/who:    ELF 64-bit LSB executable ...
letter.docx:     Microsoft Word 2007+
```

- Pregled važnih opcija:

- b Omit filenames (left column of output).
- i Print MIME types for the file, such as "text/plain" or "audio/mpeg", instead of the usual output.
- f *name_file* Read filenames, one per line, from the given *name_file*, and report their types. Afterward, process filenames on the command line as usual.

touch [options] files

- Komanda **touch** postavlja dva vremena (eng. timestamp) vezana za zadati fajl: vreme kada mu je sadržaj poslednji put promenjen (modification time), i vreme kada mu je poslednji put pristupljeno, to jest kada mu je sadržaj očitao (access time)

- Da se oba zapisa postave na trenutno (aktuelno) vreme, kuca se:

```
→ touch myfile
```

a da se postave na željeno vreme, kuca se na primer:

→ **touch -d "November 18 1975" myfile**

Ako navedeni fajl ne postoji, na ovaj način se kreira novi (prazan) pod tim imenom

- Pregled važnih opcija:

- a Change the access time only.
- m Change the modification time only.
- c If the file doesn't exist, don't create it (normally, touch creates it).
- d *timestamp* Set the file's timestamp(s). A tremendous number of timestamp formats are acceptable, from "12/28/2001 3pm" to "28-May" (the current year is assumed, and a time of midnight) to "next tuesday 13:59" to "0" (midnight today).

chown [options] user_spec files

- Komanda **chown** ("change owner") definiše vlasništvo (eng. ownership) korisnika i grupe nad fajlovima i direktorijumima; primer: da bi se korisnik smith postavio kao vlasnik dva fajla i jednog direktorijuma, kuca se:

→ **sudo chown smith myfile myfile2 mydir**

- Pomoću parametra **user_spec**, vlasnik se može zadati na različite načine:

- 1) pomoću korisničkog imena (ili UID), na primer: **chown smith myfile**
- 2) pomoću korisničkog imena (ili UID), posle koga ide dvotačka, posle koje ide ime grupe (ili GID), da se kao vlasnici postave i korisnik i grupa, na primer: **chown smith:users myfile**
- 3) pomoću imena grupe (ili GID), pre koga ide dvotačka, da se postavi samo grupa kao vlasnik, na primer: **chown :users myfile**
- 4) pomoću opcije **--reference=file** da se postave isti vlasnici (korisnik i grupa) kao što su postavljeni za fajl file dat kao referenca (po ugledu na taj fajl)

- Pregled važnih opcija:

--dereference Follow symbolic links and operate on the files they point to.

-R Recursively change permissions within a directory hierarchy.

chgrp [options] group_spec files

- Komanda **chgrp** ("change group") definiše vlasništvo grupe nad fajlovima i direktorijumima:

→ **chgrp friends myfile myfile2 mydir**

- Parametar **group_spec** može biti:

1) naziv grupe ili njen GID

2) **--reference=file**, da se postavi isto vlasništvo kao što je postavljeno za zadati fajl

- Pregled važnih opcija:

--dereference Follow symbolic links and operate on the files they point to.

-R Recursively change the ownership within a directory hierarchy.

chmod [options] permissions files

- Komanda **chmod** ("change mode") služi da se dodeljuju dozvole pristupa (**r**, **w**, **x**, kao što je već objašnjeno) fajlovima i direktorijumima; dozvole se dodeljuju: korisniku-vlasniku fajla (oznaka **u**), grupi-vlasniku fajla (oznaka **g**), svima ostalima (oznaka **o**, od Others), ili svim korisnicima istovremeno (oznaka **a**, od All, što je difolt)

- Što se tiče argumenta *permissions* ove komande, on definiše dozvolu pristupa pomoću znaka jednakosti, na primer:

→ **chmod u=rwx,g=r,o= new_file.txt**

Na ovaj način, korisnik-vlasnik dobija dozvolu da očitava, upisuje i izvršava fajlove; grupa-vlasnik samo da očitava; ostali nemaju nikakvu dozvolu

- Mogu se, radi kraćeg zapisa, praviti i kombinacije slova, kao na primer: **go=r**; umesto kombinacije **ugo** koristi se pomenuto **a**

- Treba imati u vidu da se ovakvom dodelom sve stare (već postojeće) dozvole pristupa poništavaju (resetuju), i da ubuduće važe samo one koje su ovom komandom postavljene; ovakav način dodele dozvola se naziva **apsolutni**

- Postoji i **relativni** način dodele dozvola: on definiše samo promenu u odnosu na već postojeće stanje, pomoću operatora + i – na sledeći način: + da se doda nova dozvola, – da se oduzme postojeća dozvola

- Primer:

→ **chmod a+x new_script.sh**

Na ovaj način, svi korisnici (uz sve ono šta već imaju od dozvola) dobijaju i dozvolu za izvršenje pomenutog fajla; ako, na primer, kasnije bude trebalo da se samo od korisnika-vlasnika ova dozvola oduzme (a svima osim njega da ostane), kucala bi se ista linija sa **u-x**

- Postoji još jedan način apsolutne dodele dozvola, pomoću oktalnih brojeva, ali se neće uvoditi kao nepraktičan

- Kao i kod prethodne komande, i kod **chmod** argument *permissions* može biti oblika **--reference=file**, čime se sve dozvole za željeni fajl dodeljuju po uzoru na referentni fajl

- Pregled važnih opcija:

-R Recursively change the ownership within a directory hierarchy.

chattr [options] [+ - =]attributes [files]

- Postoje još neki dodatni atributi fajlova (pored onih vezanih za dozvole pristupa), koji se podešavaju pomoću komande **chattr** ("change attributes"); kao i kod prethodne komande, može se raditi apsolutna dodela (pomoću =) i relativna dodela (pomoću + i -)

- Primer:

→ **chattr +ui myfile**

Pomoću ove linije, fajl myfile (pored ostalih atributa koje poseduje) postaje takav da se ne može obrisati (eng. undeletable) a ni menjati (eng. immutable); ovo je vrlo praktično iz očiglednih razloga

- Lista dodatnih atributa i njihovog značenja data je u sledećoj tabeli:

Attribute	Meaning
a	Append-only: appends are permitted to this file, but it cannot otherwise be edited. Root only.
A	Accesses not timestamped: accesses to this file don't update its access timestamp (atime).
c	Compressed: data is transparently compressed on writes and uncompressed on reads.
d	Don't dump: tell the dump program to ignore this file when making backups

i	Immutable: file cannot be changed or deleted (root only).
j	Journalled data (ext3 filesystems only).
s	Secure deletion: if deleted, this file's data is overwritten with zeros.
S	Synchronous update: changes are written to disk immediately.
u	Undeleteable: file cannot be deleted.

- Pregled važnih opcija:

-R Recursively process directories.

- Uz poslednju opciju, može se na primer podesiti da svi fajlovi u direktorijumu test-dir postanu nepromenljivi:

→ chattr -R +i ./test-dir/

lsattr [options] [files]

- Dodatni atributi koji su postavljeni pomoću prethodne komande mogu biti pregledani pomoću komande **lsattr** ("list attributes"); u izlazu se koriste iste oznake (slova iz gornje tabele) kao za prethodnu komandu

- Primer:

→ lsattr myfile
-u--i--- myfile

Ako se fajl ne navede, ova komanda će ispisati attribute svih fajlova u radnom direktorijumu

- Pregled važnih opcija:

- R Recursively process directories.
- a List all files, including those whose names begin with a dot.
- d If listing a directory, do not list its contents, just the directory itself.

Tema 11: Pronalaženje fajlova

- Glavne komande za pronalaženje/lociranje/pretragu (eng. location) fajlova su:

find	Locate files in a directory hierarchy.
xargs	Process a list of located files (and much more).
locate	Create an index of files, and search the index for string.
which	Locate executables in your search path (command).
type	Locate executables in your search path (bash built-in).
whereis	Locate executables, documentation, and source files.

- Jedan Linux sistem može da sadrži na hiljade fajlova: postavlja se pitanje kako pronaći gde se nalazi (lokalizovati) željeni fajl; prvi korak u cilju lakšeg pronalaženja je da se fajlovi i direktorijumi (naravno, oni kojima korisnik može da upravlja) organizuju na logičan način -- tako će korisnik znati gde je fajl i bez pretrage
- Najprostija komanda za pretraživanje je **find**: ova komanda, pomoću tehnike "grube sile", pretražuje redom, fajl po fajl, kroz celu hijerarhiju/stablo direktorijuma i fajlova (to jest kroz ceo fajlsistem)
- Mnogo brža komanda je **locate**: ona pretražuje prethodno pripremljeni indeks (registar direktorijuma i fajlova); neke Linux distribucije po difoltu svake noći generišu indeks, inače korisnik treba sâm da ga apdejtuje

- Za pronalaženje programa (izvršnih fajlova) služe komande **which** i **type**: one pretražuju sve direktorijume u šelovoj putanji pretrage (eng. shell search path) -- to je sekvenca direktorijuma u kojima šel, kada mu se zada neka komanda, traži odgovarajući program
- Brža od pomenute dve pomenute komande je **type**; ona je ugrađena u bash šel (i stoga je raspoloživa samo kada je pokrenut bash); može da detektuje i alias; komanda **which** je program (obično /usr/bin/which)
- Konačno, komanda **whereis** pretražuje poznati skup direktorijuma, umesto putanju pretrage kao prethodne dve

find [directories] [expression]

- Komanda **find** pretražuje jedan ili više direktorijuma (i rekurzivno njihove poddirektorijume), i u njima traži fajlove koji ispunjavaju zadate kriterijume; ona je veoma moćna (sadrži više od 50 opcija!), ali ima i prilično neobičnu sintaksu
- Sledi nekoliko tipičnih primera korišćenja ove komande, u kojima se pretražuje ceo fajlsistem počev od radnog direktorijuma (označenog tačkom)

- Traži se fajl pod nazivom myfile u radnom direktorijumu (detalji u nastavku):

```
→ find . -type f -name myfile -print
./myfile
```

- Traže se svi fajlovi čiji naziv počinje sa "myfile":

```
→ find . -type f -name myfile\* -print
./myfile.zip
./myfile3
./myfile
./myfile2
```

Obratiti pažnju da komanda zahteva da wildcard karakter `*` bude eskejpovan, da bi ga šel kao bukvalan karakter zadao komandi `find`

- Traže se i ispisuju nazivi svih direktorijuma:

```
→ find . -type d -print
```

```
.  
./jpegexample  
  
./dir2  
./mydir  
./mydir/dir
```

- Pregled važnih opcija za zadavanje kriterijuma pretrage:

<code>-name <i>pattern</i></code>	The name (<code>-name</code>), pathname (<code>-path</code>), or symbolic link
<code>-path <i>pattern</i></code>	target (<code>-lname</code>) of the desired file must match this shell
<code>-lname <i>pattern</i></code>	pattern, which may include shell wildcards <code>*</code> , <code>?</code> , and <code>[]</code> .

(You must escape the wildcards, however, so they are ignored by the shell and passed literally to `find`.) Paths are relative to the directory tree being searched.

<code>-iname <i>pattern</i></code>	The <code>-iname</code> , <code>-ipath</code> , and <code>-ilname</code> options are the
<code>-ipath <i>pattern</i></code>	same as <code>-name</code> , <code>-path</code> , and <code>-lname</code> , respectively, but
<code>-ilname <i>pattern</i></code>	are case-insensitive.

<code>-regex <i>regexp</i></code>	The path (relative to the directory tree being searched) must match the given regular expression.
-----------------------------------	---

<code>-type <i>t</i></code>	Locate only files of type <i>t</i> . This includes plain files (<code>f</code>), directories (<code>d</code>), symbolic links (<code>l</code>), block devices (<code>b</code>), character devices (<code>c</code>), named pipes (<code>p</code>), and sockets (<code>s</code>).
-----------------------------	---

<code>-atime <i>N</i></code>	File was last accessed (<code>-atime</code>), last modified
<code>-ctime <i>N</i></code>	(<code>-mtime</code>), or had a status change (<code>-ctime</code>) exactly <i>N</i>
<code>-mtime <i>N</i></code>	*24 hours ago. Use <code>+N</code> for "greater than <i>N</i> ," or <code>-N</code> for "less than <i>N</i> ."

<code>-amin <i>N</i></code>	File was last accessed (<code>-amin</code>), last modified (<code>-mmin</code>),
<code>-cmin <i>N</i></code>	or had a status change (<code>-cmin</code>) exactly <i>N</i> minutes ago.
<code>-mmin <i>N</i></code>	Use <code>+<i>N</i></code> for "greater than <i>N</i> ," or <code>-<i>N</i></code> for "less than <i>N</i> ."
<code>-anewer</code>	File was accessed (<code>-anewer</code>), modified (<code>-newer</code>), or
<code><i>other_file</i></code>	had a status change (<code>-cnewer</code>) more recently than
<code>-cnewer</code>	<code><i>other_file</i></code> .
<code><i>other_file</i></code>	
<code>-newer</code>	
<code><i>other_file</i></code>	
<code>-maxdepth <i>N</i></code>	Consider files at least (<code>-mindepth</code>) or at most
<code>-mindepth <i>N</i></code>	(<code>-maxdepth</code>) <i>N</i> levels deep in the directory tree being searched.
<code>-follow</code>	Dereference symbolic links.
<code>-depth</code>	Proceed using depth-first search: completely search a directory's contents (recursively) before operating on the directory itself.
<code>-xdev</code>	Limit the search to a single filesystem (i.e., don't cross device boundaries).
<code>-size <i>N</i> [bckw]</code>	Consider files of size <i>N</i> , which can be given in blocks (b), one-byte characters (c), kilobytes (k), or two-byte words (w). Use <code>+<i>N</i></code> for "greater than <i>N</i> ," or <code>-<i>N</i></code> for "less than <i>N</i> ."
<code>-empty</code>	File has zero size, and is a regular file or directory.
<code>-user <i>name</i></code>	File is owned by the given user.
<code>-group <i>name</i></code>	File is owned by the given group.

- Na kraju, navedene opcije kojima se zadaju kriterijumi pretrage mogu se i kombinovati pomoću logičkih operatora i, ili, ne; prilikom tog kombinovanja, mogu se koristiti i zagrade (koje se moraju eskejpovati)

- Oznaka za operator "i" je **-a** (od "and", ovo je i difolt); oznaka za operator "ili" je **-o** (od "or"); oznaka za negaciju je uzvičnik **!**; na taj način se grade složeni kriterijumi pretrage:

>> pomoću logičkog i: **option1 -a option2**

>> pomoću logičkog ili: **option1 -o option2**

>> pomoću negacije: **! option**

- Primer:

→ find . -type f \(! -user jsmith -mtime +30 \) -print

Traže se svi fajlovi čiji vlasnik **nije** jsmith, i koji su modifikovani pre više od 30 dana; kombinovani uslov napisan je u (eskejpovanim) zagradama

- Pregled važnih opcija za prikazivanje rezultata pretrage (mada se nad njima mogu obavljati i druge operacije):

-print	Simply print the path to the file, relative to the search directory.
-print0	Like -print , but instead of separating each line of output with a newline character, use a null (ASCII 0) character. Use when piping the output of find to another program.

xargs [options] [command]

- Kao što je već objašnjeno, procedura ulančavanja omogućava da izlaz (preciznije, stdout) jedne komande (programa) postane ulaz (preciznije, stdin) druge; ova procedura se naziva i tok podataka (eng. data stream)
- Međutim, neke komande ne prihvataju stdin, već podatke (koje treba da obrade) zahtevaju kao **argumente** -- neće se dalje objašnjavati; stoga takve komande ne mogu biti

deo lanca; u tom slučaju, od pomoći može biti komanda **xargs** (skraćeno od "extended arguments"): ona se piše ispred komande koja ne prihvata stdin:

→ **command_1 | xargs command_2**

- Komanda **xargs** konvertuje stdout komande **command_1**, odnosno stdin namenjen za **command_2**, u njen argument, čime se postiže da **command_2** obavi svoj zadatak; **xargs** konverziju odrađuje tako da dobijeni argument bude prilagođen za **command_2**; usput, ako se **command_2** ne navede, podrazumeva se **echo**

- Primer: korisnik je kreirao tekstualni fajl pod imenom **important**, u kome je napisao listu od tri fajla, jedan naziv po liniji:

```
→ cat important
/home/jsmith/mail/love-letters
/usr/local/lib/critical_stuff
/etc/passwd
```

- Pokušaj korisnika da pomoću ulančavanja na svaki fajl sa date liste primeni komandu **ls -l** ne bi bio uspešan, jer komanda **ls** ne prihvata stdin već zahteva argument; stoga korisnik mora da upotrebi komandu **xargs**:

```
→ cat important | xargs ls -l
```

Znači, komanda **xargs** će dobijenu listu fajlova zadati komandi **ls -l** kao argument

- Slično, svaki od ovih fajlova može da se prikaže pomoću komande **less**:

```
→ cat important | xargs less
```

ili da se obriše pomoću **rm**:

```
→ cat important | xargs rm -f    Careful! Destructive!
```

- I komanda **grep** zahteva upotrebu **xargs**; na primer, da bi se u radnom direktorijumu pronašli svi tekstualni fajlovi koji u svom sadržaju imaju reč "tomato", radi se sledeće:

```
→ find . -type f -print | xargs grep -l tomato
./findfile1
./findfile2
→ cat findfile1
This file contains the word tomato.
```

- U primeru, prvo komanda **find** generiše listu svih tekstualnih fajlova, a zatim (u okviru lanca) komanda **grep** pretražuje svaki od tih fajlova sa liste (i string "tomato" pronade u dva); "vezu" između njih je napravila komanda **xargs**

- Pregled važnih opcija:

- n *k* Feed *k* lines of input to each executed command. The common -n1 guarantees that each execution will process only one line of input. Otherwise, **xargs** may pass multiple lines of input to a single command.
- 0 Set the end-of-line character for input to be ASCII zero rather than whitespace, and treat all characters literally. Use this when the input is coming from **find -print0**.

locate [options]

- Komanda **updatedb** kreira/apdejtuje indeks (registar) svih fajlova i direktorijuma; taj indeks se brzo pretražuje pomoću "partnerske" komande **locate**; neke distribucije nemaju instalirane ovaj program, pa se mora instalirati paket **mlocate** koji sadrži ovu komandu
- Ovo je posebno korisno kada se radi mnogo pretraga, u hijerarhiji koja se ne menja često (u suprotnom, za pretragu jednog fajla ili kompleksniju pretragu bolje je koristiti komandu **find**); neke Linux distribucije same generišu indeks, u kom slučaju je dovoljno koristiti samo **locate**

- Za kreiranje indeksa nekog direktorijuma `directory` (i svih njegovih poddirektorijumima), i čuvanje tog indeksa u fajlu `/tmp/myindex`, kuca se sledeće:

→ **updatedb -l0 -U *directory* -o */tmp/myindex***

a zatim se u indeksu traži string (na primer, naziv fajla) pomoću:

→ **locate -d */tmp/myindex* *string***

- Postoji i jedna sigurnosna opcija: moguće je kreirati indeks takav da se prilikom njegove pretrage prikažu samo oni fajlovi za koje je korisniku dozvoljeno da ih vidi; tako da ako superkorisnik kreira indeks nekog zaštićenog direktorijuma, običan korisnik može da obavi pretragu ali neće videti zaštićene fajlove

- Ovo se postiže tako što se izostavi opcija `-l0` komande **updateb** (čime se aktivira sigurnosni režim rada), i što se komanda pokrene kao superkorisnik:

→ **sudo updatedb -U *directory* -o */tmp/myindex***

- Pregled važnih opcija za **updatedb**:

-u	Create index from the root directory downward.
-U <i>directory</i>	Create index from <i>directory</i> downward.
-l (0 1)	Turn security off (0) or on (1). The default is 1.
-e <i>directories</i>	Exclude one or more directories from the index. Separate their paths by commas.
-o <i>outfile</i>	Write the index to file <i>outfile</i> .

- Pregled važnih opcija za **locate**:

-d <i>index</i>	Indicate which index to use (in our example, <i>/tmp/myindex</i>).
-i	Case-insensitive search.
-r <i>regex</i>	Search for files matching the given regular expression.

which file

- Komanda **which** locira izvršni (eng. executable) fajl, to jest program, u šelovoj putanji pretrage

- Primer:

```
→ which who  
/usr/bin/who
```

Prikazuje se lokacija programa **who**

- Ako postoji više izvršnih programa sa istim imenom (na primer, /usr/bin/who i /usr/local/bin/who), **which** će se zaustaviti kada nađe na prvi od njih i ispiše obaveštenje

type [options] commands

- Komanda **type** se koristi isto kao i **which**, na primer:

```
→ type grep who  
grep is /bin/grep  
who is /usr/bin/who
```

ali se razlikuju u tome što je prva ugrađena (builtin) u bash šel, a druga je program na disku

- Ta razlika se može uočiti, na primer, na sledeći način:

```
→ type which type rm if  
which is /usr/bin/which  
type is a shell builtin  
rm is aliased to `/bin/rm -i'  
if is a shell keyword
```

(za komparaciju su dodate i komande **rm** i **if**)

- Pošto je ugrađena, **type** je brža od **which**, ali je raspoloživa samo u nekim šelovima (kao što je bash), dok u nekima nije

whereis [options] files

- Komanda **whereis** pokušava da locira zadati fajl tako što pretražuje jednu predefinisanu listu direktorijuma; može da locira izvršne fajlove, dokumentaciju, i sors kodove; očigledna mana ove komande je što je moguće da se na toj listi ne nalazi direktorijum u kome se traženi fajl nalazi

- Primer – traži se izvršni fajl vim:

→ **whereis vim**

vim: /usr/bin/vim /etc/vim /usr/share/vim ...

- Pregled važnih opcija:

-b	List only executables (-b), manpages (-m), or source code
-m	files (-s).
-s	
-B dirs... -f	Search for executables (-B), manpages (-M), or source code
-M dirs... -f	files (-S) only in the given directories. You must follow the
-S dirs... -f	directory list with the -f option before listing the files you seek.

Tema 12: Rad sa tekstom u fajlovima

- Glavne komande za rad sa tekstom u fajlovima (eng. file text manipulation) su:

grep	Find lines in a file that match a regular expression.
cut	Extract columns from a file.
paste	Append columns.

tr	Translate characters into other characters.
expand, unexpand	Convert between tabs and spaces.
sort	Sort lines of text by various criteria.
uniq	Locate identical lines in a file.
tee	Copy a file <i>and</i> print it on standard output, simultaneously.

- Linux je veoma moćan kada je u pitanju rad sa tekstom, kao što je na primer modifikacija tekstualnog fajla, pri čemu se često koristi ulančavanje; svaki program koji očitava standardni ulaz i upisuje u standardni izlaz spada u ovu grupu

grep [options] pattern [files]

- Komanda **grep**, već pomenuta, je od velike koristi svakom korisniku; ona, u zadatom fajlu (ili fajlovima) traži zadati string; kao rezultat, ona ispisuje sve (kompletne) linije koje sadrže taj string

- Na primer, ako fajl randomlines sadrži sledeći tekst:

```
The quick brown fox jumped over the lazy dogs!
My very eager mother just served us nine pancakes.
Film at eleven.
```

i ako se u njemu traži string (tekst) "pancake" (preciznije, traže se sve linije koje ga sadrže), piše se sledeće:

```
→ grep pancake randomlines
My very eager mother just served us nine pancakes.
```

- String koji se traži u tekstu može biti zadat i na još jedan način, koji je fleksibilniji i stoga pruža još veće mogućnosti pretraživanja: pomoću "regularnih izraza" koji se kraće nazivaju **regeksi** (od eng. regular expression = regex)

- Regeksi su prikazani u narednoj tabeli (u kojoj druga kolona nije od interesa, jer opisuje Extended regekse -- oni neće biti obrađivani, već samo obični -- Plain):

Regular expression		
Plain	Extended	Meaning
.		Any single character.
[...]		Match any single character in this list.
[^...]		Match any single character NOT in this list.
(...)		Grouping.
\		Or.
^		Beginning of a line.
\$		End of a line.
\<		Beginning of a word.
\>		End of a word.
[:alnum:]		Any alphanumeric character.
[:alpha:]		Any alphabetic character.
[:cntrl:]		Any control character.
[:digit:]		Any digit.
[:graph:]		Any graphic character.
[:lower:]		Any lowercase letter.
[:print:]		Any printable character.
[:punct:]		Any punctuation mark.

<code>[:space:]</code>		Any whitespace character.
<code>[:upper:]</code>		Any uppercase letter.
<code>[:xdigit:]</code>		Any hexadecimal digit.
<code>*</code>		Zero or more repetitions of a regular expression.
<code>\+</code>	<code>+</code>	One or more repetitions of a regular expression.
<code>\?</code>	<code>?</code>	Zero or one occurrence of a regular expression.
<code>\{n\}</code>	<code>{n}</code>	Exactly <i>n</i> repetitions of a regular expression.
<code>\{n, \}</code>	<code>{n, }</code>	<i>n</i> or more repetitions of a regular expression.
<code>\{n, m \}</code>	<code>{n, m}</code>	Between <i>n</i> and <i>m</i> (inclusive) repetitions of a regular expression, <i>n</i> < <i>m</i> .
<code>\c</code>		The literal character <i>c</i> , even if <i>c</i> is a special regular expression character. For example, use <code>*</code> to match an asterisk or <code>\\</code> to match a backslash. Alternatively, put the literal character in square brackets, like <code>[*]</code> or <code>[\\]</code> .

- Na primer, ako u navedenom fajlu treba pronaći linije koje se završavaju uzvičnikom, kuca se:

→ `grep '\!$' randomlines`

The quick brown fox jumped over the lazy dogs!

Regeks se piše pod polunavodnicima; pošto uzvičnik ima specijalno značenje, on je eskejpovan; znak `$` (prema tabeli) zahteva da zadati karakter koji se traži (u ovom slučaju uzvičnik) bude poslednji karakter linije

- Još dva primera pretrage pomoću regeksa:

→ `grep 'k..a' file.txt`

nalazi sve stringove koji počinju slovom *k*, za kojim slede dva bilo koja karaktera (tačka označava "bilo koji karakter"), za kojima sledi slovo *a* (pa će na primer pronaći reči kola, kosa i kapa)

→ **grep 'ko[ls]a' file.txt**

nalazi stringove kola i kosa: u uglastoj zagradi se pišu svi karakteri koji dolaze u obzir za pretragu

• Pregled važnih opcija komande **grep**:

-v	Print only lines that <i>do not</i> match the regular expression.
-l	Print only the <i>names</i> of files that contain matching lines, not the lines themselves.
-L	Print only the names of files that <i>do not</i> contain matching lines.
-c	Print only a count of matching lines.
-n	In front of each line of matching output, print its original line number.
-b	In front of each line of matching output, print the byte offset of the line in the input file.
-i	Case-insensitive match.
-w	Match only complete words (i.e., words that match the entire regular expression).
-x	Match only complete lines (i.e., lines that match the entire regular expression). Overrides -w .
-A <i>N</i>	After each matching line, print the next <i>N</i> lines from its file.
-B <i>N</i>	Before each matching line, print the previous <i>N</i> lines from its file.
-C <i>N</i>	Same as -A <i>N</i> -B <i>N</i> : print <i>N</i> lines (from the original file) above <i>and</i> below each matching line.

- `--color=always` Highlight the matched text in color, for better readability.
- `-r` Recursively search all files in a directory and its subdirectories.
- `-F` Use lists of fixed strings instead of regular expressions. See `fgrep`.

`cut -(b|c|f)range [options] [files]`

- Komanda `cut` izdvaja kolone teksta iz datog fajla; najprostiji je slučaj kada su kolone odvojene nekim delimiterom (tipično, Tab ili zarez), pa se zadaje redni broj kolona koje treba izdvojiti: tada se koristi opcija `-f` (da označi redni broj kolone, to jest polja = field)
- Primer: zadat je fajl u formatu CSV (comma-separated values), to jest delimiter je zarez (posle koga nema razmaka, kao što je to pravilo u normalnom tekstu):

```
→ cat data.csv
one,two,three,four,five,six,seven
ONE,TWO,THREE,FOUR,FIVE,SIX,SEVEN
1,2,3,4,5,6,7
```

- Ako korisnik hoće da iz fajla izdvoji samo 5. kolonu, kuca sledeće:

```
→ cut -f5 -d, data.csv
five
FIVE
5
```

Opcija `-d`, definiše da je delimiter zarez; difolt delimiter je Tab; opcija `-f5` definiše da treba izdvojiti peto polje

- Broj kolona koje se izdvajaju nije ograničen na jedan: korisnik može zadati opseg pomoću crtice (na primer, `-f 3-16`), sekvencu odvojenu zarezima (`-f 3,4,5,6,8,16`) ili

oba istovremeno (-f 3,4,8-16); u opsegu, ako se izostavi prvi broj, podrazumeva se 1, ako se izostavi drugi broj, podrazumeva se sve do kraja linije

- Kolona može biti definisana i rednim brojem karaktera sa leva na desno, za šta služi opcija **-C**; primer:

```
→ cut -c19 myfile
```

Ovo znači da se izdvojiti kolona koju čine 19. karakteri iz svake linije (vrste)

- Naredni primer pokazuje kako se mogu obraditi podaci o svim korisnicima, smešteni u fajlu `/etc/passwd` (vlasnik tog fajla je superkorisnik); svakom korisniku posvećena je jedna linija -- Slika 12-1

```

Output

mark:x:1001:1001:mark,,,:/home/mark:/bin/bash
[--] - [--] [--] [-----] [-----] [-----]
|      |      |      |      |      |      |
|      |      |      |      |      |      +--> 7. Login shell
|      |      |      |      |      +-----> 6. Home directory
|      |      |      +-----> 5. GECOS
|      |      +-----> 4. GID
|      +-----> 3. UID
|      +-----> 2. Password
+-----> 1. Username

```

Slika 12-1: Primer jedne linije fajla /etc/passwd

- Objašnjenje polja (delimiter je dvotačka):

1. Korisničko ime
2. Kriptovana lozinka (x znači da se lozinka čuva u fajlu /etc/shadow)
3. ID korisnika (UID)
4. ID grupe (GID)
5. Puno ime i podaci o korisniku (GECOS -- bez daljih objašnjenja)
6. Korisnikov home direktorijum
7. Šel koji se pokreće kada se korisnik uloguje

- Da bi se dobila lista svih korisničkih imena, kuca se:

```
→ cut -d: -f1 /etc/passwd
```

- Komanda **cut** se često koristi tako što prihvata izlaz neke druge komande u lancu; na primer, izdvajanje druge reči u datoj rečenici postiže se pomoću:

```
→ echo "Lorem ipsum dolor sit amet" | cut -d ' ' -f 2  
ipsum
```

U okviru jedne rečenice teksta, delimiter između reči je razmak, koji je stoga napisan između polunavodnika

- Pregled važnih opcija:

-d C Use character *C* as the *input* delimiter character between fields for the **-f** option. By default it's a tab character.

--output-delimiter=C Use character *C* as the *output* delimiter character between fields for **-f**. By default it's a tab character.

paste [options] [files]

- Komanda **paste** radi suprotno od prethodne: ona tretira zadate fajlove kao kolone i kombinuje ih na standardnom izlazu; primer:

```
→ cat letters  
A  
B  
C  
→ cat numbers  
1  
2
```

```

3
4
5
→ paste numbers letters
1 A
2 B
3 C
4
5

```

- Redosled kombinovanja teksta iz fajlova je određen redosledom kojim su fajlovi napisani posle komande **paste**; broj linija (vrsta) u dva fajla očigledno ne mora biti isti

- Pregled važnih opcija:

-d delimiters Use the given *delimiters* characters between columns; the default is a tab character. Provide a single character (-d :) to be used always, or a list of characters (-dxyz) to be applied in sequence on each line (the first delimiter is x, then y, then z, then x, then y, ...).

-s Sideways: transpose the rows and columns of output:

```

→ paste -s letters numbers
A   B   C
1   2   3   4   5

```

tr [options] charset1 [charset2]

- Komanda **tr** ("transform") obavlja neke jednostavne ali korisne transformacije datog teksta; komanda ne može da očitava standardni ulaz, pa se mora koristiti u okviru lanca ili preusmeravanja

- Komandi se zadaju dva skupa karaktera (eng. charset): **charset1** i **charset2**, koji su u najprostijem slučaju iste dužine; komanda radi tako što će prvi karakter iz **charset1**

pretvoriti u prvi karakter iz **charset2**, i tako redom; radi bolje preglednosti, preporuka je da se ovi skupovi pišu pod polunavodnicima (mada komanda radi i bez njih)

- Primer:

```
→ echo 'example' | tr 'ela' '125'  
1x5mp21
```

Zadato je da se svi karakteri "e" promene u "1", svi karakteri "l" u 2, i svi karakteri "a" u 5

- Ako je **charset1** duži od **charset2**, poslednji karakter iz **charset2** se (nevidljivo) ponavlja sve dok **charset1** i **charset2** ne postanu iste dužine; primer:

```
→ echo 'example' | tr 'ela' '12'  
1x2mp21
```

Poslednji karakter (2) je ponovljen jedanput, i dobijen je rezultat kao da je **charset2** glasio '122'

- Ako je **charset1** kraći od **charset2**, svi "višak" karakteri iz **charset2** se ignorišu:

```
→ echo 'example' | tr 'el' '125'  
1xamp21
```

Dobijen je rezultat kao da je **charset2** glasio '12'

- Komandi **tr** se mogu zadati i opsezi karaktera pomoću crtica; tako na primer se mogu sva slova iz nekog teksta pretvoriti u velika na sledeći način:

```
→ echo 'example' | tr 'a-z' 'A-Z'  
EXAMPLE
```

- Pomoću opcije **-d** navodi se samo **charset1**, i postiže se to da svi karakteri iz tog seta budu obrisani; na primer, svi samoglasnici iz teksta se brišu na sledeći način:

```
→ echo 'example' | tr -d 'aeiouAEIOU'  
xmpl
```

- U narednom primeru, iz fajla file1.txt se brišu svi Newline karakteri (oznaka \n, ASCII 10), i novi tekst upisuje u fajl file2.txt:

```
→ tr -d '\n' < file1.txt > file2.txt
```

U drugom fajlu naći će se ceo sadržaj prvog fajla, ispisan u jednoj liniji

- Skupovi karaktera mogu biti zadati na više načina:

Form	Meaning
ABDG	The sequence of characters A, B, D, G.
A-Z	The range of characters from A to Z.
[x*y]	y repetitions of the character x.
[:class :]	The same character classes ([:alnum:], [:digit:], etc.) accepted by grep.

- Mogu se zadavati i eskejp karakteri: Bell (oznaka \a, zvonice), Backspace (oznaka \b, razmak), Form Feed (FF, oznaka \f), Line Feed (LF, oznaka \n, drugi naziv Newline), Carriage Return (CR, oznaka \r), Horizontal Tab (oznaka \t) i Vertical Tab (oznaka \v)

- Pregled važnih opcija:

- d Delete the characters in *charset1* from the input.
- s Eliminate adjacent duplicates (found in *charset1*) from the input. For example, `tr -s aeiouAEIOU` would squeeze adjacent, duplicate vowels to be single vowels (reeeeeeally would become really).
- c Complement: operate on all characters *not* found in *charset1*.
- t If *charset1* is longer than *charset2*, make them the same length by truncating *charset1*. If -t is not present, the last character of *charset2* is (invisibly) repeated until *charset2* is the same length as *charset1*.

sort [options] [files]

- Komanda **sort** sortira linije teksta u zadatom fajlu; preciznije: ispisuje linije teksta zadatog fajla sortirane po abecedi (ili po nekom drugom zadatom kriterijumu); originalni fajl se ne menja; ako se zada više fajlova, svi se nadovežu i onda obavi sortiranje

- Primer:

```
→ cat fruits
peach
melon
mango
→ sort fruits
mango
melon
peach
```

- Pregled važnih opcija:

- f Case-insensitive sorting.
- n Sort numerically (i.e., 9 comes before 10) instead of alphabetically (10 comes before 9 because it begins with a "1").
- g Another numerical sorting method with a different algorithm that, among other things, recognizes scientific notation (7.4e3 means "7.4 times ten to the third power," or 7400). Run `info sort` for full technical details.
- u Unique sort: ignore duplicate lines. (If used with -c for checking sorted files, fail if any consecutive lines are identical.)
- c Don't sort, just check if the input is already sorted. If it is, print nothing; otherwise, print an error message.
- b Ignore leading whitespace in lines.
- r Reverse the output: sort from greatest to least.

-t X Use X as the field delimiter for the -k option.

-k key Choose sorting keys. (Combine with -t to choose a separator character between keys.)

- Kriterijum ("ključ" = key) sortiranja se zadaje pomoću opcije -k; sintaksa je -kF[.C]; značenje oznaka je:
>> F: redni broj polja (kolone)
>> C: redni broj karaktera u polju (1 po difoltu)

- Na primer, -k1.5 znači da sortiranje treba obaviti po petom karakteru prvog polja (u slučaju fajla fruits, sortirana lista bi bila jednaka polaznoj); "partnerska" opcija -t definiše delimiter/separator između polja, na primer zarez (umesto razmaka koji je difolt)

- Primer:

→ **cat people**

George Washington,123 Main Street,New York
Abraham Lincoln,54 First Avenue,San Francisco
John Adams,39 Tremont Street,Boston

Difolt sortiranje (bez opcija) bi naravno bilo po imenima, pa bi se linija Abraham našla na prvom mestu, George na drugom i John na trećem

- Međutim, ako korisnik hoće da sortira linije po gradu, treba da uradi sledeće: prvo, da pomoću opcije -t zada da je separator između polja zarez; drugo, da pomoću opcije -k zada da se sortiranje obavi po prvom karakteru trećeg polja (grad):

→ **sort -k3 -t, people**

John Adams,39 Tremont Street,Boston
George Washington,123 Main Street,New York
Abraham Lincoln,54 First Avenue,San Francisco

uniq [options] [files]

- Komanda **uniq** prepoznaje i radi šta joj se zada sa identičnim uzastopnim linijama datog tekstualnog fajla

- Primer: dat je fajl u kome su dve uzastopne linije, druga i treća, identične:

```
→ cat letters2
```

```
a  
b  
b  
c  
b
```

- Po difoltu (bez parametara), ova komanda će izbrisati sve uzastopne "duplikate", to jest od svih uzastopnih identičnih linija ostaviće samo jednu:

```
→ uniq letters2
```

```
a  
b  
c  
b
```

- Ova komanda se obično koristi posle sortiranja:

```
→ sort letters2 | uniq
```

```
a  
b  
c
```

Ovde je očigledno ostalo samo jedno b, jer su se posle sortiranja našla tri slova b jedno za drugim

- Takođe, umesto da se "duplirane" linije brišu, ova komanda uz opciju **-c** (count) će samo izbrojati broj uzastopnog ponavljanja linija u fajlu:

```
→ sort letters2 | uniq -c
    1 a
    3 b
    1 c
```

- Pregled važnih opcija:

- c Count adjacent duplicate lines.
- i Case-insensitive operation.
- u Print unique lines only.
- d Print duplicate lines only.
- s *N* Skip the first *N* characters on each line when detecting duplicates.
- f *N* Ignore the first *N* whitespace-separated fields on each line when detecting duplicates.
- w *N* Consider only the first *N* characters on each line when detecting duplicates. If used with -s or -f, sort will ignore the specified number of characters or fields first, then consider the next *N* characters.

tee [options] files

- Slično komandi **cat**, i komanda **tee** jednostavno ispisuje, bez promene, na standardnom izlazu podatke koje preuzme na standardnom ulazu; međutim, ona uz to kopira podatke iz standardnog ulaza u fajl
- Ova komanda se često koristi u sredini lanca: u toj situaciji ona upisuje podatke koje je dobila u fajl, kako bi se sačuvali kao međurezultat u toku izvršenja lanca, i istovremeno predaje iste te podatke dalje, narednoj komandi u lancu

- Primer:

```
→ who | tee original_who | sort
barrett pts/1 Sep 22 21:15
byrnes pts/0 Sep 15 13:51
silver :0 Sep 23 20:44
silver pts/2 Sep 22 21:18
```

- U primeru se odvija sledeće: komanda **who** formira (nesortiranu) listu korisnika; tu listu preuzima (kao druga u lancu) komanda **tee**, koja radi dve stvari:

1) beleži tu (originalnu, to jest nesortiranu) listu u fajl `original_who`

2) predaje tu listu komandi **sort** (trećoj u lancu) na sortiranje, čiji rezultat se vidi na ekranu

- Kreirani fajl je znači originalna/nesortirana lista, koja predstavlja međurezultat:

```
→ cat original_who
silver :0 Sep 23 20:44
byrnes pts/0 Sep 15 13:51
barrett pts/1 Sep 22 21:15
silver pts/2 Sep 22 21:18
```

- Pregled važnih opcija:

-a Append instead of overwriting files.

- Postoje još moćnije komande za obradu teksta, koje će biti samo pomenute: **awk**, **sed**, **m4** -- one pružaju i mogućnost jednostavnog programiranja (!)

Tema 13: Komprimovanje fajlova

- Glavne komande vezane za komprimovanje (eng. compression; obrnuti proces je dekomprimovanje, eng. uncompression) fajlova su:

tar	Package multiple files into a single file.
gzip	Compress files with GNU Zip.
gunzip	Uncompress GNU Zip files.
bzip2	Compress files in BZip format.
bunzip2	Uncompress BZip files.
bzcat	Compress/uncompress BZip files via standard input/output.
compress	Compress files with traditional Unix compression.
uncompress	Uncompress files with traditional Unix compression.
zcat	Compress/uncompress file via standard input/output (gzip or compress).
zip	Compress files in Windows Zip format.
unzip	Uncompress Windows Zip files.
munpack	Extract MIME data to files.
mpack	Convert a file into MIME format.

- Linux može da komprimuje (i naravno dekomprimuje) fajlove u veliki broj formata, a najpopularnija su dva: GNU Zip -- komprimovani fajlovi imaju ekstenziju gz, i BZip -- komprimovani fajlovi imaju ekstenziju bz2

- Ostali korišćeni formati su: Zip fajlovi iz Windows sistema (ekstenzija zip), xz fajlovi (ekstenzije xz i lzma), i standardni Unix komprimovani format (ekstenzija Z); pregled formata dat je na adresi:

http://en.wikipedia.org/wiki/List_of_archive_formats

- Tehnika komprimovanja podrazumeva smanjenje veličine fajla primenom raznih algoritama; komprimovani fajl naziva se i zastarelim terminom "arhiva"; koriste se i

kolokvijalni termini pakovati/raspakovati i arhivirati/dearhivirati; koliko će se smanjiti veličina fajla, zavisi od raznih faktora od kojih je najbitniji format originalnog fajla

- Prilikom slanja i-mejla, zahvaljujući standardu MIME (Multipurpose Internet Mail Extensions) koji omogućava slanje binarnih fajlova (na primer, fotografija) kao atačmenta u mejlu, komprimovanje atačmenta se obavlja automatski na mestu slanja (kao i dekomprimovanje na mestu prijema); postoje komande **munpack** i **mpack** kojima se takvo komprimovanje može raditi sa komandne linije (neće biti obrađivane)

tar [options] [files]

- Komanda **tar** "pakuje" to jest komprimuje zadate fajlove i direktorijume u jedan (komprimovani) fajl; ime joj dolazi od "tape archive" jer se nekada davno bekapovanje podataka vršilo na magnetnim trakama; ovo je najkorišćenija Linux komanda za komprimovanje, sa mnogo opcija
- Preciznije rečeno: ova komanda pakuje fajlove u jedan fajl, koji ne mora nužno biti i komprimovan (već ide samo "sve na gomilu"); ali korisnik se, kad već pakuje, obično odluči i da komprimuje
- Primer komprimovanja:

```
→ tar -czf myarchive.tar.gz mydir Create
```

Od direktorijuma mydir kreira se arhiva myarchive.tar.gz (postoje dve ekstenzije jedna za drugom, tar i gz – piše ih korisnik zbog sebe, Linux ih naravno ignoriše); opcije **-c**, **-z** i **-f** su objašnjene u tabeli opcija

- Pregled vlasništva nad arhivom se obavlja kao za običan fajl:

```
→ ls -lG myarchive.tar.gz  
-rw-r--r-- 1 smith 350 Nov  7 14:09 myarchive.tar.gz
```

- Pregled sadržaja arhive:

→ **tar -tf myarchive.tar.gz**

List contents

```
mydir/  
mydir/dir/  
mydir/dir/file10  
mydir/file1  
mydir/file2
```

- Dekomprimovanje se vrši pomoću iste komande:

→ **tar -xf myarchive.tar.gz**

Extract

Opcija -x služi za dekomprimovanje

- Ako se zadaju pojedinačni fajlovi, samo oni će biti dekomprimovani:

→ **tar -xvf myarchive.tar myfile myfile2 myfile3**

dok će u suprotnom biti dekomprimovana cela arhiva

- Pregled važnih opcija:

-c	Create an archive. You'll have to list the input files and directories on the command line.
-r	Append files to an existing archive.
-u	Append new/changed files to an existing archive.
-A	Append one archive to the end of another: for example, <code>tar -A -f first.tar second.tar</code> appends the contents of <code>second.tar</code> to <code>first.tar</code> . Does not work for compressed archives.
-t	List (test) the archive.
-x	Extract files from the archive.

- f *file* Read the archive from, or write the archive to, the given file. This is usually a tar file on disk (such as *myarchive.tar*) but can also be a tape drive (such as */dev/tape*).
- d Diff (compare) the archive against the filesystem.
- z Use gzip compression.
- j Use bzip2 compression.
- Z Use Unix compression.
- v Verbose mode: print extra information.
- h Follow symbolic links rather than merely copying them.
- p When extracting files, restore their original permissions and ownership.

gzip [options] [files] gunzip [options] [files]

- Komanda **gzip** komprimuje, a komanda **gunzip** dekomprimuje fajlove u GNU Zip format; komprimovani fajlovi (arhive) imaju ekstenziju gz; po difoltu, originalni fajl se briše

- Primeri upotrebe:

gzip file	Compress <i>file</i> to create <i>file.gz</i> . Original <i>file</i> is deleted.
gzip -c file	Produce compressed data on standard output.
cat file gzip	Produce compressed data from a pipeline.
gunzip file.gz	Uncompress <i>file.gz</i> to create <i>file</i> . Original <i>file.gz</i> is deleted.
gunzip -c file.gz	Uncompress the data on standard output.
cat file.gz gunzip	Uncompress the data from a pipeline.

bzip2 [options] [files] bunzip2 [options] [files]

- Komanda **bzip2** komprimuje, a komanda **bunzip2** dekomprimuje fajlove u Burrows-Wheeler format; kod njega je fajlovi više komprimuju, ali procedura duže traje; komprimovani fajlovi imaju ekstenziju bz2

- Primeri upotrebe:

bzip2 file	Compress <i>file</i> to create <i>file.bz2</i> . Original <i>file</i> is deleted.
bzip2 -c file	Produce compressed data on standard output.
cat file bzip2	Produce compressed data on standard output.
bunzip2 file.bz2	Uncompress <i>file.bz2</i> to create <i>file</i> . Original <i>file.bz2</i> is deleted.
bunzip2 -c file.bz2	Uncompress the data on standard output.
cat file.bz2 bunzip2	Uncompress the data on standard output.

compress [options] [files] uncompress [options] [files]

- Komande **compress** i **uncompress** (de)komprimuju fajlove u standardni Unix komprimovani format (prema Lempel Ziv algoritmu, koji se koristi i kod grafičkog GIF formata); komprimovani fajlovi imaju ekstenziju Z

- Primeri upotrebe:

compress file	Compress <i>file</i> to create <i>file.Z</i> . Original <i>file</i> is deleted.
compress -c file	Produce compressed data on standard output.

<code>cat file compress</code>	Produce compressed data from a pipeline.
<code>uncompress file.Z</code>	Uncompress <i>file.Z</i> to create <i>file</i> . Original <i>file.Z</i> is deleted.
<code>uncompress -c file.Z</code>	Uncompress the data on standard output.
<code>cat file.Z uncompress</code>	Uncompress the data from a pipeline.

zip [options] [files]

- Komanda **zip** komprimuje fajlove u Windows Zip format, dok ih komanda **unzip** dekomprimuje; ekstenzija komprimovanih fajlova je zip; originalni fajl ovom komandom se ne briše

- Primeri upotrebe:

<code>zip myfile.zip file1 file2 file3...</code>	Pack.
<code>zip -r myfile.zip dirname</code>	Pack recursively.
<code>unzip -l myfile.zip</code>	List contents.
<code>unzip myfile.zip</code>	Unpack.

Tema 14: Poređenje fajlova

- Glavne komande vezane za poređenje/komparaciju (eng. comparison) fajlova su:

<code>diff</code>	Line-by-line comparison of two files or directories.
<code>comm</code>	Line-by-line comparison of two sorted files.
<code>cmp</code>	Byte-by-byte comparison of two files.
<code>md5sum</code>	Compute a checksum of the given files (MD5).

- U Linuxu postoji tri načina da se porede fajlovi:

1) liniju po liniju, za tekstualne fajlove (komande **diff**, **comm**)

2) bajt po bajt, za tekstualne i binarne fajlove (komanda **cmp**)

3) poređenjem parametra checksum (komanda **md5sum**)

diff [options] file1 file2

- Komanda **diff** se prvenstveno koristi za poređenje dva tekst fajla, liniju po liniju: komanda generiše detaljan izveštaj o svim uočenim razlikama; ako su fajlovi isti, ona ne daje nikakav izlaz; ako se upotrebi za poređenje binarnih fajlova, komanda samo izvesti da li su isti ili nisu

- Difolt izlaz ove komande, pri poređenju dva tekstualna fajla (A i B), izgleda ovako:

```
Broj linije u fajlu i vrsta razlike  
< Sekcija fajla A, ako postoji  
---  
> Sekcija fajla B, ako postoji
```

Ovakav tekst se ispisuje za svaku uočenu razliku; znak < označava odgovarajuću sekciju (jednu ili više uzastopnih linija teksta) u fajlu A, a znak > označava odgovarajuću sekciju u fajlu B; separator između dve sekcije je povorka od tri crte ---

- Pri uočavanju razlika, fajl A služi kao referenca; ova komanda zapravo uočava **promene** (modifikacije) koje bi trebalo napraviti u fajlu A kako bi se dobio fajl B; da bi komanda bila od koristi, fajlovi moraju biti slični

- Primer: fileA izgleda ovako:

```
Hello, this is a wonderful file.  
The quick brown fox jumped over  
the lazy dogs.  
Goodbye for now.
```

a fileB izgleda ovako:

```
The quick blue fox jumped over
the lazy dogs.
Goodbye for now.
Linux r00lz!
```

- U tom slučaju, komanda `diff` će dati sledeći izlaz:

```
→ diff fileA fileB
1,2c1                                fileA lines 1-2 became fileB line 1
< Hello, this is a wonderful file.    Lines 1-2 of fileA
< The quick brown fox jumped over
---                                diff separator
> The quick blue fox jumped over      Line 1 of fileB
4a4                                Line 4 was added in fileB
> Linux r00lz!                       The added line
```

- Objašnjenja:

1) oznaka `1,2c1` (c=change) znači da su linije 1 i 2 fajla A "postale" linija 1 fajla B; zatim ispisuje te linije: 1 i 2 iz fajla A, pa separator, pa 1 iz fajla B

2) oznaka `4a4` (a=add) znači da je linija 4 "dodata" u fajl B, i prikazuje tu liniju

- Sâm zadatak poređenja dva tekstualna fajla je veoma komplikovan (a što su razlike veće sve je komplikovaniji); stoga je veoma teško napraviti komandu koja to uspešno radi, tako da se i za `diff` mogu uočiti mnoge mane i nelogičnosti; tako na primer, iz nepoznatih razloga, razlika brown-blue nije uočena

- Pregled važnih opcija za kontrolu izlaznog formata:

```
-y      Side-by-side format; use -w to adjust the width of the output.
-e      Create an ed script that would change fileA into fileB if run.
-q      Don't report changes, just say whether the files differ.
```

- Pomoću komande **diff** mogu se porediti i direktorijumi:

→ **diff dir1 dir2**

Komanda poredi (na objašnjeni način) fajlove sa istim nazivima koji postoje u oba direktorijuma; uz to, daje listu svih fajlova koji postoje u jednom direktorijumu ali ne i u drugom; da bi se poredila dva direktorijuma rekurzivno (svi njihovi poddirektorijumi i tako dalje), koristi se opcija **-r**

- Pregled važnih opcija:

-b Don't consider whitespace.

-B Don't consider blank lines.

-i Ignore case.

-r When comparing directories, recurse into subdirectories.

- Postoji i srodna komanda **sdiff**, koja sve uočene razlike između dva fajla ispisuje u treći fajl, a na osnovu instrukcija korisnika

comm [options] file1 file2

- Komanda **comm** poredi dva **sortirana** tekstualna fajla (ona kod kojih su linije sortirane po abecedi), i generiše izlaz u tri kolone, odvojene tabovima:

>> 1. kolonu čine sve linije koje postoje u fajlu 1 ali ne u fajlu 2

>> 2. kolonu čine sve linije koje postoje u fajlu 2 ali ne u fajlu 1

>> 3. kolonu čine sve linije koje postoje u oba fajla (zajedničke)

- Primer: dati su sortirani fajlovi commfile1 i commfile2 koji sadrže sledeće linije:

<i>commfile1:</i>	<i>commfile2:</i>
apple	baker
baker	charlie
charlie	dark

- Rezultat izvršenja će biti:

```
→ comm commfile1 commfile2
apple
        baker
        charlie
    dark
```

- Pregled važnih opcija:

- 1 Suppress column 1.
- 2 Suppress column 2.
- 3 Suppress column 3.

cmp [options] file1 file2 [offset1 [offset2]]

- Komanda **cmp** poredi dva fajla (tekstualna ili binarna), bajt po bajt: ako su fajlovi isti, ne vraća nikakav rezultat; ako nisu, komanda po defaultu daje obaveštenje o poziciji na kojoj se nalazi **prva** razlika; preciznije, daje redni broj prvog bajta koji se u dva fajla razlikuje
- Pri tome, ako se porede dva tekst fajla, obaveštenje koje daje ova komanda može biti lako interpretirano s obzirom na način zapisivanja podataka u tekst fajlovima pomoću ASCII kodova, koji je ranije objašnjen
- Primer: fajlA.txt sadrži tekst *hello ab* a fajlB.txt sadrži tekst *hello cd*; rezultat njihovog poređenja biće:

```
→ cmp fajlA.txt fajlB.txt  
fajlA.txt fajlB.txt differ: byte 7, line 1
```

Znači, uočena je prva razlika, na bajtu broj 7 (to jest na sedmom karakteru) koji se nalazi u liniji 1: zaista, u prvom fajlu tu je slovo a, a u drugom slovo c

- Da se prikažu **sve** razlike: kako njihove pozicije, kao i to u čemu se razlike sastoje, koristi se opcija **-l** (long -- videti listu opcija ispod)

- Ukoliko se upotrebi pomenuta opcija na istim fajlovima, izlaz će biti:

```
→ cmp -l fajlA.txt fajlB.txt  
7 141 143  
8 142 144
```

Pronađene su znači obe razlike: na bajtovima (karakterima) broj 7 i 8; na bajtu 7, brojna vrednost 141, u oktalnom sistemu, odgovara ASCII vrednosti slova a; brojna vrednost 143 odgovara vrednosti slova c (slično i za bajt 8, gde su slova b odnosno d)

- Ako se porede dva binarna fajla, jedina vredna informacija koju obični korisnik dobija je da li su fajlovi isti ili nisu; da bi znao u čemu se tačno razlika (onda kad postoji) sastoji, korisnik mora, kao što je objašnjeno ranije, da bude specijalista za konkretni format fajla

- Primer: docA.docx sadrži tekst *hello ab* a docB.docx sadrži tekst *hello cd*; rezultat njihovog poređenja biće:

```
→ cmp docA.docx docB.docx  
docA.docx docB.docx differ: byte 2416, line 5
```

Osim informacije da se fajlovi razlikuju, običnom korisniku je nemoguće da zaključi u čemu je razlika; uz opciju **-l** dobije se ogromna lista, iz koje takođe ne može ništa da zaključi

- Pregled važnih opcija:

-l Long output: print all differences, byte by byte:

```
→ cmp -l myfile yourfile  
225 167 127
```

This means at offset 225 (in decimal), *myfile* has a small "w" (octal 167) but *yourfile* has a capital "W" (octal 127).

-s Silent output: don't print anything, just exit with an appropriate return code; 0 if the files match, 1 if they don't. (Or other codes if the comparison fails.)

md5sum files | --check file

- Komanda **md5sum** izračunava parametar checksum za dati fajl, upotrebom algoritma koji se naziva MD5; checksum je 32-cifreni heksadecimalni broj; primer:

```
→ md5sum myfile  
48760f921ec6111e3979efa14e22535d  myfile
```

- Checksum je parametar jedinstven za dati fajl; drugim rečima, gotovo je nemoguće događaj da dva različita fajla imaju isti checksum; uz to, čak i kada su fajlovi veoma slični (na primer, kada je jedan nastao sitnom modifikacijom drugog), njihovi checksum su jasno različiti
- Stoga je, ako korisnik hoće da bude siguran da su dva fajla potpuno isti (što se naziva provera autentičnosti -- hoće na primer da bude siguran da neki PDF dokument nije falsifikovan), skoro 100% pouzdan način da to proveriti je da uporedi checksum ta dva fajla
- Dva checksum mogu da se uporede vizuelno, a mogu i pomoću komande **diff**::

```
→ md5sum myfile | cut -c1-32 > sum1  
→ md5sum myfile2 | cut -c1-32 > sum2  
→ diff -q sum1 sum2  
Files sum1 and sum2 differ
```

Iz izlaza komande **md5sum** uzima se prva 32 karaktera, to jest sâm checksum, i stavlja u fajl; dva tako nastala tekstualna fajla se porede pomoću komande **diff**

- Još pouzdanija komanda koja služi za sličnu namenu je **shasum**, zasnovana na algoritmima SHA-1 i SHA-256; stare komande **sum** i **cksum** treba izbegavati kao nepouzidane

Tema 15: Štampanje

- Glavne komande vezane za štampanje su:

lpr **Print a file.**

lpq **View the print queue.**

lprm **Remove a print job from the queue.**

- Linux ima dva popularna sistema za štampanje (eng. printing system) -- CUPS i LPRng; **sistem za štampanje** je sistemski softver koji služi za komunikaciju sa štampačem

- **CUPS** (od Common UNIX Printing System) je danas standardni sistem za Linux računare; on omogućava da računar (na kome je instaliran) služi kao server za štampanje (eng. print server): da prihvata zahteve za štampanjem od drugih računara-klijenata, procesira ih i šalje na štampanje po prioritetima; CUPS koristi Internet Printing Protocol (IPP)

- **LPRng** obavlja sličnu funkciju, samo se zasniva na drugom protokolu (Line Printer Daemon protocol); zastareo je i retko se koristi

- Oba sistema koriste iste komande (**lpr**, **lpq** i **lprm** -- nazivi očigledno zadržani iz zastarelog LPRng), ali se opcije razlikuju zavisno od sistema; ovde će biti predstavljene samo opcije koje su iste i za CUPS i za LPRng

- Da bi se instalirao štampač, potrebno je editovati kriptovani konfiguracioni fajl, tipično /etc/cups/printers.conf ili /etc/printcap; lakši način je u GNOME ili KDE okruženjima, koja u svojim sistemskim podešavanjima imaju gotove alate za konfiguraciju štampača; ti alati automatski generišu konfiguracioni fajl

- Pristup CUPS menadžment sistemu za lokalni štampač obavlja se u brauzeru, na adresi:
http://localhost:631

lpr [options] [files]

- Komanda **lpr** ("line printer") šalje fajl štampaču na štampanje:

→ **lpr -P myprinter myfile**

- Pregled važnih opcija:

-P *printername* Send the file to printer *printername*, which you have previously set up.

-# *N* Print *N* copies of the file.

lpq [options]

- Komanda **lpq** ("line printer queue"; queue = red za čekanje) ispisuje listu svih poslova štampanja (eng. print job) koji čekaju na izvršenje

- Primer:

→ **lpq -P *printername***

Rank	Owner	Job	Files	Total Size
active	root	133	pr1	467 bytes
1st	chavez	61	11726.f	74578 bytes
2nd	harvey	78	fpppp.F	12394 bytes

- Pregled važnih opcija:

-P *printername* List the queue for printer *printername*.
-a List the queue for all printers.
-l Be verbose: display information in a longer format.

lprm [options] [job_IDs]

- Komanda **lprm** ("line printer remove") zaustavlja/poništava jedan ili više poslova štampanja: da bi poništio neki posao, korisnik prvo pomoću komande **lpq** treba da sazna ID tog posla, i da ga zatim zada komandi **lprm**:

→ lprm -P *printername* 61 78

Ovim se zaustavljaju poslovi čiji su ID 61 i 78

- Ako se ne zada ID posla, onda se poništava aktivni posao (onaj koji se upravo izvršava); samo superkorisnik može da poništava poslove običnih korisnika

Tema 16: Diskovi i fajlsistemi

- Glavne komande vezane za rad sa diskovima i fajlsistemima su:

df Display available space on mounted filesystems.
mount Make a disk partition accessible.
umount Unmount a disk partition (make it inaccessible).
fsck Check a disk partition for errors.
eject Eject a CD, DVD, or other removable disk.

Fajlsistem standard

- Pojam **fajlsistem**, kao što je ranije pomenuto, u najprostijem smislu označava celu hijerarhiju fajlova i direktorijuma koji se nalaze na jednom disku; u Linuxu postoji jedan složeniji koncept koji se naziva **fajlsistem standard**; problem je što se u literaturi reč "standard" po pravilu izostavlja, što stvara konfuziju
- Skup pravila koja propisuju kako se podaci organizuju u fajlove i direktorijume, kako se fajlovi fizički čuvaju na disku, kako se preuzimaju, kako se čuvaju podaci o fajlovima (to jest podaci o podacima = metapodaci), i slično, se naziva fajlsistem standard
- Najkorišćeniji Linux fajlsistem standard se zove ext4 (naslednik je starijeg ext3); on koristi tehniku koja se naziva **journaling** koja povećava otpornost sistema na kvarove; postoji još nekoliko Linux fajlsistem standarda, na primer ZFS i XFS
- U Windowsu se fajlsistem standard zove NTFS; fajlsistem standard ISO 9660 definiše kako se podaci čuvaju na optičkim diskovima; postoje i distribuirani fajlsistem standardi (kao na primer NFS = Network File System) -- oni propisuju kako se fajlovima pristupa preko mreže

Diskovi i particije u Linuxu

- Generalno, jedan Linux sistem može sadržati više diskova, a svaki disk više particija; **disk** je hardverski uređaj za čuvanje podataka, koji može biti podeljen na **particije** tokom procedure koja se naziva **particioniranje**; svaka particija može biti posmatrana kao nezavistan uređaj
- Pre nego što se na particiju (isto važi i za disk) mogu prebaciti fajlovi, ona mora biti formatirana; **formatiranje** vrši poseban program, koji za particiju određuje koji fajlsistem standard će biti korišćen; formatiranje particija (kao i particioniranje) se obavlja tokom instaliranja Linuxa

- Usputna napomena: procedure formatiranja i particioniranja mogu na Linux sistemima biti prilično složene, ali istovremeno pružaju veliki broj mogućnosti; komande koje se za to koriste su: **mkfs** za formatiranje, i **gparted**, **parted**, **fdisk**, **sfdisk** za particioniranje

Virtualni fajlsistem

- Pored pomenutog fajlsistema u Linuxu postoji jedan, takođe složeniji, koncept koji se naziva **virtualni fajlsistem**; problem je što se u literaturi ovaj pridev "virtualni" po pravilu izostavlja, što stvara konfuziju
- Kao posledica dva novouvedena pojma čiji se nazivi u praksi skraćuju, kada se u literaturi sretne sa pojmom "fajlsistem", korisnik ne zna na šta se tačno misli – da li na fajlsistem u najprostijem smislu reči, ili na virtualni fajlsistem, ili na fajlsistem standard
- Svi fajlovi, na svim uređajima (eng. device) koji postoje u jednom Linux sistemu (na primer: disk/particija, USB memorija, eksterni hard disk, CD-ROM disk, ali i udaljeni diskovi, i još ponešto) "virtualno" se pojavljuju u jednoj **jedinstvenoj** hijerarhiji koja se stoga naziva virtualni fajlsistem; prednosti ovog koncepta neće biti objašnjavane
- Ovakav pristup je veoma različit od onoga u Windows OS, u kome se svaki uređaj vidi kao poseban/nezavistan (npr. C: je oznaka za hard disk, D: za DVD disk), a fajlovima koji se na njemu nalaze se pristupa tako što se prvo odabere taj uređaj, i zatim interna putanja na njemu
- Pošto je znači virtualni fajlsistem jedna jedinstvena hijerarhija (stablo), to znači da u njemu postoji i jedan jedinstveni root direktorijum: svaki fajl u sistemu (na kom god uređaju se fizički nalazio) se nalazi hijerarhijski "ispod" tog roota
- Linux, doduše, svakom uređaju dodeljuje posebnu oznaku (radi prepoznavanja), ali se fajlovima na tom uređaju ne pristupa uz pomoć te oznake; umesto toga, da se pristupilo

nekom fajlu na uređaju X, mora biti definisano gde će se u virtualnom fajlsistemu (preciznije, u kom direktorijumu) svi fajlovi sa uređaja X nalaziti/videti

- Preciznije, svaki uređaj ima nešto što se zove njegov device file (ili special file); na srpski bismo ovaj pojam mogli, jasnoće radi, da slobodnije prevedemo kao **matični fajl**; to je jedan sistemski fajl koji služi kao interfejs ka drajveru, softveru koji služi za pristup samom uređaju; matični fajl možemo tretirati prosto kao "oznaku" uređaja

- Matični fajlovi svih uređaja se nalaze u direktorijumu /dev; za diskove/particije, ti fajlovi nose nazive oblika sdSC gde je S slovo a C cifra; slovo a znači prvi disk, b drugi disk, i tako redom; cifra 1 znači prvu particiju, 2 drugu, i tako redom; tako je na primer /dev/sda2 matični fajl za drugu particiju na prvom disku; za CD je matični fajl na primer /dev/cdrom

Montiranje

- Procedura u okviru koje se definiše u kom će se direktorijumu, u hijerarhiji "ispod" jedinstvenog root direktorijuma, "videti" svi fajlovi sa uređaja X naziva se **montiranje** (eng. mounting) uređaja X; ono je obavezan korak u kreiranju virtualnog fajlsistema; kaže se da se "uređaj X montira na direktorijum"

- Na primer, da bi se pristupilo fajlovima na CD-ROMu, Linux prvo mora biti obavešten sledeće: "neka se fajlsistem (ovo u smislu: svi fajlovi i direktorijumi) sa ovog CD-ROMa, pojavljuje u tom-i-tom direktorijumu virtualnog fajlsistema"; taj direktorijum se naziva **tačka montiranja** (eng. mount point); tipično, to je direktorijum /media

- Preciznije, montiranje znači jednostavno logičko povezivanje: oznake jednog uređaja (to jest naziva njegovog matičnog fajla) i jednog direktorijuma, koji će postati njegova tačka montiranja, i u kome će se videti sadržaj uređaja

- Direktorijum `/media` obično već postoji u Linux sistemu, i kreiran je namenski da bude tačka montiranja za sve prenosive uređaje za čuvanje podataka, kao što su CD/DVD, USB memorije, i slično; on može da sadrži poddirektorijume od kojih na primer jedan služi kao tačka montiranja za CD/DVD, drugi za USB memorije, i slično
- Drugi direktorijum koji može biti namenjen da bude tačka montiranja je `/mnt`, ali generalno to može biti i bilo koji direktorijum u hijerarhiji; iz praktičnih razloga, dobro je da taj direktorijum (tačka montiranja) bude prazan (ako nije prazan, podaci se neće obrisati, ali će sadržaj koji se nalazio biti dostupan tek posle odmontiranja)
- Kao rezultat montiranja, na primer, ako je za neki uređaj X tačka montiranja `/mnt/new`, onda korisnik može da kreira i edituje fajl koji se na primer zove `/mnt/new/myfile`; taj fajl će se znači fizički nalaziti na uređaju X
- Samo admin (to jest superkorisnik) ima privilegiju da dozvoli montiranje fajlsistema; on je taj koji određuje tačku montiranja; kada se završi rad sa uređajem, fajlsistem se može "odmontirati" (eng. `umount`); to se na primer radi pre nego što se CD disk izbacila iz drajva
- Montiranje svih uređaja se generalno obavlja automatski, tokom butovanja sistema; ako neki uređaj nije uspešno montiran, admin to mora odraditi ručno; admin može da obavi i odmontiranje, na primer radi održavanja

`df [options] [disk devices | files | directories]`

- Komanda `df` ("disk free") prikazuje za dati uređaj: ukupan memorijski prostor, iskorišćeni prostor i slobodan prostor; bez argumenata, komanda daje informaciju o svim montiranim uređajima; ako joj se zada direktorijum, ova komanda daje informaciju o uređaju koji je montiran na taj direktorijum

- Primer:

→ **df**

Filesystem	1k-blocks	Used	Avail	Use%	Mounted on
/dev/sda	1011928	225464	735060	24%	/
/dev/sda9	521748	249148	246096	51%	/var
/dev/sda8	8064272	4088636	3565984	54%	/usr
/dev/sda10	8064272	4586576	3068044	60%	/home

Ovde se vide 4 particije, memorijski prostor i tačke montiranja (ovde su sve različite)

- Pregled važnih opcija:

- k List sizes in kilobytes (the default).
- m List sizes in megabytes.
- B *N* Display sizes in blocks of *N* bytes. (Default = 1024)
- h Print human-readable output, and choose the most appropriate unit for each size. For example, if your two disks have 1 gigabyte and 25 kilobytes free, respectively, **df -h** prints 1G and 25K. The -h option uses powers of 1024, whereas -H uses powers of 1000.
- l Display only local filesystems, not networked filesystems.
- T Include the filesystem type (ext3, vfat, etc.) in the output.
- t *type* Display only filesystems of the given type.
- x *type* Don't display filesystems of the given type.

mount [*options*] *device* | *directory*

- Komanda **mount** montira uređaj *device* na direktorijum *directory*; tako na primer ona montira particiju /dev/sda1 (kao što je pomenuto, to je zapravo matični fajl particije) na direktorijum /mnt/mydir, čime fajlovi koji se nalaze na particiji postaju dostupni u datom direktorijumu

- Da bi se to uradilo, prvo se kreira direktorijum i proveriti da li je prazan:

```
→ sudo mkdir /mnt/mydir
```

```
→ ls /mnt/mydir
```

Notice it's empty

posle čega se obavi montiranje i ustanovi da se vide tri fajla sa particije:

```
→ sudo mount /dev/sda1 /mnt/mydir
```

```
→ ls /mnt/mydir
```

```
file1 file2 file3
```

Files on the mounted partition

i na kraju proveriti pomoću komande **df** da je i gde je particija montirana:

```
→ df /mnt/mydir
```

```
Filesystem 1K-blocks  Used  Avail Use% Mounted on
/dev/sda1    1011928 285744 674780  30% /mnt/mydir
```

- Podaci o svim uređajima u sistemu i njihovim tačkama montiranja nalaze se u sistemskom konfiguracionom fajlu koji se naziva **tabela montiranja** (eng. mount table, mada se koristi i manje prikladan naziv filesystem table = tabela fajlsistema): to je tipično fajl `/etc/fstab` ili `/etc/mnttab`

- Navedeni podaci su organizovani tako da svakom uređaju odgovara jedna linija (vrsta) u fajlu; korisnik ovaj tekst fajl može da pregleda pomoću komande **cat**, ali ne može da ga uređuje

- Tipična linija u tabeli montiranja izgleda ovako:

```
/dev/sda8    /usr    ext3    defaults    1    2
```

Prvo polje je oznaka uređaja, drugo je direktorijum -- tačka montiranja, treća je fajlsistem standard, ostala tri polja nisu od interesa

- Tabela montiranja pruža mogućnost da se, prilikom narednog montiranja istog uređaja, uz komandu **mount** navede samo ime uređaja: to je dovoljna informacija jer u tabeli montiranja piše tačka montiranja za taj uređaj, čime je ona "zapamćena"

- Ako se na primer otkuca (u superkorisničkom statusu):

→ **mount /dev/sda8**

komanda **mount** će pretražiti tabelu montiranja, i u prema njenoj gore navedenoj liniji za tačku montiranja ponovo uzeti direktorijum **/usr**

- Još treba napomenuti i da se pri svakom butovanju sistema automatski izvršava komanda **mount -a** (od "all"), čime će se obaviti montiranje **svih** uređaja prema tabeli montiranja; tako admin ne mora da montira ručno

- Praktično je da admin u startu odredi svoju "politiku" montiranja (gde šta montira), i da je se pridržava u budućem radu; konačno, obične prenosive uređaji kao što su USB memorija ili DVD može montirati i odmontirati i obični korisnik (onaj koji ih koristi)

- Pregled važnih opcija (od ogromnog broja koliko postoji):

-t type Specify the type of filesystem, such as **ext3** or **ntfs**.

-l List all mounted filesystems; works with **-t** too.

-a Mount all filesystems listed in **/etc/fstab**. Ignores entries that include the **noauto** option. Works well with **-t** too.

umount [options] [device | directory]

- Komanda **umount** (obratiti pažnju da se piše **umount** a ne **unmount**) radi obrnuti proces od **mount**: ona obavlja odmontiranje uređaja, tako da više ne bude vidljiv u okviru virtualnog fajlsistema

- Treba imati u vidu da se nijedan eksterni uređaj, kao što je DVD disk (i posebno svi uređaji na koje se podaci upisuju), ne sme fizički odvojiti od računara pre nego što se odmontira, jer postoji mogućnost oštećenja (i gubljenja podataka); komanda **umount** obezbeđuje da se uređaj postavi u stanje u kome se može bezbedno odvojiti

- Na primer, korisnik može da odmontira CD na sledeći način:

→ **umount /dev/cdrom /cd**

a da se odmontiraju svi uređaji kuca se (a = all):

→ **sudo umount -a**

I komanda **umount** očitava tabelu montiranja

fsck [options] [devices]

- Komanda **fsck** ("filesystem check") proverava zadati uređaj (to jest fajlsistem na njemu) i, ako joj se to zada, ispravlja greške koje detektuje; ova komanda se automatski startuje prilikom butovanja sistema, ali korisnik može da je ručno pokrene kad hoće, ako/kad mu je to potrebno

- Preporučljivo je pre pokretanja ove komande odmontirati uređaj, da drugi programi ne bi mogli da mu pristupe

- Primer:

→ **sudo umount /dev/sda10**

→ **sudo fsck -f /dev/sda10**

Pass 1: Checking inodes, blocks, and sizes

Pass 2: Checking directory structure

Pass 3: Checking directory connectivity

Pass 4: Checking reference counts

Pass 5: Checking group summary information

/home: 172/1281696 files (11.6% non-contiguous), ...

opcija **-f** znači force (bez daljih objašnjenja)

- Komanda **fsck** je osnovna iz skupa programa za proveru fajlsistema koje se nalaze u /sbin; nazivi ostalih počinju sa fsck; podržava samo nekoliko fajlsistem standarda

- Pregled važnih opcija:

- A Check all disks listed in */etc/fstab*, in order.
- N Print a description of the checking that would be done, but exit without performing any checking.
- r Fix errors interactively, prompting before each fix.
- a Fix errors automatically (use only if you *really* know what you're doing; if not, you can seriously mess up a filesystem).

eject [options] [device_name]

- Komanda **eject** (=izbaci) obavlja istu aktivnost kao kada korisnik pritisne open/close dugme na CD-DVD drajvu:

→ **eject**

- Pregled važnih opcija:

- n Don't eject anything, just say what would be done. Combine with -v for a detailed description.
- v Produce verbose output.
- d Print the name of the default device to be ejected, such as */dev/cdrom*, and exit.

Tema 17: Bekap podataka

- Glavne komande vezane za bekap podataka su:

rsync Efficiently copy a set of files, even across a network.

dd Low-level copying of data.

growisofs Burn a DVD or Blu-ray disc.

- Postoje tri načina da se odradi bekap (eng. backup) fajlova u Linux sistemu:
 >> da se kopiraju na neki medijum za bekap, tipično eksterni hard disk
 >> da se upišu ("narežu") na CD/DVD/Blu-ray disk
 >> da se miroruju (eng. mirror = ogledalo, bez srpskog prevoda), tipično na neki udaljeni (eng. remote, onaj koji nije lokalni) računar – neće se raditi
- **Mirorovanje** podataka (eng. data mirroring) je kopiranje podataka na neki medijum u realnom vremenu; na taj način, informacija-kopija je u svakom momentu identična sa informacijom-originalom (isto kao što je odraz objekta u ogledalu u svakom momentu isti kao originalni objekat); sa druge strane, bekap se radi periodično
- Zahvaljujući mirorovanju, ne može se izgubiti ni najmanja količina podataka u slučaju kvara u sistemu: ako se desi kvar, korisnik samo nastavlja da radi sa kopijom kao da se ništa nije desilo; postoji i postupak koji se naziva server mirroring, kod koga se sve što se odvija na nekom serveru kopira u realnom vremenu na bekap server
- Postoje i druge komande za bekap koje se neće predstavljati, kao što su **cpio** (veoma fleksibilna), **dump** i **restore**

rsync [options] source destination

- Komanda **rsync** ("remote sync", sync=synchronization, to jest sinhronizacija) kopira skup fajlova za bekap; prilikom pravljenja kopije, korisnik može da odabere da li će se istovremeno iskopirati i dozvole pristupa i ostali atributi (opcija **-a**, od archive); može se izvršavati na mreži ili na jednom računaru; ima više od 50 opcija (!)
- Da se bekapuje direktorijum mydir i ceo njegov sadržaj u drugi direktorijum mydir2 na istom računaru, kuca se:

→ **rsync -a mydir mydir2**

Kopija će biti direktorijum mydir2/mydir; ako korisnik hoće da kopija bude mydir2, treba da doda kosu crtu posle naziva originala:

→ **rsync -a mydir/ mydir2**

- Da se direktorijum mydir bekapuje preko mreže na neki udaljeni host računar, na primer server.example.com, gde korisnik ima nalog sa korisničkim imenom "smith", kuca se na primer:

→ **rsync -a mydir smith@server.example.com:D2**

D2 je direktorijum (koji se naziva i destinacija) na host računaru

- Važno je napomenuti da komanda **rsync** automatski obezbeđuje sigurnu konekciju pomoću SSH (Secure Shell) kriptovanog mrežnog protokola za prenos podataka na udaljeni računar

- Ako korisnik hoće da obavlja inkrementalni bekap, može da koristi program **rsnapshot** (<https://rsnapshot.org>) koji je zasnovan na **rsync**; on nudi mnogo opcija za podešavanje bekapa

- Inkrementalni bekap se obavlja na sledeći način: kada se on pokrene, na bekap medijum se samo upisuju oni fajlovi koji su promenjeni u odnosu na poslednji bekap; oni koji nisu promenjeni se ne diraju, radi uštede u vremenu

- Pregled važnih opcija:

- o **Copy the ownership of the files. (You might need superuser privileges on the remote host.)**
- g **Copy the group ownership of the files. (You might need superuser privileges on the remote host.)**

- p Copy the file permissions.
- t Copy the file timestamps.
- r Copy directories recursively (i.e., including their contents).
- L Permit symbolic links to be copied (not the files they point to).
- D Permit devices to be copied. (Superuser only.)
- a Mirroring: copy all attributes of the original files. This implies all of the options -ogptrLD.
- x When copying a tree of files, remain within the current filesystem; do not cross over into other mounted filesystems.
- n Dry-run mode: don't actually do any copying. Just display what *would* be done.
- v Verbose mode: print information about what's happening during the copy. Add --progress to display a numeric progress meter while files are copied.

dd [options]

- Komanda **dd** kopira podatke na niskom nivou (što se zove i raw copy = sirovo kopiranje), bajt po bajt, umesto preko fajlsistema; ovakav način kopiranja ima nekih svojih prednosti na primer kada je u pitanju spasavanje izgubljenih podataka (neće se ulaziti u detalje)

- Da se kopira sadržaj jednog fajla u drugi, kuca se na primer:

```
→ dd if=fileA of=fileC
7+1 records in
7+1 records out
3816 bytes (3.8 kB) copied, 0.000356028 s, 10.7 MB/s
```

Oznaka **if** znači ulazni (input) fajl ili uređaj, **of** znači izlazni (output) fajl ili uređaj

- Prilikom kopiranja mogu se obaviti i neke konverzije podataka; na primer, mogu se sva slova u nekom tekstualnom fajlu pretvoriti u velika pomoću opcije **conv=ucase**

- Komanda **dd** može i da klonira disk sa jednog uređaja na drugi; **pažnja**: tom prilikom će se izbrisati svi postojeći podaci na uređaju na koji se kopira; primer:

→ **dd if=/dev/device1 of=/dev/device2 bs=512 conv=noerror**

Pomoću **noerror** se nalaže da se kopiranje obavi čak i ako se detektuje neka greška pri očitavanju

- Pregled važnih opcija:

if=file	Specify an input file or device.
of=file	Specify an output file or device.
bs=N	Copy <i>N</i> bytes at a time, known as the “block size.” (To set the block size differently for the input and the output, use ibs and obs , respectively.)
skip=N	Skip past <i>N</i> blocks of input before starting the copy.
seek=N	Discard <i>N</i> blocks of output before starting the copy.
conv=spec	Convert the data being copied. <i>spec</i> can be ucase (convert to uppercase), lcase (convert to lowercase), ascii (convert to ASCII from EBCDIC), and many others listed on the manpage.

growisofs [options] tracks

- Komanda (čudnog naziva) **growisofs** bekapuje podatke na optičkom disku (CD/DVD ili Blu-ray); primer:

→ **growisofs -Z /dev/sr1 -R -J /home/smith**

Ovim se home direktorijum korisnika smith upisuje na optički disk 1 (označen njegovim matičnim fajlom)

- U primeru se pomoću opcije **-Z** nalaže da se podaci upisuju na disk direktno (bez upotrebe takozvanog ISO image fajla – neće se objašnjavati); pomoću opcija **-R** i **-J** se omogućava veća fleksibilnost pri upisivanju (neće se objašnjavati)
- Konačno, ako korisnik hoće da koristi isti CD/DVD i da na njemu nastavi upisivanje nekih novih podataka (što se naziva multisession upisivanje, to jest iz više puta), on treba da upotrebi istu komandu, samo što umesto **-Z** treba da napiše **-M**

Tema 18: Pregled procesa

- Glavne komande vezane za pregled (i praćenje) procesa od strane korisnika su:

ps	List process.
uptime	View the system load.
w	List active processes for all users.
top	Monitor resource-intensive processes interactively.
free	Display free memory.

- Najprostije rečeno, **proces** (ili task) je komanda u toku izvršenja ("u akciji"), na najnižem nivou; proces nastaje kada se komanda pokrene, i nestaje kada se izvrši; neke komande mogu pokrenuti i više od jednog procesa; svaki proces je izolovan od drugih, tako da na primer ako se jedan prekine zbog neke greške, to na druge neće uticati
- Svaki moderan OS, pa i Linux, nadgleda i upravlja/kontroliše sve procese (ovaj postupak se zove **multiprocesiranje** ili multitasking); svaki proces traži pristup resursima, od kojih je CPU najdragoceniji; Linux će dodeljivati resurse prvo jednom procesu (dok ostali čekaju), pa drugom, i tako redom, po prioritetima

- Konkretnije, dodelom resursa se bavi deo Linux kernela koji se zove **process scheduler** (eng. schedule = satnica, raspored); zahvaljujući njemu, CPU je optimalno iskorišćen u smislu da najbrže odradi postavljene zadatke
- Svakom procesu je dodeljen jedinstveni broj za identifikaciju/raspoznavanje, koji se naziva process ID -- skraćeno **PID**; kada se proces završi, njegov PID postaje slobodan, i može se dodeliti nekom drugom, novom procesu
- Kao što je ranije pomenuto, proces nije isto što i posao; proces je pojam nižeg nivoa, vezan za sâm operativni sistem; posao je pojam višeg nivoa, koji postoji samo za šel u kome se izvršava odgovarajuća komanda

ps [options]

- Komanda **ps** ("process status") ispisuje listu svih procesa sâmog korisnika, i opciono procesa drugih korisnika; ima oko 80 opcija (od kojih su mnoge neupotrebljive ili nekonzistentne)
- Ako se upotrebi bez opcija, komanda daje listu svih procesa pokrenutih u tom šelu:

```
→ ps
  PID TTY          TIME CMD
 4706 pts/2    00:00:01 bash
15007 pts/2    00:00:00 emacs
16729 pts/2    00:00:00 ps
```

Ovde: **TIME** je ukupno CPU vreme dodeljeno procesu (u minutama i sekundama) od njegovog početka; **CMD** je komanda koja je pokrenula proces (na listi će se uvek pojaviti jedan proces koji odgovara komandi **bash**, i jedan koji odgovara otkucanoj komandi **ps**)

- Da se dobije (pre)detaljan prikaz, i to svih procesa u sistemu, kuca se obično:

→ **ps -ef**

Opcija **-e** traži prikaz svih procesa svih korisnika; opcija **-f** traži "full format listing"; da se dobije lista sa celim komandama, kuca se opcija **-efww**; može se dodati i opcija **-less** da bi se prikazivalo ekran po ekran

- Izlaz gornje komande je na primer:

UID	PID	PPID	C	STIME	TTY	TIME	CMD
root	0	0	0	09:36:35	?	0:00	sched
root	1	0	0	09:36:35	?	0:02	/etc/init
mari	7997	1	10	09:49:32	ttyp3	0:04	ps-ef
mike	12923	11324	9	10:19:49	ttyp5	56:12	ssh

UID je ime korisnika, PPID je PID procesa-roditelja (eng. parent process -- neće se ulaziti u detalje), C označava koliko % od ukupnog raspoloživog vremena CPU dodeljuje tom procesu, STIME je vreme kada je proces započeo

- Još nekoliko primera:

- za prikaz svih procesa korisnika smith kuca se:

→ **ps -U smith**

mada se isto to može uraditi i pretragom cele liste pomoću **grep**:

→ **ps -ef | grep smith**

- za prikaz procesa čiji su PID 1, 2 i 3505:

→ **ps -p1,2,3505**

- za prikaz svih procesa koje je pokrenula komanda emacs:

→ **ps -uax | grep emacs**

uptime

- Jednostavna komanda **uptime** daje informaciju koliko vremena je proteklo od poslednjeg butovanja sistema; to vreme se zapravo naziva uptime

- Primer:

→ **uptime**

10:54pm up 8 days, 3:44, 3 users,
load average: 0.89, 1.00, 2.15

Informacije su redom: trenutno vreme (10:54 PM); uptime (8 dana, 3 sata, 44 minuta); broj korisnika koji su se logovali (3); prosečan broj procesa u periodu od 1 minuta (0.89), 5 minuta (1.00) i 15 minuta (2.15)

htop [options]

- Komanda **htop** (od "table of processes") omogućava interaktivni pregled (monitoring) svih procesa u realnom vremenu (slično kao Task Manager u Windows OS ali mnogo detaljnije); ona je zamenila zastarelu komandu **top**; mora se instalirati
- Ova komanda periodično apdejtuje svoj prikaz (na primer, svake sekunde); prikaz je jedna raznobojna tabela koja prikazuje pregršt informacija; komanda ekvivalentna ovoj, ali za praćenje rada I/O uređaja je **iostat**, a za praćenje rada mreže je **iftop**
- Primer izgleda ekrana posle kucanja komande **htop** dat je na Slici 18-1

CPU[		0.0%]					Tasks: 103, 181 thr; 2 running				
Mem[		611M/993M]					Load average: 0.00 0.03 0.15				
Swp[		11.0M/2.00G]					Uptime: 00:43:30				
PID	USER	PRI	NI	VIRT	RES	SHR	S	CPU%	MEM%	TIME+	Command
9942	root	20	0	117M	2148	1432	R	0.0	0.2	0:00.08	htop
2671	mysql	20	0	885M	83308	1056	S	0.0	8.2	0:00.35	/usr/libexec/mysqld -
2589	root	20	0	540M	14188	3612	S	0.0	1.4	0:00.31	/usr/bin/python -Es /
1	root	20	0	120M	3040	1752	S	0.0	0.3	0:01.32	/usr/lib/systemd/syst
461	root	20	0	35096	2536	2352	S	0.0	0.2	0:00.18	/usr/lib/systemd/syst
487	root	20	0	123M	684	684	S	0.0	0.1	0:00.00	/usr/sbin/lvmetad -f

Slika 18-1: rezultat komande **htop**

- U zaglavlju ekrana su date informacije o radu CPU, o stanju memorije, kao i taskovi i uptime; zatim je prikazana lista svih procesa, sortirana po tome koliko proces koristi CPU; na dnu (nije prikazano na slici) su prikazane razne opcije za prikaz: Help, Setup, Search, Filter, Tree, SortBy, Nice i slično

- Još nekoliko primera:

- da se prikaz na ekranu apdejtuje sa željenim vremenskim periodom, kuca se:

→ **htop -d 10**

gde se brojna vrednost unosi u desetinkama sekunde (ovde se postavlja da se ekran osvežava na svaku sekundu)

- da se prikažu podaci samo o procesima sa zadatim PID:

→ **htop -p 1,2,3,3891**

- da prikaz bude sortiran po koloni PID (to jest po brojnim vrednostima PID):

→ **htop -s PID**

free [options]

- Komanda **free** (mora se instalirati) ispisuje korišćenje memorije u kilobajtima:

→ **free**

	total	used	free	shared	buffers	cached
Mem:	523812	491944	31868	0	67856	199276
-/+ buffers/cache:		224812	299000			
Swap:	530104	0	530104			

- Najbolja estimacija količine slobodne RAM memorije koja se može napraviti na osnovu prethodnog izlaza ona se nalazi u vrsti **buffers/cache**, u koloni **free** (u ovom primeru, 299000K)

- Pregled važnih opcija:

- s *N* Run continuously and update the display every *N* seconds.
- b Display amounts in bytes.
- m Display amounts in megabytes.
- t Add a totals row at the bottom.
- o Don't display the "buffers/cache" row.

Tema 19: Kontrola procesa

- Glavne komande vezane za kontrolu procesa su:

kill Terminate a process (or send it a signal).

timeout Kill a command that runs for too long.

nice Invoke a program at a particular priority.

renice Change a process's priority as it runs.

flock Ensure that only one copy of a process runs at the same time, using locks.

- Korisnik može da obavlja razne operacije nad procesima: on može da zaustavi proces, da ga ponovo pokrene (pa da proces nastavi gde je stao), da mu postavi drugačiji prioritet, i još ponešto

kill [options] [process_ids]

- Pomoću komande **kill**, proces se može **ubiti** (eng. to kill), bez ostavljanja mogućnosti da se nastavi; kao sinonim koristi se pojam **obustaviti** (eng. to terminate); to je default opcija; uz to, proces se može i **prekinuti** (eng. to interrupt), to jest zaustaviti uz mogućnost da se ponovo nastavi – neće se raditi

- Komanda funkcioniše tako što procesu šalje odgovarajući **signal**; postoji oko 30 tipova signala; numerisana lista svih se može videti pomoću **kill -l**; default signal je SIGTERM (broj 15): njega proces može ignorisati; postoji i "jači" signal SIGKILL (broj 9) koga proces ne može ignorisati – ovo se naziva forsirano obustavljanje, i uvek daje rezultat

- Korisnik može da ubije samo svoj proces, a superkorisnik može da ubije svaki proces svakog korisnika; na primer, da se ubije/obustavi proces čiji je PID jednak 13243, kuca se:

→ **kill 13243**

- Kao što je pomenuto, neki programi (proces) jednostavno ignorišu ovako zadatu komandu, to jest signal SIGTERM koga su od nje dobili: da bi se obustavljanje zasigurno odradilo, koristi se pomenuti signal SIGKILL:

→ **kill -KILL 13243**

Kada se tip signala piše kao opcija, tekst SIG se izostavlja

- Treba voditi računa o tome da ova poslednja komanda ne pravi "čisti izlaz" (eng. clean exit) iz programa, jer može da se desi da neki resursi ostanu dodeljeni procesu (ili da se stvore neki drugi problemi) i posle njegovog obustavljanja; takav proces se naziva **zombi**

- Ako korisnik ne zna koji je PID procesa, može da ga sazna kao što je pokazano pomoću komande **ps** ili (još bolje) pomoću komande **pidof**, koja će da prikaže PID procesa na osnovu imena odgovarajućeg programa:

→ **pidof emacs**
8374

- Korisnik može da ubije proces u jednoj liniji, samo na osnovu naziva programa, pomoću ranije objašnjenog backquote:

→ **kill `pidof emacs`**

- Korisnik takođe može da koristi komandu **killall** da obustavi sve procese za dati program, na primer:

→ **killall emacs**

timeout [options] seconds command...

- Komanda **timeout** definiše (zadaje) maksimalno trajanje izvršenja nekog programa, u sekundama; navodi se zatim odgovarajuća komanda (koja pokreće taj program); ako izvršenje traje duže od definisanog limita, komanda **timeout** će ubiti odgovarajući proces
- Trivijalni primer:

→ **timeout 3 sleep 60** *Killed after 3 seconds*

Pokrenuta je komanda **sleep** koja u Linuxu služi da se čeka, to jest "spava" zadato vreme; to vreme ovde je 60 sekundi; ali posle 3 sekunde, komanda **timeout** ubija odgovarajući proces "spavanja"

- Kao praktičniji primer, korisnik može da zada da se reprodukuju svi muzički MP3 fajlovi tokom jednog sata, i da se zatim reprodukcija obustavi:

→ **timeout 3600 mplayer *.mp3**

- Pregled važnih opcija:

-s signal Send a signal other than the default (TERM). The choices are the same ones listed by **kill -l**.

-k seconds If the program doesn't die after the first signal, wait this many seconds longer and send a deadly KILL signal.

nice [-n level] command_line

- Kada korisnik hoće da pokrene neku komandu (odnosno proces) za koju zna da će biti "skúpa" (eng. expensive), to jest da će zahtevati mnogo računarskih resursa (posebno CPU vremena), on može da bude ljubazan/uviđavan (eng. nice) prema ostalim procesima (to jest korisnicima) tako što će smanjiti prioritet svog procesa; za to služi komanda **nice**
- Drugim rečima, ako se nekom procesu smanji prioritet, on će manje "zahtevati pažnje" od process schedulera, i tako će ustupiti mesto ostalim (na primer, važnijim ili manje skupim) procesima da dobiju priliku da se završe brže
- Koliko je korisnik rešio da bude ljubazan, pokazuje vrednost parametra koji se naziva **ljubaznost** (eng. niceness) koju je odabrao; što je ljubaznost veća, to je prioritet koga korisnik traži za svoj proces manji (i obrnuto)
- Ljubaznost je celobrojni parametar koji može iznositi od -20 (traži se najviši prioritet procesa) do 19 (traži se najniži prioritet procesa); difolt vrednost za ljubaznost je 0
- Na korisniku je znači da odredi, prilikom upotrebe komande **nice**, koliku će ljubaznost odabrati; što je proces skuplji, preporučljivo je da zada veću vrednost ljubaznosti; ta vrednost treba da bude zadata na osnovu procene i iskustva
- Sa druge strane, korisnik ne može da bude "neljubazan/neuviđavan" (preciznije: bezobrazan) pa da poveća prioritet svog procesa tako što će smanjiti ljubaznost u odnosu na difolt vrednost 0 -- to može samo superkorisnik; znači: korisnik može zadati ljubaznost u opsegu od 0 do 19, a superkorisnik od -20 do 19

- Primer:

→ **nice -n 7 sort hugefile > outfile**

- Uz komandu **nice** istovremeno se zadaje i odgovarajuća komanda; u ovom primeru sortira se ogroman fajl, pa korisnik očekuje da će proces biti skup, i povećava ljubaznost sa default vrednosti 0 na vrednost 7, uz pomoć opcije **-n**; ako se uz komandu **nice** ta vrednost ne zada, podrazumeva se 10

- Superkorisnik, kao što je pomenuto, može da poveća prioritet svakog procesa:

→ **sudo nice -n -7 important_script**

Ovim se ljubaznost podešava na -7; nije preporučljivo da se zadaje vrednost manja od -15, jer time može da se naruši rad vitalnih sistemskih procesa

- Da bi korisnik video vrednost ljubaznosti (obično označenu sa NI) za neki proces sa poznatim PID, to može da uradi na više načina, a jedan od njih je:

→ **ps -o nice -p PID**

Opcija **-o** služi da se zada koji parametar procesa korisnik hoće da prikaže, a sa **nice** traži da to bude ljubaznost

renice [-n N] [options] PID

- Komanda **renice** menja prioritet procesa, to jest ljubaznost, u toku njegovog izvršenja (za razliku od **nice** koja pokreće izvršenje programa uz zadati prioritet); da bi komanda **renice** mogla da se upotrebi, potrebno je da odgovarajući proces bude nevidljiv

- Za razliku od komande **nice** kojoj se zadaje vrednost ljubaznosti, kod komande **renice** se (iz nepoznatih razloga) zadaje inkrement (promena); pri tom, taj inkrement (takode iz nepoznatih razloga) može da bude samo pozitivan, to jest ljubaznost se uz pomoć **renice** može samo (dodatno) povećati, ali ne i smanjiti

- Na primer, na sledeći način se povećava ljubaznost za 5, procesu sa poznatim PID:

→ **renice -n 5 -p PID**

- Ilustrativni primer: korisnik pokrene u pozadini, sa ljubaznošću 8, proces koji "spava" jedan minut; zatim poveća ljubaznost za 3 (znači na 11) u toku izvršenja:

```
→ nice -n 8 sleep 60 &  
[1] 4908
```

```
→ ps -o nice -p 4908
```

```
NI
```

```
8
```

```
→ renice -n 3 -p 4908
```

```
→ ps -o nice -p 4908
```

```
NI
```

```
11
```

- Pregled važnih opcija:

-p pid Affect the given process ID. You can omit the -p and just provide a PID (renice -n 5 28734).

-u username Affect all processes owned by the given user.

flock [options] lockfile command...

- Pomoću komande **flock** ("file lock" = fajl za zaključavanje) sprečava se paralelno izvršenje neke dve komande, to jest programa, za koje iz nekih razloga nije dobro da se paralelno izvršavaju

- Drugim rečima, ako je pomoću komande **flock** naznačeno da program_1 i program_2 ne smeju paralelno da se izvršavaju, onda: ako se program_1 izvršava i pokuša sa pokretanjem programa_2, taj pokušaj će biti bezuspešan, i to:

>> pomoću opcije **-n**, pokretanje programa_2 će trenutno biti onemogućeno

>> pomoću opcije **-w** sačekaće se zadato vreme, i pokušati ponovo sa pokretanjem programa_2 (koje će biti uspešno ako se u međuvremenu program_1 završio)

- Da bi se navedeno ostvarilo, potrebno je i `program_1` i `program_2` pokrenuti koristeći komandu **flock**, i na oba mesta navesti ime istog lock fajla (eng. lockfile); kao lock fajl, najbolje je napraviti jedan prazan fajl, kao na primer `/tmp/mylock`

- Na taj način će, kada se `program_1` pokrene, u lock fajlu biti upisana odgovarajuća oznaka (koja znači: "program_1 se trenutno izvršava"); ako se zatim pokuša sa pokretanjem programa_2, proveriće se taj lock fajl, ustanoviti da postoji ta oznaka, i pokretanje neće biti moguće

- Ovo je praktično, na primer, ako je korisnik podesio da se automatski bekap (pomoću komande **rsync**) obavlja na svakih sat vremena: ako se radi o velikoj količini podataka, može se desiti da prethodni bekap nije završen u trenutku kada se naredni pokreće: ovo bi moglo da izazove ozbiljne probleme u radu, pa se zato koristi naredba **flock**

- Navedeno se obavlja na sledeći način:

→ **flock -n /tmp/mylock rsync ...**

Izvršenje **rsync** komande, koja se startuje pomoću **flock**, neće biti pokrenuto ukoliko se jedna instanca iste komande još uvek izvršava (a zadata je bila takođe pomoću **flock** i uz navođenje istog lock fajla)

- Da bi se videla ova komanda na trivijalnom primeru, mogu se otvoriti dva šela i u svakom od njih upisati na primer:

→ **flock -n /tmp/mylock sleep 60**

U prvom šelu izvršenje komande **sleep** će započeti, ali u drugom neće; to bi se naravno desilo i za svaku drugu komandu koja bi se otkucala u drugom šelu, i koja koristi isti lock fajl

- Pregled važnih opcija (već su pomenute):

- n Instantly fail if another command is already running.
- w *N* Fail after waiting *N* seconds, if another command is already running.

Tema 20: Zakazivanje poslova

- Glavne komande za zakazivanje (eng. scheduling) obavljanja poslova (na primer, da se zadati program startuje u određeno vreme ili periodično) su:

sleep Wait a set number of seconds, doing nothing.

watch Run a program at set intervals.

at Schedule a job for a single, future time.

crontab Schedule jobs for many future times.

sleep time_specification

- Komanda **sleep** ne radi ništa, to jest "spava" (čeka) tokom definisanog vremenskog intervala; zadata vrednost mora biti ceo broj, a može se posle broja staviti oznaka jedinice, kao što je **s** (sekunde -- difolt), zatim **m** (minute), **h** (sati) i **d** (dani); primer:

→ **sleep 5m** *Do nothing for 5 minutes*

- Komanda je pogodna kada korisnik hoće da se neka druga komanda startuje sa vremenskom zadržkom (eng. time delay):

→ **sleep 10 && echo 'Ten seconds have passed.'**

(10 seconds pass)

Ten seconds have passed.

watch [options] command

- Komanda **watch** izvršava zadatu komandu **periodično**, po difoltu na svake 2 sekunde; rezultati se ispisuju u full-screen modu, kako bi korisnik mogao da prati nastanak eventualnih promena; na primer:

```
watch -n 60 date
```

izvršava komandu **date** na svakih 60 sekundi; za izlazak se kuca ^C

- Pregled važnih opcija:

-n seconds Set the time between executions, in seconds.

-d Highlight differences in the output, to emphasize what has changed from one execution to the next.

-g Exit when the command produces output that is different from the previous execution.

at [options] time_specification

- Komanda **at** (=u koje vreme), koja se mora instalirati, pokreće zadatu komandu **jednom**, u zadato vreme (terminu); drugim rečima, ona zakazuje izvršenje komande u zatom terminu; preciznije, može se zadati i sekvenca komandi

- Primer za sekvencu komandi:

→ **at 15:27**

at> echo Remember to go shopping

at> lpr shopping_list

at> <EOT>

job 559 at Fri Nov 11 15:27:00 2022

- Pošto korisnik otuca prvu liniju iz primera:

>> pojavljuje se u narednoj liniji prompt **at>**, gde korisnik otuca komandu pa Enter

>> u novoj liniji se pojavljuje ponovo **at>**, gde kuca narednu komandu (koja će se izvršiti u isto vreme) -- tako se unosi željena sekvenca komandi...

>> kada korisnik hoće da završi sa unosom komandi, pritisne **^D**

>> na kraju se ispiše informacija o rednom broju odgovarajućeg posla (559), i zakazani termin u kome će se sekvenca komandi izvršiti (istog dana u 15:27)

- Umesto pomoću prompta, može se termin izvršenja zadati i pomoću ulančavanja:

```
→ echo "command_to_be_run" | at 09:00
```

Da bi se izvršio željeni šel skript u određeno vreme, koristi se opcija **-f**, kao na primer:

```
→ at 09:00 -f /home/linuxize/script.sh
```

- Korisnik ima veliku fleksibilnost prilikom zadavanja vremenskog termina; može zadati: vreme pa datum, ili samo datum (što podrazumeva zadatog dana, a u isto vreme kad se pokrene komanda **at**), ili samo vreme (što podrazumeva prvi put kad bude toliko sati, istog ili narednog dana)

- Uz to, korisnik može zadati i specijalne reči, kao **now** ili **midnight**; konačno, može zadati i sve od navedenog ali sa zadatim ofsetom, na primer **+3 days**

- Primeri: **december 25 2015**, **25 december 2015**, **december 25**, **25 december**, **12/25/2015**, **25.12.2015**, **20151225**, **today**, **thursday**, **next thursday**, **next month**, **next year**; nazivi meseci mogu biti skraćeni na tri slova (**jan**, **feb**...); zadavanje vremena je takođe fleksibilno: **8pm**, **8 pm**, **8:00pm**, **8:00 pm**, **20:00**, **2000**

- Pri zadavanju vremena, pravi se referenca u odnosu na aktuelni trenutak (zadavanja komande **at**), pa tako **"next year"** znači **"naredne godine u isto ovo vreme"**, **"thursday"** znači **"sledećeg četvrtka u isto ovo vreme"**

- Treba imati u vidu da će komanda zadata uz **at** biti pokrenuta tek u zadatom terminu, tako da će na primer wildcard i varijable biti evaluirani tek tada (a ne u vreme pokretanja **at**)

- Da se pogleda lista zakazanih poslova, koristi se komanda **atq** ("at queue"):

```
→ atq
559 2015-09-14 07:00 a smith
```

a da se zakazani posao obriše, koristi se komanda **atrm** ("at remove"):

```
→ atrm 559
```

- Pregled važnih opcija:

```
-f filename    Read commands from the given file instead of standard
                input.
```

```
-c job_number  Print the job commands to standard output.
```

crontab [options] [file]

- Pre objašnjenja komande **crontab**, treba objasniti nekoliko uvodnih pojmova; prvo, **daemon** (srpski prevod "demon" se ne koristi) je Linux program koji se izvršava (to jest proces) neprekidno i u pozadini, bez uticaja i interakcije sa korisnikom (koji ga samo eventualno podesi)

- Postoji mnogo daemona koji se automatski pokreću prilikom podizanja sistema i ne smeju da se prekinu, jer su neophodni za ispravan rad samog sistema; na primer, postoje daemoni koji non-stop proveravaju ispravnost rada hard-diskova

- Pokretanje i zaustavljanje daemona korisnik može da obavi pomoću komande **service**:

```
→ service naziv_daemona start
→ service naziv_daemona stop
```

Lista svih daemona prikazuje se pomoću:

```
→ service --status-all
```

- Mogu se pomenuti sledeći demoni (nazivi demoni se obično završavaju slovom d radi raspoznavanja): sshd (opslužuje zahteve klijenata preko SSH konekcije), syslogd (System logging daemon – prikuplja sistemske poruke), cupsd (CUPS printer daemon) i httpd (Web server daemon - klijentima isporučuje fajlove pomoću HTTP protokola)
- Daemon pod nazivom **crond** služi za obavljanje periodičnih (repetitivnih) poslova: on se "budi" na svaki minut (ostalo vreme "spava"), proveriti listu ranije zakazanih poslova, pokrene izvršenje onih za koje je došlo vreme, i nastavi da "spava"
- Na pomenuti način korisnik može da automatizuje izvršenje komandi i skriptova, to jest odgovarajućih poslova koji se nazivaju cron poslovi (eng. cron jobs), tako da se znači izvršavaju periodično
- Na primer, ako korisnik stavi na listu cron poslova bekap podataka svake ponoći, kada se crond bude "probudio" u 0:00 sati i video na listi da je došlo vreme za bekap, pokreneće odgovarajuću komandu, zatim nastaviti da "spava" do 0:01 i tako dalje
- Pomenuta lista cron poslova nalazi se u jednom posebnom tekstualnom fajlu koji se naziva crontab (od eng. cron table) fajl; svaki korisnik ima svoj takav fajl; crontab fajlovi svih korisnika se nalaze u sistemskom direktorijumu /var/spool/cron
- Za kreiranje i uređivanje crontab fajla koristi se komanda **crontab**; primeri upotrebe:
 - **crontab -e**
pokreće (od edit) editovanje crontab fajla u difolt editoru;
 - **crontab -l**
ispisuje (od list) sadržaj crontab fajla na standardnom izlazu;
 - **crontab -r**
briše (od remove) crontab fajl;

→ crontab myfile

postavlja myfile kao crontab fajl

- Superkorisnik može da koristi opciju **-u** (od user) da bi pristupao crontab fajlovima ostalih korisnika
- U jednoj liniji crontab fajla zakazuje se jedan posao (prazne linije, kao i one koje počinju karakterom # se ignorišu); svaka linija sadrži 6 polja, razdvojena razmakom: prvih 5 polja definišu vreme/termin kada komanda treba da se startuje (minut, sat, dan, mesec, dan u nedelji), a šesto polje je sâma komanda
- **Minut** se zadaje kao ceo broj, od 0 do 59; to može biti jedan broj (30), zatim sekvenca odvojena zarezima (0,15,30,45), opseg (20-30) ili njihova kombinacija (0-15,50-59); zvezdica * označava "sve"; sufiks /n označava "na svakih n"; na primer, */12 i 0-59/12 označavaju "od 0 do 59, na svakih 12 minuta", to jest 0,12,24,36,48
- **Sat** se zadaje istom sintaksom kao minut, sa vrednostima od 0 do 24; slično i **dan** (u smislu: redni broj dana u mesecu), sa vrednostima od 1 do 31; **mesec** takođe (od 1 do 12) ali se mogu koristiti i troslovne skraćenice jan, feb, mar...
- **Dan u nedelji** se zadaje kao ceo broj, gde 0 označava nedelju, 1 ponedeljak i tako dalje, sintaksa je ista kao za minut, a mogu se koristiti i troslovne skraćenice sun, mon, tue...
- Primeri navedenih 5 polja (minut, sat, dan, mesec, dan u nedelji, objašnjenje):

* * * * * Every minute

45 * * * * * 45 minutes after each hour (1:45, 2:45, etc.)

45 9 * * * Every day at 9:45 am

45	9	8	*	*	The eighth day of every month at 9:45 am
45	9	8	12	*	Every December 8 at 9:45 am
45	9	8	dec	*	Every December 8 at 9:45 am
45	9	*	*	6	Every Saturday at 9:45 am
45	9	*	*	sat	Every Saturday at 9:45 am
45	9	*	12	6	Every Saturday in December, at 9:45 am

- Komanda koja se izvršava (polje broj 6) može biti bilo koja šel komanda (ili skript), koja će se izvršavati u korisnikovom login okruženju, tako da ona može koristiti varijable okruženja (kao što je \$HOME)

- Preporuka je da se za komande koriste apsolutne putanje do odgovarajućih izvršnih fajlova (na primer, /usr/bin/who umesto **who**), jer (kao što je pomenuto) Linux može imati više programa sa istim imenom

- Primer linije u crontab fajlu:

45 23 * * 6 /home/scripts/export_dump.sh

određuje da se skript fajl export_dump.sh pokreće svake subote u 23:45

- Ako se zakazana komanda izvršila i proizvela neki izlaz, korisniku će biti poslat odgovarajući i-mejl sa tim izlazom

Tema 21: Superkorisnik

- Pre nego što se pređe na složenije pojmove, prvo je potrebno napraviti malu rekapitulaciju; na prvom mestu: svaki Linux sistem ima svog administratora (admina) čiji je posao da se stara i preuzima odgovornost za ispravno funkcionisanje sistema

- Admin se bavi i instalacijom Linuxa (što je prvi korak u njegovom korišćenju); tokom te instalacije, on treba da odabere lozinku za superkorisnički nalog (odgovarajuće korisničko ime je root, UID je 0, home direktorijum je /root), koja se naziva superkorisnička lozinka (koristi se i termin root lozinka, ali će ovde biti izbegavan)
- Superkorisnička lozinka mora biti posebno dobro odabrana, čuvana, i menjana periodično (preporuka: na 3 meseca), i kad god admin posumnja da je neko neovlašćen mogao da je sazna; jedini ko zna tu lozinku je isključivo admin, i nikom je ne daje ni pod kakvim uslovima
- Zatim, prvi obični nalog koga admin otvara je njegov lični nalog; kasnije on kreira, u svojstvu superkorisnika, i druge naloge (za ostale korisnike), kao i grupe; svaki put kada počne sa radom, admin može da bira da li će se ulogovati kao superkorisnik ili kao obični korisnik
- Kao što je pomenuto, obični korisnici mogu da menjaju (osim u izuzetnim slučajevima kad im je drugačije dozvoljeno) samo one fajlove čiji su vlasnici; superkorisnički status/privilegije daju pun pristup celom sadržaju sistema, i omogućavaju sve raspoložive operacije nad njim
- Za mnoge važne operacije u Linux sistemu potreban je superkorisnički status; sa druge strane, taj status admin treba da koristi namenski i pažljivo, da ne bi slučajno poremetio rad sistema; naime, Linux je otvoren sistem u kome superkorisnik ima veliku slobodu ali može mnogo toga nepovratno i da pokvari ili uništi
- Stoga postoji preporuka da admin koristi superkorisnički nalog samo po potrebi, dok u ostalo vreme on radi i obavlja svakodnevne zadatke kao i svi drugi korisnici, pomoću svog "običnog" naloga

- Postoje dva načina da admin postane superkorisnik (to jest da dobije taj status): prvi i direktni način je da se normalno uloguje i radi kao superkorisnik; međutim, ovaj način iz mnogih bezbedonosnih razloga (neće se objašnjavati) treba obavezno izbegavati; uz to, neke Linux distro ovo uopšte i ne dozvoljavaju
- Drugi (i jedini koji praktično preostaje) način da admin postane superkorisnik je pomoću komande **su** (od "switch user" = promeni korisnika)

su [options] [-] [user]

- Generalno, ova komanda omogućava korisniku X da se uloguje i otvori novi šel kao neki drugi korisnik Y; pošto X otkuca komandu **su** za kojom sledi korisničko ime korisnika Y, treba da upiše i njegovu lozinku (koju X treba da zna, izuzev ako je X superkorisnik – od njega se lozinka i ne traži), posle čega mu se otvara šel (isti kakav vidi Y kad se uloguje)
- Usput se može navesti da je ovakva "promena" korisnika pogodna kada na primer kolega X (admin ili neki drugi) kolegi Y treba da pomogne u radu ili otklanjanju nekog problema: to će X najlakše uraditi ako vidi sve ono što vidi Y
- Ako se to drugo korisničko ime (Y) ne navede, ono je po difoltu root (to jest korisničko ime superkorisnika); ovo je ono šta koristi admin, koji će zatim otkucati superkorisničku lozinku, i otvoriće mu se **šel superkorisnika**: sve šta kuca u tom šelu biće u statusu superkorisnika
- Znači, admin kuca:

```
→ su -l
Password: *****          root password
#
```

Može se primetiti da je prompt promenjen u **#**, što je znak superkorisničkog statusa; opcija **-l** znači da prozor šela koji se otvori bude isti kao pri normalnom logovanju

- Kada završi sa poslom u statusu superkorisnika, admin otkuca **exit** ili **^D** kako bi zatvorio šel superkorisnika, i nastavlja rad kao običan korisnik; naravno, iz razloga sigurnosti, ovo ne sme da zaboravi da uradi
- Komanda **su** ima i opciju **-c** (od "command") koja omogućava da se u istoj liniji unese i neka komanda koja će biti odrađena u superkorisničkom statusu, naravno uz unos superkorisničke lozinke, ali bez otvaranja superkorisničkog šela; ovo je pogodno za obavljanje nekih sitnih zadataka
- Međutim, postoji i jedan još praktičniji način da se neka komanda pokrene u statusu superkorisnika; naime, admin može nekom običnom korisniku dati ovlašćenje da pokrene neke komande u statusu superkorisnika; pri tom, nije potrebno da taj obični korisnik zna superkorisničku lozinku
- Komanda koja se koristi u navedenu svrhu je **sudo**; ona znači omogućava da se "premosti" preveliki jaz između privilegija superkorisnika (koji može sve u sistemu) i običnog korisnika (koji može vrlo ograničeno), tako što će znači nekim običnim korisnicima biti dozvoljeno da odrade neke operacije sa superkorisničkim privilegijama

sudo [command]

- Posle komande **sudo** korisnik kuca željenu komandu, koju bi da izvrši u superkorisničkom statusu; od korisnika će se tražiti da unese **svoju** (ne superkorisničku!) lozinku; ako on ima (unapred datu) dozvolu admina, uneta komanda će se i izvršiti; posle te komande, on nastavlja svoj rad kao običan korisnik

- Primer:

→ **sudo rm protected_file**

Password: *********

Your own password

- Admin koristi sistemski fajl /etc/sudoers kako bi naznačio kome od korisnika dozvoljava da pokrene koju komandu pomoću komande **sudo**; korisnik kome je to dozvoljeno naziva se "sudoer"(= onaj kome je dozvoljeno da izvrši **sudo**)
- Da bi se sudoeru olakšao rad, sistem omogućava da on samo prilikom prve **sudo** komande unese svoju lozinku, i da narednih 5 minuta (ovo vreme se može i promeniti) koristi **sudo** bez unošenja lozinke, i radi sve za šta ima dozvolu
- Finalna napomena: da sâm admin ne bi morao da unosi superkorisničku lozinku da bi prešao u superkorisnički status, on obično svoj (lični, to jest obični) nalog doda na listu sudoer-a, i dâ sebi sve neophodne dozvole; tako i on može da koristi komandu **sudo** za obavljanje superkorisničkih poslova, ali uz kucanje svoje lične lozinke
- Neke Linux distro ne dozvoljavaju da admin, iz razloga bezbednosti, čak ni pomoću **su** postane superkorisnik; one zahtevaju da on, kao što je pomenuto, obavlja superkorisničke poslove kao sudoer, i da piše komandu **sudo** pre svake komande koju treba izvršiti u superkorisničkom statusu (kao što je ovde i prikazano)

Tema 22: Ulogovanje, izlogovanje i isključenje

- Na početku svog rada, korisnik se **uloguje** (eng. login); po završetku rada, on se **izloguje** (eng. logout); superkorisnik može da isključi sistem (eng. shutdown ili halt) pomoću komande **shutdown** ili **systemctl**, a može da obavi i rebootovanje/restart sistema; Linux sistemi su stabilni, i ove operacije se retko rade

shutdown [options] time [message]

- Komanda **shutdown** dovodi do isključenja ili rebootovanja Linux sistema; u sledećem primeru, sistem se isključuje kroz 10 minuta, uz ispisivanje poruke "scheduled maintenance" svim ulogovanim korisnicima (kako bi stigli da završe šta su započeli):

→ **sudo shutdown -h +10 "scheduled maintenance"**

- Obavezni parametar time ove komande može biti apsolutno vreme u časovima i minutima (na primer 16:25), a može biti i broj minuta sa znakom + (kao u primeru), čime se zadaje vremenska zadržka/kašnjenje navedenog trajanja; konačno, parametru se može zadati i vrednost now (što znači da se sistem isključuje odmah – nije preporučljivo)

- Ako joj se ne zadaju opcije, komanda **shutdown** stavlja sistem u single-user mod, koji služi za održavanje sistema: u njemu je ulogovan samo jedan korisnik (admin), i onemogućeni su svi servisi koji nisu esencijalni za sistem

- Da bi se izašlo iz single-user moda, treba: ili ponoviti komandu **shutdown** da bi se sistem isključio (opcija **-h**) ili rebootovao (opcija **-r**), ili otkucati **^D** da se sistem vrati u normalni multi-user mod

- Pregled važnih opcija:

-r Reboot the system.

-h Halt the system.

-k Kidding: don't really perform a shutdown, just broadcast warning messages to all users as if the system were going down.

-c Cancel a shutdown in progress (omit the *time* argument).

-f On reboot, skip the usual filesystem check performed by the **fsck** program

-F On reboot, require the usual filesystem check.

systemctl [options] command [arguments]

- U nekim Linux distribucijama, komanda **shutdown** je samo simbolički link za komandu **systemctl** -- to je višenamenska komanda koja pokreće i zaustavlja razne servise; ona je deo novog softverskog paketa koji se naziva **systemd**; preciznije, to je daemon koji upravlja radom celog sistema

- Primeri upotrebe:

sudo systemctl poweroff Shut down the system.

sudo systemctl reboot Reboot the system.

sudo systemctl suspend Suspend the system.

Tema 23: Korisnici i njihova okruženja

- Glavne komande za rad sa korisnicima (njihovim imenima, vremenima logovanja i slično) i njihovim okruženjima (eng. environment) su:

logname Print your login name.

whoami Print your current, effective username.

id Print the user ID and group membership of a user.

who List logged-in users, long output.

users List logged-in users, short output.

finger Print information about users.

last Determine when someone last logged in.

printenv Print your environment.

logname

- Komanda **logname** ispisuje korisniku na ekranu korisničko ime (username) pod kojim se ulogovao; ovo izgleda trivijalno, ali može biti od koristi u šel skriptovima:

```
→ logname  
smith
```

a isti efekat se postiže i prikazivanjem vrednosti varijable okruženja **LOGNAME**:

```
→ echo $LOGNAME
```

whoami

- Komanda **whoami** ispisuje na ekranu **efektivno korisničko ime** (eng. effective username):
>> za admina, ovo ime nije uvek isto kao njegovo "obično" korisničko ime pod kojim se ulogovao (detalji slede)
>> za običnog korisnika, ovo ime je uvek isto kao i njegovo korisničko ime (pod kojim se ulogovao), pa će njemu komanda **whoami** uvek dati isti izlaz kao i **logname**

- Admin se, kao što je objašnjeno, uvek inicijalno loguje pomoću svog običnog korisničkog imena (na primer, smith), i zatim radi kao i svi drugi korisnici; njegovo efektivno korisničko ime je tada isto kao njegovo obično korisničko ime:

```
→ whoami  
smith
```

- Samo po potrebi, pomoću komande **sudo**, admin jednokratno prelazi u superkorisnički status; ostvaruje ga sâmim kucanjem komande **sudo** – komanda koja sledi posle nje je superkorisnička; tada je njegovo efektivno korisničko ime root:

```
→ sudo whoami  
root
```

- Sa druge strane, komanda **logname** ispisuje ime pod kojim se admin ulogovao, bez obzira da li u tom trenutku ima superkorisnički status ili ne:

```
→ logname
smith
→ sudo logname
smith
```

- Treba napomenuti i da sve što je navedeno za admina važi i za sudoere (jer i oni ostvaruju superkorisnički status); takođe, sve navedeno važi i kada se koristi komanda **su**

id [options] [username]

- Kao što je ranije objašnjeno, svaki korisnik ima svoj UID (user ID), i član je jedne ili više grupa, od kojih je jedna primarna, i od kojih svaka ima svoj GID (group ID); komanda **id** ispisuje sve navedene informacije (numeričke vrednosti), zajedno sa odgovarajućim imenima (korisničkim imenom i grupnim imenima)

- Primer:

```
→ id
uid=500(smith) gid=500(smith)
groups=500(smith),6(disk),490(src),501(cdwrite)
```

U prvoj liniji prikazani su UID, i GID primarne grupe; u drugoj liniji (groups) su prikazani GID svih grupa čiji je korisnik član (sve sa odgovarajućim imenima); očigledno je primarna grupa ovog korisnika njegova privatna grupa, a on je uz to član i još tri grupe

- Ako se komanda **id** upotrebi sa korisničkim imenom (ili UID) kao argumentom, dobija se ista informacija ali za drugog (navedenog) korisnika sistema

- Pregled važnih opcija:

- u Print the effective user ID and exit.
- g Print the effective group ID and exit.
- G Print the IDs of all other groups to which the user belongs.
- n Print names (for users and groups) rather than numeric IDs. Must be combined with -u, -g, or -G. For example, `id -Gn` produces the same output as the `groups` command.
- r Print login values instead of effective values. Must be combined with -u, -g, or -G.

who [options] [filename]

- Komanda **who** ispisuje listu svih ulogovanih korisnika; ako neki korisnik koristi N šelova, njegovo ime biće ispisano N puta; primer:

```
→ who
smith    pts/0      Sep  6 17:09 (:0)
barrett  pts/1      Sep  6 17:10 (10.24.19.240)
jones    pts/2      Sep  8 20:58 (192.168.13.7)
jones    pts/4      Sep  3 05:11 (192.168.13.7)
```

Prikazano je vreme kada se korisnik ulogovao, i IP adresa sa koje sa daljinski ulogovao (ako je to slučaj); postoji i komanda **users** koja ispisuje samo korisnička imena

- Po defaultu, komanda **who** preuzima podatke iz sistemskog fajla `/var/run/utmp`, ali pomoću argumenta `filename` može da joj se zada i neki drugi fajl za tu namenu, kao što je: `/var/log/wtmp` za loginove iz prošlosti, ili `/var/log/btmp` za neuspele loginove (pod uslovom da je sistem konfigurisan da beleži takve podatke)

- Pregled važnih opcija:

- H Print a row of headings as the first line.
- -lookup For remotely logged-in users, print the hostnames of origin.
- u Also print each user's idle time at his/her terminal.
- m Display information only about yourself (i.e., the user associated with the current terminal).
- q Quick display of usernames only, and a count of users. Much like the `users` command, but it adds a count.

`finger [options] [user[@host]]`

- Komanda **finger**, prilično slična prethodnoj (na nekim distro treba da se instalira), ispisuje informacije o svim ulogovanim korisnicima:

→ **finger**

Login	Name	Tty	Idle	Login Time
smith	Sandy Smith	:0		Sep 6 17:09
barrett	Daniel Barrett	:pts/1	24	Sep 6 17:10
jones	Jill Jones	:pts/2		Sep 8 20:58

- Može dati i detaljnije informacije o jednom korisniku:

→ **finger smith**

```

Login: smith                               Name: Sandy Smith
Directory: /home/smith                     Shell: /bin/bash
On since Sat Sep 6 17:09 (EDT) on :0
Last login Mon Sep 8 21:07 (EDT) on pts/6 from web1
No mail.
Project:
Enhance world peace
Plan:
Mistrust first impulses; they are always right.
```

- Argument `user` je korisničko ime lokalnog korisnika, ili udaljenog korisnika u obliku `user@host`; udaljeni hostovi će odgovoriti na komandu **finger** samo ako su tako konfigurisani

- Pregled važnih opcija:

- l **Print in long format.**
- s **Print in short format.**
- p **Don't display the Project and Plan sections, which are ordinarily read from the user's `~/.project` and `~/.plan` files, respectively.**

last [options] [users] [ttys]

- Komanda **last** ispisuje istoriju logovanja, u obrnutom hronološkom redu (poslednji login prvi na listi); prikazuju se vremena kad se korisnik ulogovao i kad se izlogovao, kao i trajanje sesije:

→ **last**

```
bob pts/3 localhost Mon Sep 8 21:07 - 21:08 (00:01)
sue pts/6 :0          Mon Sep 8 20:25 - 20:56 (00:31)
bob pts/4 myhost      Sun Sep 7 22:19 still logged in
```

- Može se zadati i korisničko ime jednog korisnika, da se vidi kada se logovao samo on; podaci koje ova komanda očitava i ispisuje nalaze se u fajlu `/var/log/wtmp`

- Pregled važnih opcija:

- N **Print only the latest *N* lines of output, where *N* is a positive integer.**
- i **Display IP addresses instead of hostnames.**

- R Don't display hostnames.
- x Also display system shutdowns and changes in system runlevel (e.g., from single-user mode into multiuser mode).
- f *filename* Read from some other data file than */var/run/wtmp*; see the *who* command for more details.

printenv [environment_variables]

- Komanda **printenv** ispisuje sve varijable okruženja (eng. environment variable) koje su šelu dostupne, i njihove vrednosti:

```
→ printenv
HOME=/home/smith
MAIL=/var/spool/mail/smith
NAME=Sandy Smith
SHELL=/bin/bash
```

- Mogu se navesti jedna ili više konkretnih varijabli, kako bi se prikazale samo njihove vrednosti:

```
→ printenv HOME SHELL
/home/smith
/bin/bash
```

Tema 24: Upravljanje korisničkim nalogima

- Glavne komande za upravljanje korisničkim nalogima (eng. user account management) su:

```
useradd  Create an account.
userdel  Delete an account.
usermod  Modify an account.
```

passwd Change a password.
chfn Change a user's personal information.
chsh Change a user's shell.

- Tokom instalacije Linuxa, kao što je objašnjeno, od onoga ko obavlja instaliranje (a to je administrator sistema) se zahteva da kreira superkorisnički nalog (čije je korisničko ime uvek root) i još jedan (običan) korisnički nalog za sebe; on će naknadno dodavati i (obične) naloge drugim korisnicima sistema

- Kreiranje/dodavanje novih korisnika (naloga) je važan zadatak administratora: svaki kreirani nalog je moguća prilika za neovlašćena zlonamerna lica da dobiju pristup sistemu; svaki korisnik, kao prvi korak, mora imati jaku korisničku lozinku, tešku za pogađanje

useradd [options] username

- Pomoću komande **useradd**, superkorisnik kreira novi korisnički nalog:

→ **sudo useradd smith**

Zadaje se korisničko ime smith, a UID će mu biti automatski dodeljen (može ga dodeliti i admin, ali se to ne preporučuje)

- Pravila i preporuke za kreiranje korisničkog imena: samo mala slova i (ako je baš potrebno) cifre; preporuka je da se ime jednoobrazno kreira na sledeći način: prvo slovo imena za kojim ide prvih 6 slova prezimena (znači za Petra Petrovića: ppetrov)

- Ova komanda se uvek koristi sa opcijama, kao na primer:

→ **sudo useradd -d /home/smith -s /bin/bash **
-g users smith

Ovim se korisniku dodeljuju: home direktorijum (opcija **-d**), difolt šel (opcija **-s**) i primarna grupa (opcija **-g**) – ovo je osnovno šta superkorisnik treba da postavi

- Kao što je ranije napomenuto, navedenom linijom će se automatski kreirati i privatna grupa, koja će se takođe zvati smith; po defaultu će ona biti postavljena za korisnikovu primarnu grupu; pomoću opcije **-g** u gornjem primeru se to menja, i postavlja da mu je primarna grupa users (potrebno je naravno da takva grupa postoji)

- Pomoću opcije **-G** se navode i druge/dodatne grupe (odvojene zarezima) u koje se korisnik prima kao član (sve te grupe naravno treba da postoje), znači u gornjoj liniji je moglo da piše i:

... -G webadmins,developers ...

- Korisna opcija **-C** (od eng. comment) obično služi da se upiše puno ime i prezime korisnika pod navodnicima, znači u gornjoj liniji je moglo da piše i:

... -c "John Smith" ...

- Pregled važnih opcija:

-d <i>dir</i>	Set the user's home directory to be <i>dir</i> .
-s <i>shell</i>	Set the user's login shell to be <i>shell</i> .
-u <i>uid</i>	Set the user's ID to be <i>uid</i> . Unless you know what you're doing, omit this option and accept the default.
-c <i>string</i>	Set the user's comment field (historically called the GECOS field). This is usually the user's full name, but it can be any string. The <code>chfn</code> command can also set this information.
-g <i>group</i>	Set the user's initial (default) group to <i>group</i> , which can either be a numeric group ID or a group name, and which must already exist.

-G *group1,group2,...* Make the user a member of the additional, existing groups *group1, group2*, and so on.

userdel [-r] *username*

- Pomoću komande **userdel** briše se postojeći korisnički nalog:

→ **sudo userdel smith**

Po defaultu, ovim se ne brišu fajlovi u korisnikovom home direktorijumu; da bi se to uradilo, koristi se opcija **-r**

- Ovakvo brisanje je bespovratno; preporuka je da administrator uvek prvo deaktivira/zaključa nalog, pomoću **usermod -L** (detalji u nastavku); takođe, treba da obezbedi bekap svih fajlova korisnika čiji nalog se briše, jer će možda zatrebati u budućnosti

usermod [options] *username*

- Pomoću komande **usermod**, administrator menja razna svojstva datog korisničkog naloga; na primer, home direktorijum korisnika menja se na sledeći način:

→ **sudo usermod -d /home/another smith**

- Pregled važnih opcija (**-d**, **-s**, **-g**, **-G** slične kao za **useradd**):

-d <i>dir</i>	Change the user's home directory to <i>dir</i> .
-s <i>shell</i>	Change the user's login shell to <i>shell</i> .
-g <i>group</i>	Change the user's initial (default) group to <i>group</i> , which can either be a numeric group ID or a group name, and which must already exist.

- G *group1,group2,...* Make the user a member *only* of the additional, existing groups *group1, group2*, and so on. If the user previously belonged to other groups, but you don't specify them here, the user will no longer belong to them.
- L Disable (lock) the account so the user cannot log in.
- U Unlock the account after a lock (-L) operation.

passwd [options] [username]

- Komanda **passwd** služi za promenu korisničke lozinke; takođe, prilikom kreiranja novog korisničkog naloga (pomoću **useradd**), da bi nalog postao validan potrebno je da kreator naloga (superkorisnik) pomoću **passwd** definiše inicijalnu lozinku, na primer:

→ **passwd smith**

- Po defaultu, korisnik sâm sebi menja lozinku:

→ **passwd**

posle čega treba da ukuca staru lozinku, pa novu, i novu još jednom

- Preporuka za kreiranje lozinke je: bar 8 karaktera, od kojih bar po jedno: malo slovo, veliko slovo, cifra i specijalni karakter; može se koristiti i generator lozinke kao što je passwordsgenerator.net; primer tako generisane (odlične) lozinke: D_T>ie0y
- Lozinke se, zajedno sa ostalim podacima o korisnicima (korisničko ime, UID, GID i slično), čuvaju u fajlu /etc/passwd, za svakog korisnika po jedna linija; ovom fajlu može svako pristupiti, ali se lozinka ne može pročitati: ili je tu zapisana kao kriptovana, ili je napisano samo x, što znači da se čuva u zaštićenom fajlu /etc/shadow

- Superkorisnik može da promeni lozinku nekog korisnika:

→ **sudo passwd smith**

s tim što on ne treba da ukucava staru lozinku; ako korisnik zaboravi svoju lozinku, superkorisnik na ovaj način definiše novu (koju korisnik kasnije može promeniti)

- Ova komanda ima opcije, većina od njih vezane za dužinu trajanja validnosti lozinke; treba ih koristiti samo u slučaju dobro osmišljene politike bezbednosti

chfn [options] [username]..

- Komanda **chfn** ("change finger") dodaje/menja neke podatke o korisniku u njegov nalog: puno ime, brojeve telefona i slično, a koje prikazuje komanda **finger**

- Ako korisnik pozove **chfn** bez korisničkog imena, promena će se ticati njegovog naloga; ako superkorisnik pozove komandu sa nečijim korisničkim imenom, promena će se ticati naloga tog korisnika

- Ako se ne unesu opcije, **chfn** će tražiti od korisnika da otkuca željene informacije:

→ **chfn**

Password: *********

Name [Shawn Smith]: Shawn E. Smith

Office [100 Barton Hall]:

Office Phone [212-555-1212]: 212-555-1234

Home Phone []:

U uglastim zagradama pišu postojeći podaci

- Pregled važnih opcija:

-f *name* Change the full name to *name*.

-h *phone* Change the home phone number to *phone*.

- p *phone* Change the office phone number to *phone*.
- o *office* Change the office location to *office*.

chsh [options] [username]

- Komanda **chsh** ("change shell") definiše difolt šel program koga će korisnik koristiti; ako korisnik pozove komandu bez korisničkog imena, promena će se ticati njegovog naloga; ako superkorisnik pozove komandu sa nekim korisničkim imenom, promena će se ticati naloga tog korisnika

- Ako se ne unesu opcije, komanda će od korisnika tražiti da navede šel:

```
→ chsh
Changing shell for smith.
Password: *****
New shell [/bin/bash]: /bin/tcsh
```

Kao šel se može odabrati neki od onih koji su navedeni u fajlu /etc/shells

- Pregled važnih opcija:

```
-s shell Specify the new shell.
-l          List all permissible shells.
```

Tema 25: Upravljanje grupama

- Glavne komande za upravljanje grupama korisnika (eng. group management) su:

```
groups      Print the group membership of a user.
groupadd    Create a group.
groupdel    Delete a group.
groupmod    Modify a group.
```

- **Grupa** je skup korisnika (svaki sa svojim nalogom) koju Linux tretira kao samostalni entitet; ako grupa ima dozvolu da nešto uradi (na primer, da modifikuje neki fajl), onda ta dozvola važi za svakog korisnika člana te grupe; grupe formira i njima upravlja admin

- Već je pokazano kako se menja vlasništvo grupe nad fajlovima – primer:

```
→ chgrp friends /tmp/sample
→ chmod g=rwx /tmp/sample
```

U prvoj liniji, grupa friends se postavlja kao vlasnik fajla /tmp/sample; u drugoj liniji, novopostavljena grupa-vlasnik dobija dozvolu da očitava, upisuje i izvršava taj fajl

- Dodavanje novog člana (korisnika) u grupu superkorisnik može obaviti i modifikacijom fajla /etc/group (različite distribucije Linuxa čuvaju listu članova grupe na različite načine)

groups [usernames]

- Komanda **groups** ispisuje korisniku listu svih grupa kojima on pripada, ili kojima pripada neki drugi korisnik; primer:

```
→ groups
smith users
→ groups jones root
jones : jones users
root : root bin daemon sys adm disk wheel src
```

Korisnik smith (koji kuca komandu) pripada grupama smith (njegova privatna) i users; on takođe vidi i kojim grupama pripadaju druga dva korisnika (jones i root)

groupadd [options] group

- Pomoću komande **groupadd** kreira se nova grupa; da bi se izbeglo da se unese ime već postojeće grupe, preporučljivo je koristiti opciju **-f**:

→ **sudo groupadd -f friends**

Sistem novoj grupi automatski dodeljuje GID (to može ručno da uradi i admin, ali nije preporučljivo)

- Pregled važnih opcija:

-g gid Specify your own numeric group ID instead of letting groupadd choose one.

-f If the specified group exists already, complain and exit.

groupdel group

- Pomoću komande **groupdel** briše se postojeća grupa:

→ **sudo groupdel friends**

- Pre nego što se ovo (bespovratno) uradi, preporučljivo je da se identifikuju svi fajlovi čiji je vlasnik ta grupa, tako da se kasnije reši šta sa njima da se radi:

→ **sudo find / -group friends -print**

Ovim se pronalaze svi postojeći fajlovi čiji je grupa-vlasnik friends

- Razlog za to je što komanda **groupdel** ne menja grupno vlasništvo nad fajlovima; ona samo briše ime grupe iz svih zapisa u sistemu; ako se izlistaju ti fajlovi, umesto imena izbrisane grupe videće se njen (bivši) GID

groupmod [options] group

- Komanda **groupmod** menja podatke o datoj grupi, tako što na primer menja njeno ime (može se menjati i GID ali se to ne preporučuje):

→ **sudo groupmod -n newname friends**

Ona ne utiče na fajlove koji su vlasništvo ove grupe, niti na druga svojstva grupe

- Pregled važnih opcija:

-n name Change the group's name to *name* (safe).

-g gid Change the group's ID to *gid* (risky).

Tema 26: Ispisivanje na ekranu

- Glavne komande za ispisivanje teksta na ekranu su:

echo Print simple text on standard output.

printf Print formatted text on standard output.

yes Print repeated text on standard output.

seq Print a sequence of numbers on standard output.

clear Clear the screen or window.

- Svaka od komandi za ispisivanje teksta na ekranu (to jest na standardnom izlazu) ima svoju namenu i funkcionalnosti; one su neizostavne kada korisnik piše šel skripte

echo [options] strings

- Komanda **echo** ispisuje svoj argument na ekranu:

→ **echo We are having fun**
We are having fun

- Zabunu može da unese to što postoji nekoliko (!?) **echo** komandi koje nisu sasvim iste: postoji `/bin/echo`, ali je Linux šel obično zanemaruje na račun komande **echo** koja je u njega ugrađena (eng. builtin); da bi korisnik ipak bio siguran koja od te dve se koristi, kuca:

```
→ type echo
echo is a shell builtin
```

- Pregled važnih opcija:

- n Don't print a final newline character.
- e Recognize and interpret escape characters. For example, try `echo 'hello \a'` and `echo -e 'hello\a'`. The first prints literally and the second makes a beep.
- E Don't interpret escape characters: the opposite of -e.

- Raspoloživi eskejp (eng. escape) karakteri su:

<code>\a</code>	Alert (play a beep)
<code>\b</code>	Backspace
<code>\c</code>	Don't print the final newline (same effect as -n)
<code>\f</code>	Form feed
<code>\n</code>	Line feed (newline)
<code>\r</code>	Carriage return
<code>\t</code>	Horizontal tab
<code>\v</code>	Vertical tab
<code>\\</code>	A backslash
<code>\'</code>	Single quote
<code>\"</code>	Double quote
<code> nnn</code>	The character whose ASCII value is <i>nnn</i> in octal

- Primer:

```
→ echo -e 'a\tb'  
a      b
```

Znači prepoznat je eskejp karakter \t, čime je između a i b napravljen jedan Tab

printf *format_string* [*arguments*]

- Komanda **printf** je "obogaćeni" **echo**: ona ispisuje formatirani tekst; funkcioniše slično kao istoimena funkcija printf() programskog jezika C

- Ova komanda zahteva jedan obavezni argument, koji se naziva **format-string**; on se piše između navodnika; to je string koga će komanda ispisati, uz neke modifikacije

- Format-string sadrži $N \geq 0$ takozvanih **format-specifikacija**; svaka je označena karakterom % posle koga ide slovo (na primer, %s); njima se označavaju mesta, u okviru format-stringa, na kojima će se upisivati vrednosti N argumenata; te su vrednosti upisane u sekvenci, odvojene razmacima, posle format-stringa

- Primer:

```
→ printf "User %s is %d years old.\n" sandy 29  
User sandy is 29 years old.
```

- U primeru, format-string "... " sadrži $N=2$ format-specifikacije: %s (označava mesto gde će se upisati vrednost tipa string), a %d (označava mesto gde će se upisati vrednost tipa decimalni ceo broj); posle format-stringa su date dve vrednosti koje će se upisati na označenim mestima (od kojih je naravno prva string, a druga decimalni ceo broj)

- Postoje dve **printf** komande (?!): jedna ugrađena u bash šel, a druga je /usr/bin/printf; srećom, one se razlikuju samo u nekim sitnim detaljima

- Najvažnije format-specifikacije su:

%d **Decimal integer**
%x **Hexadecimal integer**
%f **Floating point**
%s **String**

- U okviru format-specifikacije, može se upisati celobrojna vrednost koja pokazuje minimalni broj karaktera u zapisu; tako na primer **%5d** označava petocifreni decimalni ceo broj, dok **%6.2f** označava realan broj sa šest cifara pre i dve posle decimalnog zareza

seq [options] specification

- Komanda **seq** ("sequence") ispisuje sekvencu (aritmetički niz) celih brojeva, pogodnu za ulančavanje sa drugim programima
- Ako se komandi zada samo jedan ceo broj, kreiraće se sekvenca celih brojeva od 1 pa do tog broja, u koracima po 1:

```
→ seq 3  
1  
2  
3
```

- Ako se zadaju dva cela broja, kreiraće se sekvenca celih brojeva od prvog pa do drugog broja (oba uključena), u koracima po 1:

```
→ seq 2 5  
2  
3  
4  
5
```

- Ako se zadaju tri cela broja, oni se interpretiraju kao donji limit, inkrement (korak) -- može biti i negativan, i gornji limit:

```
→ seq 5 -1 2
```

```
5  
4  
3  
2
```

clear

- Komanda `clear` briše sadržaj prozora šela

Tema 27: Datum i vreme

- Glavne komande za prikaz i zadavanje datuma i vremena na Linux sistemu su:

```
cal      Print a calendar.
```

```
date     Print or set the date and time.
```

```
ntpd     Set the system time using a remote timeserver.
```

`cal [options] [month [year]]`

- Komanda `cal` ispisuje kalendar za jedan mesec, po defaultu aktuelni/tekući:

→ **cal**

November 2015

Su	Mo	Tu	We	Th	Fr	Sa
1	2	3	4	5	6	7
8	9	10	11	12	13	14
15	16	17	18	19	20	21
22	23	24	25	26	27	28
29	30					

Za prikaz meseca avgusta 2016. kuca se **cal 8 2016**; ako se napiše samo godina (**cal 2016**), ispisaće se kalendar za tu celu godinu

date [options] [format]

- Komanda **date** ispisuje trenutni datum i vreme; format u kome će biti ispisani zavisi od lokalnih podešavanja sistema (država i jezik) -- primeri će biti dati za Ameriku i engleski; po difoltu, komanda ispisuje sistemski datum i vreme u lokalnoj vremenskoj zoni (srpska zona je CET = Central European Time):

→ **date**

Fri Mar 18 22:32:04 EDT 2016

- Format izlaza se može zadati pomoću format-stringa koji počinje znakom plus, na primer:

→ **date '+%D'**

03/18/16

→ **date '+The time is %l:%M %p on a lovely %A in %B'**

The time is 10:32 PM on a lovely Friday in March

- Tabela format-specifikacija:

Format	Meaning	Example (US English)
Whole dates and times:		
%c	Full date and time, 12-hour clock	Sun 28 Sep 2003, 09:01:25 PM EDT

%D	Numeric date, 2-digit year	09/28/03
%x	Numeric date, 4-digit year	09/28/2003
%T	Time, 24-hour clock	21:01:25
%X	Time, 12-hour clock	09:01:25 PM

Words:

%a	Day of week (abbreviated)	Sun
%A	Day of week (complete)	Sunday
%b	Month name (abbreviated)	Sep
%B	Month name (complete)	September
%Z	Time zone	EDT
%p	AM or PM	PM

Numbers:

%d	Day of month, leading zero	02
%e	Day of month, leading blank	2
%m	Month number, leading zero	09
%y	Year, 2 digits	03
%Y	Year, 4 digits	2003
%M	Minute, leading zero	09
%S	Seconds, leading zero	05
%l	Hour, 12-hour clock, leading blank	9
%k	Hour, 24-hour clock, leading blank	9
%H	Hour, 24-hour clock, leading zero	09
%s	Seconds since the beginning of Linux time: midnight January 1, 1970	1068583983

- Pomoću opcija se mogu ispisati neki drugi (ne trenutni/aktuelni) datum i vreme:

- d *string*** Display the given date or time *string*, formatted as you wish.
- r *filename*** Display the last-modified timestamp of the given file, formatted as you wish.
- s *string*** Set the system date and/or time to be *string*; only the superuser can do this.

ntpdate timeserver

- Komanda **ntpdate** postavlja aktuelno sistemsko vreme i datum, pošto kontaktira i dobije informaciju od nekog vremenskog servera (eng. timeserver) -- ovo može da uradi samo superkorisnik:

```
→ sudo /usr/sbin/ntpdate timeserver.someplace.edu
7 Sep 21:01:25 ntpdate[2399]: step time server
178.99.1.8
offset 0.51 sec
```

- NTP je skraćenica od Network Time Protocol, koji služi za sinhronizaciju računarskih sistema; za listu raspoloživih NTP vremenskih servera, treba pretražiti internet za pojam "public ntp time server"
- Da bi sistemsko vreme bilo stalno sinhronizovano sa nekim vremenskim serverom, treba koristiti daemon **ntpd** (www.ntp.org) – bez više detalja

Tema 28: Instaliranje softvera

- Admin, i neki korisnici Linuxa će zasigurno imati potrebu da instaliraju neki softver; načini instalacije variraju, jer Linux podržava nekoliko tipova instalacionih paketa (pa je i tu zbrka): tri najkorišćenija tipa su: RPM, DEB i TAB; zavisno od Linux distribucije, instaliranje se može obaviti iz GUI ili iz CLI (gde izbor komandi takođe zavisi od distro)

- Instaliranje softvera na velikim Linux sistemima je ozbiljan i obiman posao; korisnik ima na raspolaganju mnogo opcija i podešavanja; ovde se neće previše ulaziti u detalje, jer je potrebno sve dobro proraditi u praksi (za onoga ko će time da se bavi)
- RPM instalacioni paket (fajlovi *.rpm) se koriste u Red Hat, Fedora, CentOS i još nekim distribucijama Linuxa; za instalaciju se koriste komande: **dnf**, **yum** i **rpm**
- DEB instalacioni paket (fajlovi *.deb) se koriste u Debian, Ubuntu i još nekim distribucijama Linuxa; za instalaciju se koriste komande: **apt-get**, **dpkg** i **aptitude**
- TAR instalacioni paket (fajlovi *.tar.gz i *.tar.bz2) je, vidi se iz naziva, komprimovana kolekcija fajlova, to jest ne radi se o instalacionom paketu u pravom smislu reči; instalacija na ovaj način je komplikovanija u odnosu na DEB i RPM, jer zahteva više manuelnih koraka -- neće biti objašnjavano
- Znači, pre nego što krene sa instaliranjem, korisnik mora da zna koji tip paketa je potreban za njegov Linux sistem, to jest koji paket podržava njegova distribucija; na sreću, moderni Linux sistemi već imaju package manager, tako da je na korisniku samo da ga koristi
- Korisnik može da proveriti koju Linux distribuciju (i verziju) koristi na jedan od sledećih načina (zavisno od distro):

```
→ cat /etc/os-release
→ cat /etc/issue
→ hostnamectl
→ cat /etc/*release
→ lsb_release -a
```

Isto se može uraditi i iz GUI

- Većinu softvera instalira admin to jest superkorisnik, tako da se prilikom instalacije obično koristi **sudo** komanda; primer:

```
→ sudo rpm -ivh mypackage.rpm
Password: *****
```

- Svaka Linux distribucija ima i GUI program za instalaciju, ali je instaliranje sa komandne linije, kao i obično, vrlo često fleksibilnije i jednostavnije (i biće prikazano); u svakom slučaju, na korisniku je da odabere ono šta njemu odgovara

dnf [options] [packages]

- Komanda **dnf** je najnovija komanda za RPM instalacione pakete; sledeća tabela prikazuje listu najčešćih primera kako se ona koristi

Action	dnf command
Search for a package that meets your needs (supports wildcards * and ?)	<code>dnf search <i>command_name</i></code>
Check if a package is installed	<code>dnf list installed <i>package_name</i></code>
Download a package but don't install it	<code>dnf download <i>package_name</i></code>
Download and install a package	<code>sudo dnf install <i>package_name</i></code>
Install a package file	<code>sudo dnf install <i>file.rpm</i></code>
Learn about a package	<code>dnf info <i>package_name</i></code>

List the contents of a package	<code>rpm -ql <i>package_name</i></code>
Discover which package an installed file belongs to	<code>dnf provides <i>/path/to/file</i></code>
Update an installed package	<code>sudo dnf upgrade <i>package_name</i></code>
Remove an installed package	<code>sudo dnf remove <i>package_name</i></code>
List all packages installed on the system	<code>dnf list installed less</code>
Check for updates for all packages on the system	<code>dnf check-update</code>
Update all packages on the system	<code>sudo dnf upgrade</code>

`yum [options] [packages]`

- Komanda `yum` je često korišćena komanda za RPM instalacione pakete; postoje izvedena i u GUI, na primer u paketu PackageKit za Fedoru
- Naredna tabela prikazuje tipične operacije koje se izvode pomoću ove komande:

Action	yum command
Search for a package that meets your needs (supports wildcards * and ?)	<code>yum search <i>command_name</i></code>
Check if a package is installed	<code>yum list installed <i>package_name</i></code>

Download a package but don't install it. Requires the <code>downloadonly</code> plugin; to install it, run:	<code>sudo yum --downloadonly install <i>package_name</i></code>
Download and install a package	<code>sudo yum install <i>package_name</i></code>
Install a package file	<code>rpm -ivh <i>package.rpm</i></code>
Learn about a package	<code>yum info <i>package_name</i></code>
List the contents of a package	<code>rpm -ql <i>package_name</i></code>
Discover which package an installed file belongs to	<code>yum provides <i>/path/to/ file</i></code>
Update an installed package	<code>sudo yum update <i>package_name</i></code>
Remove an installed package	<code>sudo yum remove <i>package_name</i></code>
List all packages installed on the system	<code>yum list installed less</code>
Check for updates for all packages on the system	<code>yum check-update</code>
Update all packages on the system	<code>sudo yum update</code>

Ova komanda ne radi operacije nad lokalnim fajlovima, za to se koristi **rpm**

`rpm [options] [files]`

- Ako korisnik hoće da daunloaduje i instalira RPM paket ručno, treba da koristi komandu **rpm**; ovu komandu zapravo u pozadini pokreće i **yum**; ali za razliku od **yum**, **rpm** radi na lokalnom računaru, i ne traži instalacioni softver po internetu
- Komanda **rpm** ne samo što instalira softver, već i proverava da li sistem ispunjava sve zahteve za instalaciju; na primer, ako paket A zahteva da na računaru prethodno bude instaliran paket B, rpm neće instalirati A pre nego što se instalira B
- Nazivi RPM fajlova obično su sledećeg oblika: name-version.architecture.rpm; na primer, *emacs-23.1-17.i386.rpm* označava emacs paket, verziju 23.1-17, za računare Intel 386 i više; komanda rpm nekad kao argument traži kompletan naziv RPM fajla, a nekad samo naziv paketa (kao na primer emacs)
- Naredna tabela prikazuje tipične operacije koje se izvode pomoću ove komande:

Action	rpm command
Check if a package is installed	<code>rpm -q package_name</code>
Install a package file	<code>sudo rpm -ivh package_file.rpm</code>
Learn about a package	<code>rpm -qi package_name</code>
List the contents of a package	<code>rpm -ql package_name</code>
Discover which package an installed file belongs to	<code>rpm -qf /path/to/file</code>
Update an installed package	<code>sudo rpm -Uvh package_file.rpm</code>

Remove an installed package `sudo rpm -e package_name`

List all packages installed on the system `rpm -qa | less`

`apt-get [options] packages` `apt-file [options] string`

`apt-cache [options] packages` `dpkg [options] packages`

- Paket komandi APT ("Advance Packaging Tool", koji se sastoji od 4 komande: `apt-get`, `apt-file`, `apt-cache` i `dpkg`) služi da instalira, briše i manipuliše DEB instalacionim paketima

- Naredna tabela prikazuje tipične operacije:

Action	APT command
Search for a package that meets your needs	<code>apt-file search package_name</code>
Check if a package is installed	<code>dpkg -s package_name</code>
Download a package but don't install it	<code>apt-get -d package_name</code>
Download and install a package	<code>sudo apt-get install package_name</code>
Install a package file	<code>dpkg -i package_file.deb</code>
Learn about a package	<code>apt-cache show package_name</code>
List the contents of a package	<code>dpkg -L package_name</code>

Discover which package an installed file belongs to	<code>dpkg -S /path/to/file</code>
Update an installed package	<code>sudo apt-get upgrade package_name</code>
Remove an installed package	<code>sudo apt-get remove package_name</code>
List all packages installed on the system	<code>dpkg -l</code>
Check for updates for all packages on the system	<code>sudo apt-get -u upgrade</code>
Update all packages on the system (to include kernel packages, replace upgrade by dist-upgrade)	<code>sudo apt-get upgrade</code>

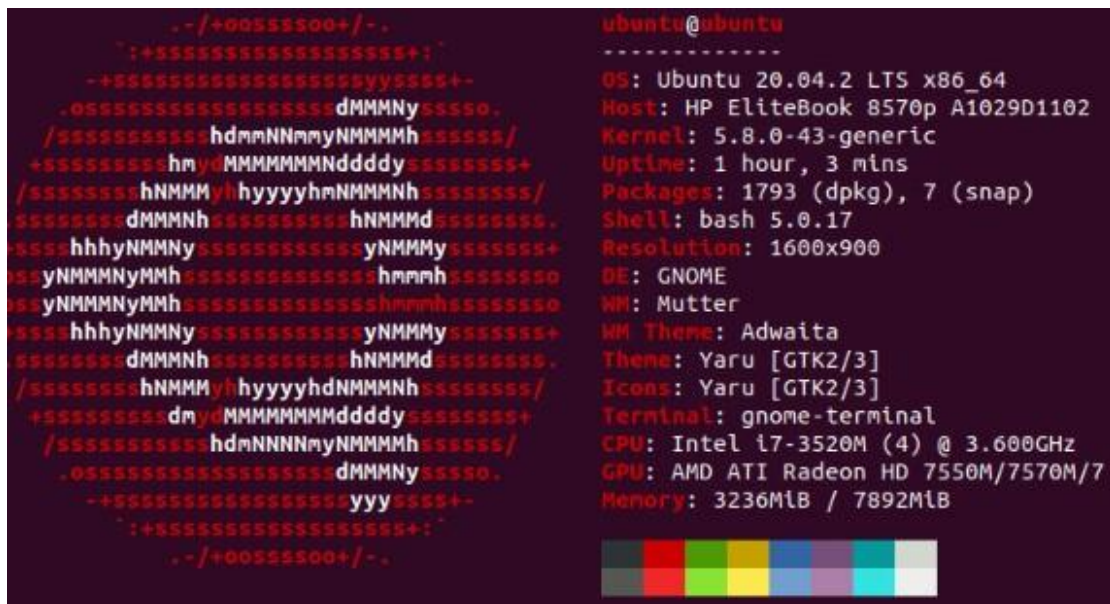
`aptitude [options] [packages]`

- Komanda **aptitude** je alternativa paketu komandi APT, koja takođe radi sa DEB instalacionim paketima; korisnik često mora da u svakom slučaju koristi **dpkg** jer komanda **aptitude** ne omogućava rad sa lokalnim .deb fajlovima
- Naredna tabela prikazuje tipične operacije:

Action	aptitude command
Search for a package that meets your needs	<code>aptitude search package_name</code>
Check if a package is installed (examine the output for "State: not installed" or "State: installed")	<code>aptitude show package_name</code>

Download a package but don't install it	<code>aptitude download package_name</code>
Download and install a package	<code>sudo aptitude install package_name</code>
Install a package file	<code>dpkg -i package_file.deb</code>
Learn about a package	<code>aptitude show package_name</code>
List the contents of a package	<code>dpkg -L package_name</code>
Discover which package an installed file belongs to	<code>dpkg -S /path/to/file</code>
Update an installed package	<code>sudo aptitude safe- upgrade package_name</code>
Remove an installed package	<code>sudo aptitude remove package_name</code>
List all packages installed on the system	<code>aptitude search '~i' less</code>
Check for updates for all packages on the system	<code>aptitude --simulate full- upgrade</code>
Update all packages on the system	<code>sudo aptitude full-upgrade</code>

- Primer: instalacija jednog modernog i vrlo popularnog alata Neofetch, koji se koristi za prikazivanje raznih informacija o sistemu; korisnik može da bira šta i kako će se prikazivati, u raznim bojama (Slika 28-1)



Slika 28-1: Difolt ekran Neofetch-a

- Način instalacije zavisno od distro:

→ sudo apt install neofetch	Ubuntu
→ sudo pacman -S neofetch	Arch
→ sudo dnf install neofetch	Fedora
→ sudo apt-get install neofetch	Debian
→ sudo zypper install neofetch	OpenSUSE

Tema 29: Programiranje pomoću šel skriptova

- Kao što je pomenuto, bash šel poseduje ugrađen jednostavni programski jezik; to omogućava korisniku da piše jednostavne programe koji se nazivaju šel skriptovi (muški rod, jednina: skript, eng. script), kako bi odradio neke složenije zadatke koji se ili ne mogu uopšte odraditi kucanjem komandu po komandu, ili mogu ali na komplikovan način
- Kao i svaki drugi programski jezik, šel ima varijable, kondicionale (if-then-else), petlje, ulaz/izlaz i slično; postoje čitave knjige koje su namenjene samo pisanju šel skriptova to jest Linux programiranju

29.1 Pisanje i pokretanje šel skriptova

- Da se **kreira** šel skript, jednostavno se bash komande ispišu jedna ispod druge u tekstualnom fajlu, kao kad bi se redom kucale jedna za drugom; tako se zapravo kreira programski kôd, zato Linux (sa punim pravom) posmatramo i kao programski jezik

- Primer: korisnik pokrene editor u kome će kucati skript:

→ **nano myscript**

zatim otkuca linije koda (o prvoj liniji biće reči u nastavku):

```
#!/bin/bash  
echo hello
```

posle čega snimi fajl (^O) i izađe (^X); sadržaj skripta se uvek može videti pomoću:

→ **cat myscript**

- Da se **pokrene** izvršenje napisanog šel skripta, postoji dva načina:

1) kreiranjem izvršnog fajla od šel skripta

2) startovanjem pomoću komande **bash**

- U načinu 1, koji je jednostavniji, kao prvu liniju šel skripta treba napisati već prikazani string **#!/bin/bash** (u načinu 2 to nije potrebno)

- Zatim se koristi ranije objašnjena komanda **chmod** za promenu moda fajla, tako što će se skriptu dodati dozvola za izvršenje:

→ **chmod u+x myscript**

- Na kraju, dobijeni izvršni fajl (program) se startuje na uobičajeni način, zadavanjem istoimene komande:

→ **myscript**

- Preporuka je da fajl myscript ostane u onom direktorijumu gde ga je korisnik napisao (na primer, /users/home/smith/myscripts); takođe, preporuka je da se taj direktorijum **ne** dodaje u šelovu putanju pretrage (kako korisnik ne bi greškom napisao skript sa istim nazivom kao neka postojeća šel komanda); zato se može desiti da šel ne može da pronade komandu myscript ("command not found")

- Da bi komanda bila pronađena, potrebno je da se direktorijum gde se nalazi myscript proglasi za radni, i da se komanda zatim startuje kao:

→ **./myscript**

Drugo (još sigurnije) rešenje je da se otkuca cela putanja do navedenog fajla

- U načinu 2, komandi **bash** se ime skripta samo zada kao argument, i ona izvrši skript:

→ **bash myscript**

- Napomena: u pisanju skripta, kao i u običnom pisanju komandi, ako se neka duga komanda mora rastaviti u više linija, svaku (izuzev poslednje) linije treba završiti beksleš karakterom \ koji u ovom kontekstu označava "nastavak u narednoj liniji"; primer linije iz gornjeg skripta:

```
echo hello \  
world
```

29.2 Variable

- Šel varijable/promenljive su ranije objašnjene; primer šel varijable **MYVAR** tipa integer (ceo broj):

```
→ MYVAR=6  
→ echo $MYVAR  
6  
→ expr $MYVAR + 1  
7
```

- Podsećanje: kada se pravi referenca na varijablu **VARIJABLA** (kada se nalazi sa desne strane dodele, ili kada se nad njom obavlja neka operacija), ispred njenog naziva se dodaje karakter **\$** (koji znači "vrednost"), znači piše se **\$VARIJABLA**

- Treba voditi računa da se u izrazu pišu razmaci; ako se to ne uradi, zadata operacija se ne obavi kako treba:

```
→ expr $MYVAR+1  
6+1
```

- Varijabla može biti i tipa string, označen pomoću navodnika; ako je string numerički (svi karakteri su mu cifre), šel ga tretira kao ceo broj, pa se nad njim može obaviti neko izračunavanje; primer:

```
→ NUMBER="10"  
→ expr $NUMBER + 5  
15
```

- Kada se u šel skriptu pravi referenca na neku varijablu, jedna preporuka je da se njen naziv piše pod navodnicima, kako bi se sprečile neke greške (kao što je runtime error – neće biti objašnjavano); znači, preporučljivo je da referenca na varijablu bude uvek **"\$VARIJABLA"**

- Jedan od razloga za ovo: varijabla tipa string koji sadrži razmake može prouzrokovati pogrešne rezultate ukoliko nije označena navodnicima; kao što je ranije pokazano, razmake u svim nazivima obavezno treba izbegavati (jer generišu i razne druge probleme); na njihovom mestu treba koristiti donju crtu **_**

- Primer:

→ FILENAME="My Document"	<i>Space in the name</i>
→ ls \$FILENAME	<i>Try to list it</i>
ls: My: No such file or directory	<i>ls saw 2 arguments</i>

```
ls: Document: No such file or directory
→ ls -l "$FILENAME"           List it properly
My Document                    ls saw only 1 argument
```

Znači, po preporuci, treba koristiti `My_Document` umesto `My Document`

29.3 Ulaz i izlaz

- Kao što je objašnjeno, skript ispisuje svoj izlaz pomoću komandi `echo` i `printf`:

```
→ echo "Hello world"
Hello world
→ printf "I am %d years old\n" `expr 20 + 20`
I am 40 years old
```

- Ulaz za skript se obezbeđuje pomoću komande `read`, koja očitava jednu liniju iz standardnog ulaza, i čuva je kao varijablu:

```
→ read NAME
John Smith <ENTER>
→ echo "I read the name $NAME"
I read the name John Smith
```

29.4 Povratni kodovi i testiranje vrednosti varijable

Povratni kôd

- Svaka Linux komanda, pošto se izvrši, vraća šelu jednu celobrojnu (integer) varijablu koja se naziva **povratni kôd** (eng. return code; koristi se i pojam izlazni status, eng. exit status, ali će se ovde izbegavati); ona može uzeti vrednosti od 0 do 255
- Kada šel dobije taj povratni kôd, interpretira ga na sledeći (ne baš intuitivni) način:
>> vrednost 0 znači da nije bilo greške, to jest da je izvršenje bilo uspešno

>> svaka druga (nenulta, od 1 do 255) vrednost znači da je došlo do neke greške, to jest da izvršenje nije bilo uspešno

- Drugim rečima, povratni kod se može intepretirati kao odgovor na pitanje "da li je prilikom izvršenja došlo do greške?", pri čemu se 0 interpretira kao "ne" a svaka druga vrednost (1,2,...255) kao "da"

- Pri tome, na osnovu te nenulte vrednosti može se (u određenoj meri) zaključiti kakva je greška u izvršenju nastupila, to jest kakav se problem pojavio koji je sprečio izvršenje

- Znači, povratni kôd se interpretira na sledeći način:

>> 0: izvršenje OK

>> od 1 do 255: desila se neka greška u izvršenju, pri čemu sâma ta celobrojna vrednost predstavlja **kôd greške** (eng. error code)

- Autor komande (programer), tokom njenog kreiranja, može da bude manje ili više precizan kada su u pitanju kodovi greške koje definiše; što je autor bio u tome precizniji, to je korisniku lakše da obavi debugovanje; za svaku komandu dati su odgovarajući kodovi greške na njenoj man stranici

- U Linuxu postoje i **standarni kodovi greške**; tako na primer:

>> 1: "generalna greška" (kao na primer deljenje nulom), nije od pomoći pri debugovanju

>> 2: neispravna upotreba komande

>> 126: komanda se ne može izvršiti (na primer, ako korisnik nema dozvolu)

>> 127: komanda nije pronađena

>> 130: izvršenje prekinuto pomoću ^C

- Sa druge strane, kada korisnik piše svoj skript, od koristi je da definiše sopstvene kodove greške pomoću komande **exit** (više detalja u nastavku); pri tom, preporučljivo je da izbegava standardne kodove, i da za svoje potrebe koristi opseg između 11 i 49

- Vrednost povratnog kôda poslednje komande se čuva u specijalnoj varijabli **\$?**; primer:

```
→ cat myfile
My name is Sandy Smith and
I really like Ubuntu Linux
→ grep Smith myfile
My name is Sandy Smith and      A match was found...
→ echo $?
0                                ...so return code is "success"
→ grep aardvark myfile
→ echo $?
1                                No match was found...
                                ...so return code is "failure"
```

- U primeru, oba puta pošto se izvrši komanda **grep**, u narednoj liniji se ispiše vrednost specijalne varijable **\$?**, i tako prikaže vrednost povratnog koda; u slučaju komande **grep**, vraća se 0 kao znak da je pretraga dala rezultat, i 1 ako nije

Testiranje vrednosti varijable

- Ugrađena komanda **test** testira/ispituje vrednosti string i numeričkih promenljivih; ona može testirati i neka svojstva fajlova, kao na primer datum kreiranja; sintaksa joj je:

test *expression*

- Kao rezultat tog testiranja, komanda **test** ne ispisuje rezultat na ekranu, ali vraća povratni kôd koji može imati samo dve vrednosti (takođe kontra-intuitivno):

>> 0: ako je rezultat testiranja tačan

>> 1: ako je rezultat testiranja netačan (formalno, ovo je znači kôd greške)

Testiranje se koristi za građenje kondicionala (detalji u nastavku)

- Primeri testiranja:

```
→ test 10 -lt 5      Is 10 less than 5?
→ echo $?
1                    No, it isn't
→ test -n "hello"    Does "hello" have nonzero length?
→ echo $?
0                    Yes, it does
```

Opcije `-lt` i `-n` objašnjene su u nastavku

- Opcije za testiranje fajlova:

```
-d name      File name is a directory
-f name      File name is a regular file
-L name      File name is a symbolic link
-r name      File name exists and is readable
-w name      File name exists and is writable
-x name      File name exists and is executable
-s name      File name exists and its size is nonzero
f1 -nt f2    File f1 is newer than file f2
f1 -ot f2    File f1 is older than file f2
```

- Opcije za testiranje vrednosti string promenljivih:

```
s1 = s2      String s1 equals string s2
s1 != s2     String s1 does not equal string s2
-z s1        String s1 has zero length
-n s1        String s1 has nonzero length
```

- Opcije za testiranje vrednosti celobrojnih promenljivih:

<code>a -eq b</code>	Integers <i>a</i> and <i>b</i> are equal
<code>a -ne b</code>	Integers <i>a</i> and <i>b</i> are not equal
<code>a -gt b</code>	Integer <i>a</i> is greater than integer <i>b</i>
<code>a -ge b</code>	Integer <i>a</i> is greater than or equal to integer <i>b</i>
<code>a -lt b</code>	Integer <i>a</i> is less than integer <i>b</i>
<code>a -le b</code>	Integer <i>a</i> is less than or equal to integer <i>b</i>

- Pri testiranju se mogu i praviti i složeniji logički izrazi, pomoću logičkih operacija konjunkcije, disjunkcije i negacije:

<code>t1 -a t2</code>	And: Both tests <i>t1</i> and <i>t2</i> are true
<code>t1 -o t2</code>	Or: Either test <i>t1</i> or <i>t2</i> is true
<code>! your_test</code>	Negate the test (i.e., <i>your_test</i> is false)
<code>\(your_test \)</code>	Parentheses are used for grouping, as in algebra

- Komanda **test** ima neobičan alias, levu uglastu zagradu `[`, koja se po konvenciji uvek koristi za skraćeni/čitkiji zapis prilikom kreiranja kondicionala; u tom slučaju, korisnik mora da kao poslednji argument navede desnu uglastu zagradu `]`, kako bi označio gde se komanda (to jest ispitivanje) završava

- Naredna dva primera su identična kao dva ranije prikazana:

```
→ [ 10 -lt 5 ]
→ echo $?
1
→ [ -n "hello" ]
→ echo $?
0
```

- Treba imati u vidu da je [jedna uobičajena Linux komanda, kod koje svi argumenti koji slede, pa i poslednji, to jest], moraju međusobno biti odvojeni razmacima; ako se to ne ispuni, generisaće se greška

- Primer:

```
→ [ 5 -lt 4]           No space between 4 and ]
bash: [: missing '']
```

Pre desne uglaste zagrade izostavljen je razmak, pa se 4] tretira kao poslednji argument; naravno, takav format argumenta nije ni validan, i ispisuje se poruka da nedostaje završna desna uglasta zagrada

29.5 Kondicional

- Kondicional ("pitalica", eng. conditional) se kreira pomoću komande **if**; predstavlja izbor jedne od dve alternative, a na osnovu toga da li je zadati uslov ispunjen ili ne; za označavanje kraja kondicionala piše se komanda (prikladnog imena) **fi**

- Najprostiji oblik kondicionala je **if-then-fi**, čiji je opšti oblik:

```
if condition           If return code is 0
then
    body
fi
```

- Na mestu *condition* se piše uslov kreiran pomoću komande **test**, uz zapis sa uglastim zgradama (kao što je pokazano u prethodnoj sekciji); telo (eng. body) kondicionala se ovde indentiralo samo zbog preglednosti, inače nije potrebno; kraj kondicionala se označava pomoću **fi**

- U ovom kondicionalu, znači (slično kao i u ostalim programskim jezicima):

>> ako je uslov ispunjen (preciznije: ako je povratni kôd komande **test** jednak 0), telo će se izvršiti

>> ako uslov nije ispunjen (preciznije: ako je povratni kôd komande **test** jednak 1), neće se desiti ništa

U oba slučaja, prelazi se zatim na prvu narednu liniju koda (koja sledi posle **fi**)

- Primer skripta sa jednostavnim kondicionalom:

```
→ cat script-if
#!/bin/bash
FILE = myfile
if [ -s $FILE ]
then
echo "File $FILE exists."
fi
```

Ako fajl myfile (naziv mu je dodeljen varijabli **FILE**) postoji, ispisuje se odgovarajuća poruka; koristi se ranije uvedena opcija **-s** za testiranje fajlova

- Generalniji oblik kondicionala je **if-then-else-fi**, čiji je opšti oblik:

```
if condition
then
    body1
else
    body2
fi
```

- Primer skripta:

```
→ cat script-else
#!/bin/bash
if [ `whoami` = "root" ]
then
    echo "You are the superuser"
else
    echo "You are a mere mortal"
fi
```

Ovde se koristi ranije objašnjena komanda **whoami** u kombinaciji sa backquote

- Ako ovaj skript pokrene obični korisnik, ispisaće se prvi tekst:

```
→ ./script-else  
You are a mere mortal
```

a ako ga pokrene superkorisnik ispisaće se drugi:

```
→ sudo ./script-else  
Password: *****  
You are the superuser
```

- Najgeneralniji oblik kondicionala je **if-then-elif-then-...-else-fi**, gde se **elif-then** može pojaviti proizvoljan broj puta; opšti oblik je:

```
if condition1  
then  
    body1  
elif condition2  
then  
    body2  
elif ...  
...  
else  
    bodyN  
fi
```

- Primer:

```
→ cat script-elif  
#!/bin/bash  
echo "How old are you?"  
read AGE  
if [ "$AGE" -le 21 ]  
then  
    echo "You are too young. Leave now!"  
elif [ "$AGE" -ge 60 ]  
then
```

```

        echo "You are too old! Go away!"
    else
        echo "You are just the right age. Welcome!"
    fi

```

Zavisno od vrednosti unete varijable **AGE**, ispisuje se jedna od tri odgovarajuće poruke

- Uslov se može zadati na još jedan način, a to je pomoću komande **case** (neće se obrađivati)

29.6 Petlja

- Petlja (eng. loop) je takođe osnovni programerski koncept; prvi tip je **while** petlja: ona ponavlja datu sekvencu komandi (telo) sve dok je zadati uslov **ispunjen**; uslov se zadaje isto kao kod kondicionala

- Opšti oblik je:

```

while condition      while return code = 0
do
    body
done

```

Celokupna sintaksa je, znači, **while-do-done**; telo se ponavlja sve dok je povratni kôd komande **test** jednak 0

- Primer:

```

→ cat script-while
#!/bin/bash
i=0
while [ $i -lt 3 ]
do
    echo "$i"
    i=`expr $i + 1`

```

```
done
→ ./script-while
0
1
2
```

- Drugi tip je **until** petlja, kod koje se telo ponavlja sve dok dati uslov **ne postane** ispunjen; opšti oblik:

```
until condition           while return code = 1
do
    body
done
```

Sintaksa je znači **until-do-done**; telo se ponavlja sve dok je povratni kôd komande **test** jednak 1

- Primer koji generiše isti izlaz kao prethodni:

```
→ cat script-until
#!/bin/bash
i=0
until [ $i -ge 3 ]
do
    echo "$i"
    i=`expr $i + 1`
done
→ ./script-until
0
1
2
```

- Prilikom generisanja i **while** i **until** petlje, korisnik treba da osigura da nikad ne dođe do nastanka beskonačne petlje; da bi se izbegla, telo petlje mora ima uticaj na uslov, tako da posle određenog broja ponavljanja povratni kôd dobije vrednost koja znači kraj petlje

- Konačno, **for** petlja iterira kroz vrednosti neke liste, i za svaku odrađuje telo; opšti oblik je:

```
for variable in list  
do  
    body  
done
```

Kompletna sintaksa je znači **for-do-done**

- Primer:

```
→ cat script-for  
#!/bin/bash  
for NAME in Tom Jane Harry  
do  
    echo "$NAME is my friend"  
done  
→ ./script-for  
Tom is my friend  
Jane is my friend  
Harry is my friend
```

Lista se ovde kreira tako što se prosto svi njeni članovi (u ovom slučaju, tri imena) napišu jedan za drugim

- Ova petlja je na primer pogodna za procesiranje liste fajlova; primer: ispisivanje liste fajlova sa ekstenzijom docx u radnom direktorijumu:

```
→ cat script-for2  
#!/bin/bash  
for FILE in *.docx  
do  
    echo "$FILE is a Word file"  
done  
→ ./script-for2  
letter.docx is a Word file
```

Ovde listu predstavljaju svi fajlovi sa ekstenzijom docx; kroz nju se iterira pomoću promenljive petlje **FILE**

- Može se koristiti i ranije pomenuta komanda **seq** da generiše listu celih brojeva, kroz koju će **for** petlja iterirati; primer:

```
→ cat script-seq
#!/bin/bash
for NUM in $(seq 1 3)
do
    echo "iteration $NUM"
done
→ ./script-seq
iteration 1
iteration 2
iteration 3
```

29.7 Zadavanje argumenata sa komandne linije

- Šel skript može da prihvati argumente (kao i opcije) sa komandne linije, kao i svaka druga Linux komanda; u okviru skripta, na te argumente se pravi referenca pomoću oznaka **\$1**, **\$2**, **\$3** i tako redom

- Primer:

```
→ cat script-args
#!/bin/bash
echo "My name is $1 and I come from $2"

→ ./script-args Johnson Wisconsin
My name is Johnson and I come from Wisconsin
→ ./script-args Bob
My name is Bob and I come from
```

Pri pozivu ovog skripta treba zadati dva argumenta, koja će se u okviru komande **echo** ispisati na predviđenim mestima (ako se drugi ne zada, neće se ispisati)

- Broj argumenata koliko je skriptu zadato sa komandne linije čuva se u specijalnoj varijabli `$#`; ova varijabla može da se koristi u okviru skripta; postoji i specijalna varijabla `$0` koja sadrži naziv sâmog šel skripta: koristi se često prilikom ispisivanja poruka o grešci

- Primer:

```
→ cat script-args2
#!/bin/bash
if [ $# -lt 2 ]
then
    echo "$0 error: you must supply two arguments"
else
    echo "My name is $1 and I come from $2"
fi
```

Ako se zada manje od dva argumenta, ispisuje se poruka o grešci

- Primer izvršenja ovog skripta kad se zada samo jedan argument:

```
→ ./script-args2 Barbara
./script-args2 error: you must supply two arguments
```

- Sa komandne linije se skriptu može zadati i lista argumenata; ona se čuva u specijalnoj varijabli `$@`; na taj način se u okviru skripta može kroz tu listu iterirati pomoću **for** petlje

- Primer:

```
→ cat script-args3
#!/bin/bash
for ARG in $@
do
    echo "I found the argument $ARG"
done
→ ./script-args3 One Two Three
I found the argument One
I found the argument Two
I found the argument Three
```

29.8 Izlazak pomoću povratnog koda

- Komanda **exit** trenutno prekida izvršenje skripta, i šelu vraća povratni kôd n koji joj je zadat; sintaksa je znači:

exit n

- Po konvenciji, kao što je ranije objašnjeno, vraća se $n=0$ ako nema greške, i $1 \leq n \leq 255$ ako je ima, a ta sama vrednost n predstavlja kôd greške; ako ne dođe do izvršenja nijedne **exit** komande, povratni kôd je jednak 0

- Primer:

```
→ cat script-exit
#!/bin/bash
if [ $# -lt 2 ]
then
    echo "$0 error: you must supply two arguments"
    exit 11
else
    echo "My name is $1 and I come from $2"
fi
exit 0
```

Poslednja linija je opciona, jer bi i bez nje vrednost. povratnog koga bila 0

- U primeru, ako se zada manje od dva argumenta, ispisuje se odgovarajući tekst, i vraća povratni kôd 11: ovaj broj je odabran u ranije predloženom opsegu, da se izbegnu standardni kodovi; tako da formalno povratni kôd 11 simbolizuje grešku "nedovoljan broj zadatah argumenata"

- Izvršenje:

```
→ ./script-exit Bob
./script-exit error: you must supply two arguments
→ echo $?
11
```

Da je potrebno, mogao je na primer kôd 11 da simbolizuje grešku "uneto 0 argumenata", 12 "unet 1 argument", 13 "uneto 3 argumenta" i slično